

GUIDA COMPLETA ALL'INTELLIGENZA ARTIFICIALE

Dal Codice all'Esperto — Edizione 2026

Per sviluppatori junior che vogliono padroneggiare l'IA dalla teoria alla produzione

VOLUME 2 — COSTRUIRE

Deep Learning, LLM, Agenti e Infrastruttura

Introduzione al Volume → Cosa cambia rispetto al ML classico → Setup PyTorch e ambiente GPU → Come strutturare un progetto di deep learning

Cap. 8 — Deep Learning con PyTorch → Tensor: l'array che capisce la GPU → Autograd: come PyTorch calcola i gradienti automaticamente → Dataset, DataLoader e training loop in PyTorch → Il modulo nn.Module: costruire reti come LEGO → Salvare, caricare e condividere un modello → *Esercizio pratico: MNIST da zero con PyTorch*

Cap. 9 — Reti Convoluzionali (CNN) → Perché le CNN funzionano sulle immagini: località e invarianza → Convoluzione, pooling, padding, stride — spiegati visivamente → Architetture: LeNet → AlexNet → ResNet (skip connections intuitive) → Transfer learning: riusare un modello pre-addestrato per il tuo task → Data augmentation per immagini → *Esercizio pratico: classificatore di immagini con ResNet pre-addestrato*

Cap. 10 — Sequenze: RNN, LSTM e il problema del contesto lungo → Perché i dati sequenziali richiedono architetture diverse → RNN: memoria a breve termine e vanishing gradient → LSTM e GRU: porte che controllano il flusso di informazioni → Perché le RNN sono state superate (e quando usarle ancora) → *Esercizio pratico: predizione di serie temporale con LSTM*

Cap. 11 — Il Transformer e il Meccanismo di Attenzione → L'idea chiave: ogni parola "guarda" tutte le altre contemporaneamente → Query, Key, Value: l'analogia con una biblioteca → Self-Attention e Multi-Head Attention: specializzazione parallela → Positional Encoding: come il Transformer sa l'ordine delle parole → Architettura completa: encoder, decoder, decoder-only → Perché il Transformer ha vinto su tutto → *Esercizio pratico: implementa un mini-Transformer da zero*

Cap. 12 — Large Language Models (LLM) → Tokenizzazione BPE: come il testo diventa numeri → Architettura interna GPT/LLaMA step by step → Pre-training: cosa impara un modello su trilioni di token → SFT, RLHF, DPO: da modello linguistico ad assistente utile → Scaling laws e capacità emergenti: perché i modelli grandi sorprendono → Panoramica modelli 2026: open source vs. closed, quando usare quale → *Esercizio pratico: usa l'API Anthropic/OpenAI per costruire il tuo chatbot*

Cap. 13 — Prompt Engineering Avanzato → Zero-shot, few-shot, chain-of-thought: la gerarchia delle tecniche → System prompt: come istruire il modello in modo robusto → Structured output: ottenere JSON garantito → Tree of Thoughts e Self-Consistency per task complessi → Come testare e misurare la qualità dei prompt → *Esercizio pratico: costruisci un evaluator automatico di prompt*

Cap. 14 — RAG: Dare Memoria Esterna agli LLM → Perché il RAG risolve il problema della conoscenza privata e obsoleta → Pipeline completa: chunking → embedding → vector DB → retrieval → generazione → Embedding e similarità coseno: come il modello "capisce" la semantica → Vector database: ChromaDB in locale, Pinecone in produzione → Tecniche avanzate: re-ranking, hybrid search, HyDE → *Esercizio pratico: sistema Q&A su documenti PDF tuoi*

Cap. 15 — Agenti IA → Cos'è un agente: dal LLM passivo al sistema autonomo → Tool use e function calling: come l'agente usa gli strumenti → Il ciclo ReAct: Ragionamento → Azione → Osservazione → Memoria degli agenti: in-context, esterna, episodica → Pattern architetturali: ReAct, Plan&Execute, Multi-Agent → Insidie degli agenti: loop infiniti, errori a cascata, costi esplosivi → *Esercizio pratico: agente che cerca info sul web e risponde con fonti*

Cap. 16 — MCP – Model Context Protocol → Il problema che MCP risolve: integrazioni custom vs. standard universale → Architettura Host/Client/Server spiegata con diagrammi → Tools, Resources, Prompts: i tre mattoni di un server MCP → Configurare MCP su Claude Desktop → Scrivere il tuo primo MCP Server in Python da zero → Server MCP esistenti e come sceglierli → *Esercizio pratico: MCP Server che espone dati da un tuo file CSV*

Cap. 17 — Hardware e CUDA → Perché la GPU è indispensabile: parallelismo vs. velocità → CUDA, CUDA Core, Tensor Core: cosa fa ognuno → Gerarchia di memoria GPU: dove finiscono i tuoi dati → Quantizzazione: FP32 → BF16 → INT8 → INT4 senza perdere qualità → Come scegliere l'hardware: consumer GPU, cloud, locale → Monitorare memoria e utilizzo GPU con strumenti pratici → *Esercizio pratico: profila un modello e ottimizza l'uso di VRAM*

Progetto di Fine Volume 2 → Chatbot RAG con agente integrato: risponde a domande sui tuoi documenti, cerca sul web quando necessario, usa un MCP server custom

APPENDICI (in tutti i volumi)

Appendice A — Glossario completo (ogni termine tecnico spiegato in italiano semplice)

Appendice B — Cheatsheet Python per l'IA (NumPy, pandas, PyTorch, HuggingFace) **Appendice**

C — Risorse aggiornate: corsi, paper fondamentali, community **Appendice D** — Roadmap di studio consigliata con milestone verificabili **Appendice E** — Soluzioni commentate degli esercizi pratici

VOLUME 2 — COSTRUIRE

Deep Learning, LLM, Agenti e Infrastruttura

Introduzione al Volume

"Nel Volume 1 hai imparato a far imparare una macchina dai dati. In questo volume imparerai a costruire le macchine che imparano."

Cosa cambia rispetto al Machine Learning classico

Nel Volume 1 hai lavorato con alberi decisionali, regressioni, Random Forest, XGBoost. Strumenti potenti, ancora ampiamente usati in produzione — specialmente su dati tabellari. Ma tutti condividono un limite: richiedono che tu, lo sviluppatore, definisca manualmente le *feature*, cioè le caratteristiche rilevanti dei tuoi dati.

Per classificare email spam hai scelto tu di usare lunghezza dell'email, numero di link, parole in maiuscolo. Funziona perché le regole dello spam sono abbastanza semplici da intuire. Ma prova a fare la stessa cosa con una foto di un gatto. Quali feature sceglieresti? Il colore medio dei pixel? La posizione delle orecchie? Questi approcci hanno fallito per decenni proprio perché le feature rilevanti per immagini, audio e testo sono troppo complesse e numerose per essere definite a mano.

Il **deep learning** ribalta il problema: invece di costruire feature manualmente, lasci che la rete neurale le impari da sola dai dati grezzi. Le foto entrano come pixel, il testo entra come caratteri o parole — e la rete decide da sola cosa guardare.

Questo cambia il modo di lavorare in quattro modi concreti:

Input grezzo invece di feature ingegnerizzate. Passi i pixel dell'immagine direttamente, non una lista di caratteristiche pensate da te. La rete costruisce le proprie rappresentazioni interne.

Architetture specializzate per tipo di dato. CNN per immagini, Transformer per testo, RNN per sequenze temporali. Ogni struttura dati ha la sua architettura ottimale — lo vedremo capitolo per capitolo.

GPU obbligatoria per reti grandi. Addestrare una rete profonda su CPU è possibile solo per reti piccole e dataset giocattolo. Dal Capitolo 9 in poi userai sempre la GPU.

Più dati, più parametri, più tempo. I modelli di deep learning hanno milioni o miliardi di parametri. Hanno bisogno di molti esempi per generalizzare — e di ore o giorni di training per reti grandi.

Setup PyTorch e ambiente GPU

Per questo volume useremo **Google Colab** come ambiente principale: ha tutto il necessario già installato, GPU gratuita inclusa.

Passo 1 — Apri un nuovo notebook Vai su colab.research.google.com e crea un nuovo notebook.

Passo 2 — Attiva la GPU Menu in alto → Runtime → Change runtime type → Hardware accelerator → T4 GPU → Salva.

Passo 3 — Verifica che tutto funzioni

```
import torch
import torchvision

print(f"PyTorch: {torch.__version__}")
print(f"Torchvision: {torchvision.__version__}")
print(f"GPU disponibile: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"Modello GPU: {torch.cuda.get_device_name(0)}")
    vram = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f"VRAM disponibile: {vram:.1f} GB")
```

Se vedi `GPU disponibile: True` e il nome della GPU (tipicamente `Tesla T4`), sei pronto.

Nota su Colab gratuito: la GPU T4 è disponibile con un limite di ore giornaliero. Se il runtime si disconnette, i file locali vengono persi. Salva sempre il modello su Google Drive o scaricalo prima di chiudere la sessione.

Installazione locale (opzionale) Se hai una GPU NVIDIA e vuoi lavorare in locale:

```
# Crea ambiente virtuale
python -m venv venv-dl
source venv-dl/bin/activate    # Windows: venv-dl\Scripts\activate

# Installa PyTorch con supporto CUDA
# Verifica la versione CUDA corretta su https://pytorch.org/get-started/locally/
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

# Librerie aggiuntive usate in questo volume
pip install transformers datasets accelerate matplotlib numpy
```

Come strutturare un progetto di deep learning

Un progetto ben strutturato segue sempre la stessa organizzazione di base. Conoscerla subito ti evita di riscrivere tutto a metà lavoro.

```
mio_progetto/
├── data/
│   ├── raw/           # dati originali – non modificare mai questi file
│   └── processed/      # dati pre-processati pronti per il training
├── notebooks/         # esplorazione e sperimentazione su Colab
│   ├── 01_esplorazione.ipynb
│   └── 02_training.ipynb
├── src/               # codice riutilizzabile (non notebook)
│   ├── dataset.py     # classi Dataset personalizzate
│   ├── model.py       # architettura della rete
│   ├── train.py       # training loop
│   └── evaluate.py     # metriche e valutazione
├── checkpoints/       # pesi del modello salvati durante il training
├── results/           # grafici, metriche, report
└── requirements.txt    # lista delle librerie necessarie
```

Per ora tutto il codice di questo volume andrà in un singolo notebook Colab. Riprenderemo questa struttura nel Volume 3, quando passeremo dal prototipo al deploy in produzione.

Capitolo 8

Deep Learning con PyTorch

"NumPy ti fa fare matematica. PyTorch ti fa addestrare reti neurali. La differenza sembra piccola finché non devi calcolare i gradienti di un milione di parametri."

8.1 Perché serve uno strumento nuovo?

Nel Volume 1 hai usato NumPy per manipolare numeri e scikit-learn per addestrare i modelli. Funzionava bene. Ma prova a immaginare di voler addestrare una rete neurale con un milione di parametri — come quelle che riconoscono le immagini o generano testo.

Il problema non è la matematica in sé. Il problema è che durante il training devi calcolare come ciascuno di quei milioni di parametri ha contribuito all'errore finale, e poi aggiornarli tutti. Farlo a mano — o con NumPy, che non è pensato per questo — è impraticabile.

PyTorch nasce per risolvere esattamente questo. Aggiunge due superpoteri a un normale array di numeri.

Superpower 1 — la GPU. Una GPU moderna ha migliaia di piccoli processori che lavorano in parallelo. NumPy non sa come usarli: calcola su CPU, un'operazione alla volta. PyTorch è progettato per sfruttare la GPU fin dall'inizio. La stessa operazione che su CPU richiede ore, su GPU richiede minuti.

Superpower 2 — autograd. PyTorch tiene traccia di ogni operazione che fai sui tuoi numeri. Quando gli dici "dammi i gradienti", risale tutta la catena di operazioni al contrario e li calcola automaticamente — per tutti i parametri, in un colpo solo. Questa tecnologia si chiama **autograd**, ed è il motore invisibile che permette il training di qualsiasi rete neurale moderna.

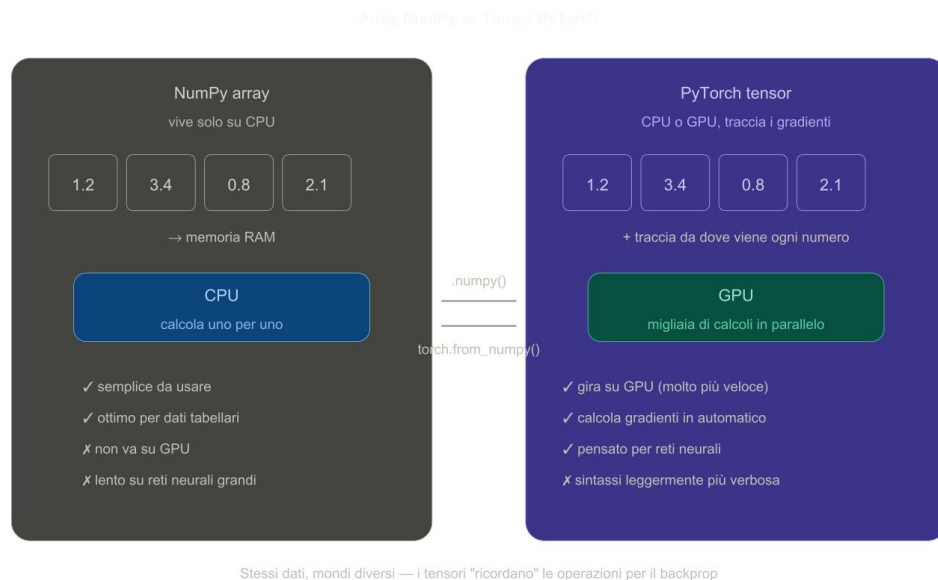


Figura 8.1 — Array NumPy vs Tensor PyTorch: stessi dati, capacità molto diverse — i tensori "ricordano" le operazioni per il backprop

Analogia: NumPy è come una calcolatrice — ottima per il risultato, ma appena premi = dimentica tutto. PyTorch è come un foglio di carta con tutti i passaggi scritti: quando il risultato finale è sbagliato, puoi risalire passo per passo e capire dove hai sbagliato. Nel training delle reti neurali, questa memoria è tutto.

8.2 I Tensor: l'array che capisce la GPU

Un **tensor** è l'unità fondamentale di PyTorch. Concettualmente è identico a un array NumPy — una struttura che contiene numeri organizzati in dimensioni — ma con due superpoteri: gira su GPU e tiene traccia delle operazioni per il calcolo automatico dei gradienti.

Apri un nuovo notebook Colab con GPU T4 attiva e inizia:

```
import torch

print(torch.__version__)
print("GPU disponibile:", torch.cuda.is_available())
```

Creare tensor

```
import torch

# Da una lista Python — il modo più diretto
t1 = torch.tensor([1.0, 2.0, 3.0, 4.0])
print(t1)          # tensor([1., 2., 3., 4.])
print(t1.shape)    # torch.Size([4]) — un vettore di 4 elementi
print(t1.dtype)    # torch.float32 — tipo di dato default

# Tensor 2D (matrice) — come una tabella con 2 righe e 3 colonne
t2 = torch.tensor([[1.0, 2.0, 3.0],
                   [4.0, 5.0, 6.0]])
```

```

print(t2.shape)      # torch.Size([2, 3])

# Tensor 3D – come un batch di immagini: (batch, altezza, larghezza)
t3 = torch.zeros(8, 28, 28)  # 8 immagini 28×28, tutti zeri
print(t3.shape)      # torch.Size([8, 28, 28])

# Inizializzazioni utili
torch.zeros(3, 3)      # tutti zeri
torch.ones(3, 3)       # tutti uno
torch.rand(3, 3)       # valori casuali uniformi tra 0 e 1
torch.randn(3, 3)      # valori casuali con distribuzione normale (media=0, std=1)

```

Spostare un tensor su GPU

```

# Controlla se la GPU è disponibile, poi scegli il dispositivo
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Uso:", device)

# Sposta un tensor sulla GPU
t_cpu = torch.randn(1000, 1000)  # creato su CPU
t_gpu = t_cpu.to(device)         # spostato su GPU

# Ora le operazioni su t_gpu girano sulla GPU
risultato = t_gpu @ t_gpu.T      # moltiplicazione matriciale su GPU
print(risultato.shape)          # torch.Size([1000, 1000])

```

Nota pratica: tutti i tensori coinvolti in un'operazione devono stare sullo stesso dispositivo. Se provi a sommare un tensor CPU con uno GPU, PyTorch ti darà un errore esplicito. È uno degli errori più comuni per i principianti.

Operazioni sui tensor

```

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

# Operazioni elemento per elemento
print(a + b)      # tensor([5., 7., 9.])
print(a * b)      # tensor([4., 10., 18.])
print(a ** 2)     # tensor([1., 4., 9.])

# Riduzioni
print(a.sum())    # tensor(6.)
print(a.mean())   # tensor(2.)
print(a.max())    # tensor(3.)

# Moltiplicazione matriciale (fondamentale nelle reti neurali)
m1 = torch.randn(3, 4)  # matrice 3×4
m2 = torch.randn(4, 5)  # matrice 4×5
m3 = m1 @ m2            # risultato: matrice 3×5
print(m3.shape)         # torch.Size([3, 5])

# Reshape: cambia la forma senza cambiare i dati
t = torch.arange(12, dtype=torch.float32)  # [0,1,2,...,11]
print(t.shape)                             # torch.Size([12])
t_2d = t.reshape(3, 4)                    # stessi 12 numeri, ora 3 righe × 4 colonne
print(t_2d.shape)                         # torch.Size([3, 4])

```

Convertire tra NumPy e PyTorch

```
import numpy as np

# Da NumPy a PyTorch
array_np = np.array([1.0, 2.0, 3.0])
tensor_pt = torch.from_numpy(array_np)
print(tensor_pt)          # tensor([1., 2., 3.], dtype=torch.float64)

# Da PyTorch a NumPy (il tensor deve essere su CPU)
tensor_pt2 = torch.tensor([4.0, 5.0, 6.0])
array_np2 = tensor_pt2.numpy()
print(array_np2)          # [4. 5. 6.]
```

8.3 Autograd: PyTorch che calcola i gradienti al posto tuo

Questa è la tecnologia che rende possibile addestrare reti neurali con milioni di parametri senza impazzire. Vediamo come funziona partendo da un esempio minimo.

Quando crei un tensor con `requires_grad=True`, stai dicendo a PyTorch: "tieni traccia di tutto quello che faccio con questo numero, perché dopo ti chiederò il gradiente".

```
# requires_grad=True: "registra ogni operazione su questo tensor"
x = torch.tensor(3.0, requires_grad=True)

# Definiamo y = x^2 + 2x + 1
y = x**2 + 2*x + 1
print(y)
# tensor(16., grad_fn=<AddBackward0>)
# ↑ grad_fn ci dice che y è stato prodotto da una somma
# PyTorch ha già registrato tutta la storia delle operazioni

# Chiediamo il gradiente: quanto cambia y se cambia x?
y.backward()

# La derivata di y rispetto a x è: dy/dx = 2x + 2
# Per x=3: 2(3) + 2 = 8
print(x.grad)  # tensor(8.) ← PyTorch l'ha calcolato da solo
```

Internamente PyTorch costruisce un **grafo computazionale** — una mappa di tutte le operazioni fatte da `x` fino a `y`. Quando chiami `.backward()`, percorre questo grafo al contrario e calcola la derivata in ogni punto. È esattamente la **backpropagation** di cui hai letto nel Capitolo 4, eseguita automaticamente.

Analogia: immagina una catena di domino — ogni tessera fa cadere quella successiva. `x` è la prima tessera, `y` è l'ultima. PyTorch ha visto come hai posizionato ogni tessera mentre costruivi l'espressione. Quando chiami `.backward()`, percorre la catena al contrario e calcola quanto ogni tessera ha contribuito al crollo finale.

In pratica non chiamerai mai `.backward()` su una variabile creata da te: lo chiamerai sulla **loss**, il numero che misura l'errore del modello. PyTorch calcolerà automaticamente i gradienti di tutti i milioni di parametri della rete — in un colpo solo.

8.4 Dataset e DataLoader: gestire i dati per il training

Prima di costruire una rete neurale, dobbiamo capire come PyTorch gestisce i dati. Due classi sono fondamentali.

Dataset: una classe che sa come caricare un singolo esempio dai tuoi dati. **DataLoader**: una classe che usa il Dataset per fornire i dati in batch, in modo efficiente.

```
from torch.utils.data import Dataset, DataLoader
import torch
import numpy as np

# Creiamo un Dataset personalizzato - il pattern che userai sempre
class MioDataset(Dataset):

    def __init__(self, X, y):
        # Costruttore: salva i dati come tensor
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        # Quanti esempi ha il dataset?
        return len(self.X)

    def __getitem__(self, idx):
        # Restituisce il singolo esempio all'indice idx
        return self.X[idx], self.y[idx]

# Dati di esempio: 100 punti con 5 feature ciascuno
X = np.random.randn(100, 5)  # 100 esempi, 5 feature
y = np.random.randn(100)     # 100 etichette

dataset = MioDataset(X, y)
print(f"Esempi totali: {len(dataset)}")  # 100

# DataLoader: batch da 16, mescola i dati ad ogni epoca
loader = DataLoader(dataset, batch_size=16, shuffle=True)

for batch_X, batch_y in loader:
    print(f"Shape del batch: {batch_X.shape}")  # torch.Size([16, 5])
    break
```

Perché `shuffle=True`? Se il dataset è ordinato (es. prima tutte le email spam, poi tutte le normali), i batch non sarebbero rappresentativi. Mescolare i dati garantisce che ogni batch contenga un campione vario, rendendo il training più stabile.

PyTorch include già dataset standard pronti all'uso:

```
from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

```

])

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
print(f"Esempi di training: {len(train_dataset)}")    # 60000

```

8.5 nn.Module: costruire reti neurali come blocchi LEGO

In PyTorch ogni rete neurale è una classe Python che eredita da `nn.Module`. Devi definire due metodi:

- `__init__`: dove dichiarare i layer della rete
- `forward`: dove descrivi come i dati fluiscono attraverso i layer

```

import torch.nn as nn

class MiaRete(nn.Module):

    def __init__(self):
        super().__init__()
        # nn.Linear(in_features, out_features): strato completamente connesso
        # ogni neurone di input è connesso a ogni neurone di output
        self.layer1 = nn.Linear(784, 256)    # 784 pixel → 256 neuroni
        self.layer2 = nn.Linear(256, 128)
        self.layer3 = nn.Linear(128, 10)    # 10 classi (cifre 0-9)

        self.relu = nn.ReLU()
        # Dropout: spegne casualmente il 20% dei neuroni durante il training
        # serve a evitare l'overfitting
        self.dropout = nn.Dropout(p=0.2)

    def forward(self, x):
        # x ha shape [batch_size, 784]
        x = self.relu(self.layer1(x))
        x = self.dropout(x)
        x = self.relu(self.layer2(x))
        x = self.layer3(x)    # nessuna attivazione finale: la loss la gestisce
        return x

device = "cuda" if torch.cuda.is_available() else "cpu"
modello = MiaRete().to(device)

totale = sum(p.numel() for p in modello.parameters())
print(f"Parametri totali: {totale:,}")    # 235,146

```

8.6 Il Training Loop: i 5 passi che si ripetono

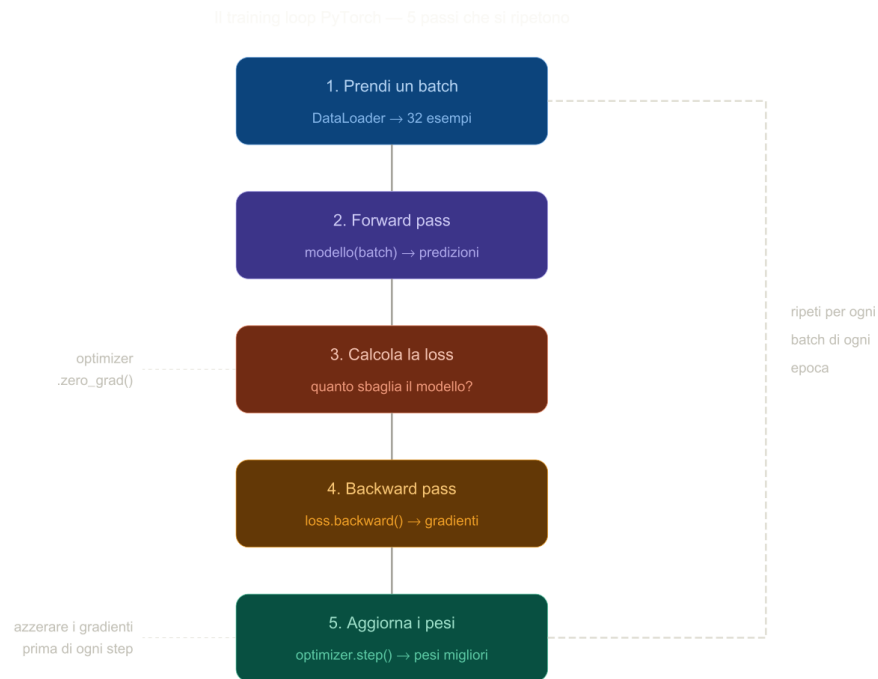


Figura 8.2 — Il training loop PyTorch — il ciclo che si ripete migliaia di volte durante l'addestramento.

Il training loop è il cuore di ogni progetto di deep learning. Come mostra il diagramma (vedi figura 8.2), si ripete per ogni batch di ogni epoca.

```
# Funzione di loss: CrossEntropyLoss per classificazione multi-classe
# calcola Softmax + log + media internamente
criterio = nn.CrossEntropyLoss()

# Ottimizzatore Adam: il punto di partenza consigliato per quasi tutto
# lr = learning rate: quanto grandi sono i passi di aggiornamento
ottimizzatore = torch.optim.Adam(modello.parameters(), lr=0.001)

EPOCHHE = 10

for epoca in range(EPOCHHE):

    modello.train() # modalità training: attiva dropout
    loss_totale = 0

    for batch_X, batch_y in train_loader:
        batch_X = batch_X.to(device)
        batch_y = batch_y.to(device)

        # Appiattisce le immagini: [64, 1, 28, 28] → [64, 784]
        batch_X = batch_X.view(batch_X.size(0), -1)

        # PASSO 1: azzerare i gradienti — senza questo si accumulano e il training
        # diverge
        ottimizzatore.zero_grad()

        # PASSO 2: forward pass — il modello produce le predizioni
```

```

predizioni = modello(batch_X)      # shape: [64, 10]

# PASSO 3: calcola la loss - misura l'errore
loss = criterio(predizioni, batch_y)

# PASSO 4: backward pass - calcola i gradienti di tutti i parametri
loss.backward()

# PASSO 5: aggiorna i pesi nella direzione che riduce la loss
ottimizzatore.step()

loss_totale += loss.item()      # .item() converte da tensor a numero Python

print(f"Epoca {epoca+1}/{EPOCHE} - Loss: {loss_totale/len(train_loader):.4f}")

```

Perché `zero_grad()` prima di ogni batch? PyTorch accumula i gradienti per default. Se non azzeri, i gradienti del secondo batch si sommano a quelli del primo, del terzo al secondo, e così via — il risultato è un training instabile e scorretto.

8.7 Valutare il modello sul test set

Durante il training il modello migliora sul training set. Ma come sappiamo se sta davvero *generalizzando*? Valutiamo su dati che non ha mai visto.

```

test_dataset = datasets.MNIST(root="./data", train=False, download=True,
transform=transform)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

def valuta(modello, loader, device):
    modello.eval()      # disattiva dropout per predizioni deterministiche
    corretti = 0
    totale = 0

    with torch.no_grad():      # disabilita autograd: più veloce, usa meno memoria
        for batch_X, batch_y in loader:
            batch_X = batch_X.to(device).view(batch_X.size(0), -1)
            batch_y = batch_y.to(device)

            predizioni = modello(batch_X)
            classi_pred = predizioni.argmax(dim=1)      # indice della classe più
probabile
            corretti += (classi_pred == batch_y).sum().item()
            totale += batch_y.size(0)

    return corretti / totale

accuracy = valuta(modello, test_loader, device)
print(f"Accuracy sul test set: {accuracy*100:.2f}%")

```

8.8 Salvare e caricare un modello

```
# SALVA - solo i pesi, non l'intera struttura (metodo raccomandato)
torch.save(modello.state_dict(), "modello_mnist.pth")

# CARICA - ricrea prima l'architettura, poi carica i pesi
modello_caricato = MiaRete().to(device)
modello_caricato.load_state_dict(
    torch.load("modello_mnist.pth", map_location=device)
)
modello_caricato.eval()

accuracy_caricato = valuta(modello_caricato, test_loader, device)
print(f"Accuracy del modello caricato: {accuracy_caricato*100:.2f}%")
# Deve essere identica a prima - conferma che il salvataggio ha funzionato
```

`state_dict()` è un dizionario Python che contiene tutti i parametri del modello. Salvare solo il `state_dict` è la pratica raccomandata perché è portatile: funziona anche se cambi versione di PyTorch o la struttura del progetto.

8.9 Esercizio guidato completo — MNIST con PyTorch

Mettiamo tutto insieme. Questo notebook è il tuo primo progetto di deep learning da zero.

```
# =====
# MNIST con PyTorch - classificatore di cifre scritte a mano
# Esegui su Google Colab con GPU T4 attiva
# =====

import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Dispositivo: {device}")

# --- Dati ---
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_data = datasets.MNIST("./data", train=True, download=True,
transform=transform)
test_data = datasets.MNIST("./data", train=False, download=True,
transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

# --- Architettura ---
class Classificatore(nn.Module):
    def __init__(self):
        super().__init__()
        # nn.Sequential incatena i layer in sequenza - più compatto
```

```

        self.rete = nn.Sequential(
            nn.Linear(784, 512), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(512, 256), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(256, 10)
        )
    def forward(self, x):
        return self.rete(x.view(x.size(0), -1))

modello = Classificatore().to(device)
print(f"Parametri: {sum(p.numel() for p in modello.parameters()):,}")

# --- Training ---
criterio = nn.CrossEntropyLoss()
ottimizzatore = torch.optim.Adam(modello.parameters(), lr=0.001)
storia_loss, storia_acc = [], []

for epoca in range(10):
    modello.train()
    loss_ep = 0
    for X, y in train_loader:
        X, y = X.to(device), y.to(device)
        ottimizzatore.zero_grad()
        loss = criterio(modello(X), y)
        loss.backward()
        ottimizzatore.step()
        loss_ep += loss.item()

    modello.eval()
    corretti = 0
    with torch.no_grad():
        for X, y in test_loader:
            X, y = X.to(device), y.to(device)
            corretti += (modello(X).argmax(1) == y).sum().item()

    acc = corretti / len(test_data)
    storia_loss.append(loss_ep / len(train_loader))
    storia_acc.append(acc)
    print(f"Epoca {epoca+1:2d} | Loss: {storia_loss[-1]:.4f} | Acc: {acc*100:.2f}%")

# --- Grafici ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(storia_loss, color="coral")
ax1.set_title("Loss durante il training")
ax1.set_xlabel("Epoca"); ax1.set_ylabel("Loss")

ax2.plot([a*100 for a in storia_acc], color="steelblue")
ax2.set_title("Accuracy sul test set")
ax2.set_xlabel("Epoca"); ax2.set_ylabel("Accuracy (%)"); ax2.set_ylim(90, 100)
plt.tight_layout(); plt.show()

# --- Visualizza alcune predizioni ---
modello.eval()
esempi_X, esempi_y = next(iter(test_loader))
with torch.no_grad():
    predizioni = modello(esempi_X.to(device)).argmax(1).cpu()

fig, assi = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(assi.flat):

```

```

ax.imshow(esempi_X[i].squeeze(), cmap="gray")
vero, pred = esempi_y[i].item(), predizioni[i].item()
ax.set_title(f"Vero: {vero} Pred: {pred}", color="green" if vero==pred else
"red", fontsize=10)
ax.axis("off")
plt.suptitle("Predizioni sul test set (verde=corretto, rosso=sbagliato)")
plt.tight_layout(); plt.show()

```

Con questo codice dovresti ottenere circa **97–98% di accuracy** dopo 10 epoche. Un classificatore casuale otterrebbe il 10% (1 classe su 10 a caso).

Riepilogo del Capitolo 8

Hai imparato:

- **Tensor vs array:** un tensor PyTorch è come un array NumPy, ma gira su GPU e traccia le operazioni per il calcolo automatico dei gradienti.
- **Autograd:** con `requires_grad=True` e `.backward()`, PyTorch calcola i gradienti di tutti i parametri risalendo il grafo computazionale — questo è il cuore della backpropagation.
- **Dataset e DataLoader:** il pattern standard per caricare dati in batch con shuffling.
- **nn.Module:** dichiara i layer in `__init__`, descrivi il flusso in `forward`.
- **Il training loop in 5 passi:** `zero_grad()` → `forward` → `loss` → `backward()` → `step()`. Si ripete per ogni batch di ogni epoca.
- **`train()` vs `eval()`:** modalità distinte — `train()` attiva dropout, `eval()` lo disattiva per predizioni deterministiche.
- **Salvare e caricare:** `state_dict()` è il modo corretto per persistere un modello addestrato.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: adattare il classificatore a Fashion-MNIST, che contiene capi di abbigliamento. La sfida è più difficile perché alcune classi si somigliano molto (Shirt vs T-shirt, Coat vs Pullover).

Traccia:

Fashion-MNIST ha la stessa struttura di MNIST (60.000 immagini 28×28, 10 classi), con classi: T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

1. Sostituisci `datasets.MNIST` con `datasets.FashionMNIST`. La normalizzazione corretta per Fashion-MNIST è `mean=0.2860`, `std=0.3530`.
2. Esegui il training per 10 epoche con la stessa architettura. Quanta accuracy ottieni? (Aspettati circa 87–89%.)
3. Aggiungi un quarto layer `nn.Linear(256, 128)` con `nn.ReLU()` tra il terzo e il quarto layer originale. L'accuracy migliora? Di quanto?
4. Prova ad aumentare il dropout dal 20% al 40%. Cosa succede alla curva di loss? Spiega il comportamento osservato in 2–3 righe.
5. **Domanda di interpretazione:** guarda il grafico delle predizioni finali. Quali classi vengono confuse più spesso? Perché ha senso intuitivo?

Consegna: il notebook Colab con i grafici di loss e accuracy, il grafico delle predizioni, e le risposte scritte ai punti 3, 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Reti Convoluzionali (CNN) — scopriremo perché un tipo di rete specializzata per le immagini raggiunge il 99% su MNIST invece del 97–98% ottenuto con un MLP.

Capitolo 9

Reti Convoluzionali (CNN)

"Un MLP guarda ogni pixel indipendentemente dagli altri. Una CNN guarda i pixel nel contesto dei loro vicini — esattamente come fai tu quando riconosci una forma."

9.1 Perché un MLP non basta per le immagini?

Nel Capitolo 8 hai costruito un classificatore MNIST che raggiunge il 97–98% di accuracy. Sembra ottimo. Ma c'è un problema strutturale nel modo in cui quel modello funziona.

Un MLP (Multi-Layer Perceptron) appiattisce l'immagine in un vettore prima di elaborarla. Un'immagine 28×28 diventa un vettore di 784 numeri, e ogni neurone del primo layer è connesso a tutti e 784 i pixel. Questo ha tre conseguenze negative.

Ignora la struttura spaziale. Un pixel non è indipendente dai suoi vicini — un bordo è fatto di pixel adiacenti con valori simili, un angolo è l'incontro di due bordi. L'MLP non lo sa: per lui i pixel sono numeri in una lista, senza relazioni di vicinanza.

Non riconosce pattern in posizioni diverse. Se addestri l'MLP a riconoscere un "7" centrato nell'immagine, farà fatica con un "7" spostato di 3 pixel a sinistra. Ogni posizione è trattata come un caso separato.

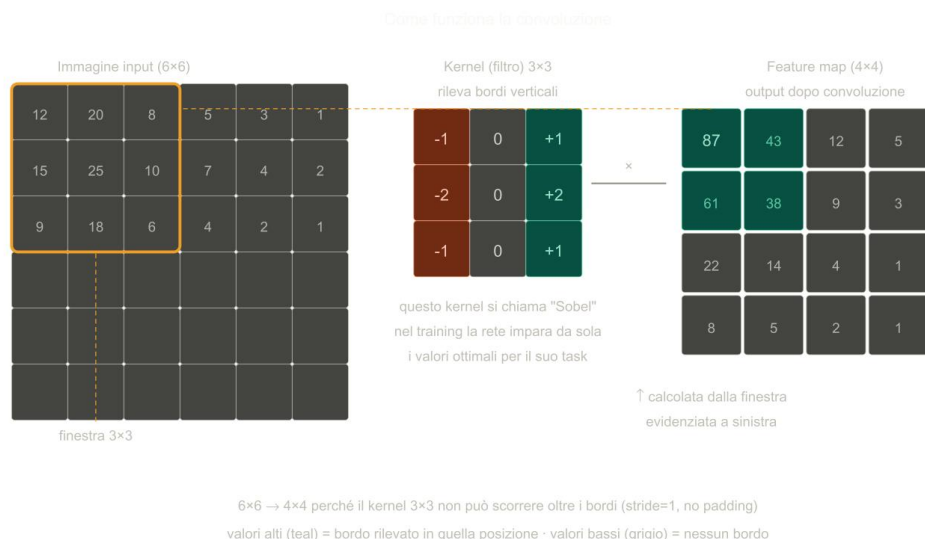
Troppi parametri. Una rete per immagini a colori 224×224 (dimensione standard) avrebbe un primo layer con $224 \times 224 \times 3 = 150.528$ input. Connettere tutto a tutto genera milioni di parametri già al primo layer, il che richiede enormi quantità di dati per evitare l'overfitting.

Le **Reti Convoluzionali** (CNN) risolvono tutti e tre questi problemi con un'idea elegante: invece di connettere ogni neurone a tutti i pixel, usano dei piccoli filtri che scorrono sull'immagine guardando una zona alla volta.

***Analogia:** immagina di dover cercare la parola "gatto" in un libro. Non leggi ogni parola isolata — guardi la sequenza di lettere nel contesto delle lettere vicine. Una CNN fa lo stesso con i pixel: cerca pattern locali (bordi, texture, angoli) scorrendo un piccolo "occhio" su tutta l'immagine.*

9.2 La convoluzione: un filtro che scorre sull'immagine

(Figura 9.1 — $6 \times 6 \rightarrow 4 \times 4$ perché il kernel 3×3 non può scorrere oltre i bordi (stride=1, no padding) valori alti (teal) = bordo rilevato in quella posizione · valori bassi (grigio) = nessun bordo)



Il mattone fondamentale di una CNN è l'operazione di **convoluzione**. Come mostra la Figura 9.1, funziona così: prendi un piccolo filtro (detto **kernel**), tipicamente 3×3 o 5×5, e fallo scorrere su tutta l'immagine. In ogni posizione, moltiplica i valori del kernel per i pixel corrispondenti e somma tutto — il risultato è un singolo numero nella **feature map** di output.

Il kernel della figura è il filtro di Sobel per i bordi verticali: ha valori negativi a sinistra e positivi a destra. Dove l'immagine ha un bordo verticale (il valore dei pixel cambia bruscamente da sinistra a destra), il prodotto dà un numero grande. Dove l'immagine è uniforme, il risultato si avvicina a zero.

La cosa fondamentale è questa: **i valori del kernel non sono scritti da te — vengono appresi durante il training**. La rete scopre da sola quali filtri sono utili per il task. Per il riconoscimento di immagini, i primi layer imparano filtri per bordi e texture; i layer successivi combinano questi elementi per riconoscere forme più complesse; i layer finali riconoscono parti di oggetti e poi oggetti interi.

I parametri della convoluzione

Prima di scrivere codice, definiamo i termini che troverai ovunque.

Stride — di quanti pixel si sposta il kernel ad ogni passo. Con stride=1 si sposta di un pixel alla volta (default). Con stride=2 si sposta di due, producendo una feature map più piccola. Stride alto = downsampling dell'immagine.

Padding — pixel artificiali aggiunti ai bordi dell'immagine prima della convoluzione. Senza padding (padding=0), una convoluzione 3×3 su un'immagine 6×6 produce una feature map 4×4 — l'immagine rimpicciolisce ad ogni layer. Con padding=1 l'output ha la stessa dimensione dell'input, il che è comodo per reti profonde.

Numero di filtri (out_channels) — quanti kernel diversi applichi. Con 32 filtri ottieni 32 feature map diverse, ognuna che cerca un pattern diverso. Più filtri = più capacità di rappresentazione, ma anche più parametri.

```
import torch
import torch.nn as nn
```

```
# nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)
conv = nn.Conv2d(
    in_channels=1,      # immagine in scala di grigi (1 canale)
    out_channels=32,     # 32 filtri diversi → 32 feature map
    kernel_size=3,       # kernel 3×3
    stride=1,           # si sposta di 1 pixel alla volta
    padding=1           # padding=1 mantiene dimensioni input = output
)

# Simuliamo un batch di 8 immagini 28×28 in scala di grigi
x = torch.randn(8, 1, 28, 28)  # (batch, canali, altezza, larghezza)
y = conv(x)
print(y.shape)  # torch.Size([8, 32, 28, 28])
                # 8 immagini, 32 feature map, ancora 28×28 grazie al padding
```

9.3 Pooling: ridurre le dimensioni mantenendo l'informazione

Dopo ogni layer di convoluzione si aggiunge tipicamente un layer di **pooling**, che riduce le dimensioni spaziali delle feature map. Il tipo più comune è il **MaxPooling**: divide la feature map in finestre (tipicamente 2×2) e prende il valore massimo di ciascuna.

Perché è utile? Tre ragioni.

Riduce il numero di parametri nei layer successivi, rendendo la rete più veloce e meno soggetta a overfitting. Introduce invarianza alla posizione: se un bordo si sposta di qualche pixel, il valore massimo nella finestra 2×2 cambia poco. E aumenta il "campo recettivo" — ogni neurone nei layer successivi vede una porzione più grande dell'immagine originale.

```
# MaxPool2d(kernel_size, stride)
# default: stride = kernel_size (nessuna sovrapposizione tra finestre)
pool = nn.MaxPool2d(kernel_size=2, stride=2)

x = torch.randn(8, 32, 28, 28)  # feature map dopo Conv2d
y = pool(x)
print(y.shape)  # torch.Size([8, 32, 14, 14])
                # le dimensioni spaziali si dimezzano: 28 → 14
```

9.4 Una CNN completa in PyTorch

Mettiamo insieme convoluzione, pooling e layer fully connected in una CNN funzionante per MNIST. La struttura rispecchia la Figura 9.1: prima i layer convoluzionali estraggono le feature, poi il flatten le appiattisce, infine i layer lineari classificano.

```
import torch
import torch.nn as nn

class CNN_MNIST(nn.Module):

    def __init__(self):
        super().__init__()

        # --- Blocco 1: estrazione feature basso livello ---
```

```

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        # in: [B, 1, 28, 28]
        # out: [B, 32, 28, 28] - 32 feature map, stessa dimensione grazie a
padding=1

        self.pool1 = nn.MaxPool2d(2)
        # out: [B, 32, 14, 14] - dimezzate

        # --- Blocco 2: feature più astratte ---
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        # out: [B, 64, 14, 14]

        self.pool2 = nn.MaxPool2d(2)
        # out: [B, 64, 7, 7] - dimezzate di nuovo

        # --- Classificatore ---
        self.flatten = nn.Flatten()
        # out: [B, 64*7*7] = [B, 3136]

        self.fc1 = nn.Linear(3136, 128)
        self.fc2 = nn.Linear(128, 10) # 10 classi

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # Blocco 1
        x = self.relu(self.conv1(x))
        x = self.pool1(x)

        # Blocco 2
        x = self.relu(self.conv2(x))
        x = self.pool2(x)

        # Classificatore
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

device = "cuda" if torch.cuda.is_available() else "cpu"
modello = CNN_MNIST().to(device)

# Verifica le dimensioni con un batch finto
x_test = torch.randn(4, 1, 28, 28).to(device)
print(modello(x_test).shape) # torch.Size([4, 10]) ← corretto

# Conta i parametri
totale = sum(p.numel() for p in modello.parameters())
print(f"Parametri totali: {totale:,}") # ~431,000
# Meno di 500K parametri, contro i 700K+ dell'MLP del Cap. 8
# Eppure raggiungerà ~99% di accuracy invece del 97-98%

```

Il training loop è identico a quello del Capitolo 8 — `forward`, `loss`, `backward`, `step`. Le CNN non cambiano come si addestra un modello, cambiano solo l'architettura.

```

from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
train_data = datasets.MNIST("./data", train=True, download=True,
transform=transform)
test_data = datasets.MNIST("./data", train=False, download=True,
transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

criterio = nn.CrossEntropyLoss()
ottimizzatore = torch.optim.Adam(modello.parameters(), lr=0.001)

for epoca in range(10):
    modello.train()
    loss_ep = 0
    for X, y in train_loader:
        X, y = X.to(device), y.to(device)
        ottimizzatore.zero_grad()
        loss = criterio(modello(X), y)
        loss.backward()
        ottimizzatore.step()
        loss_ep += loss.item()

    modello.eval()
    corretti = 0
    with torch.no_grad():
        for X, y in test_loader:
            X, y = X.to(device), y.to(device)
            corretti += (modello(X).argmax(1) == y).sum().item()

    acc = corretti / len(test_data)
    print(f"Epoca {epoca+1:2d} | Loss: {loss_ep/len(train_loader):.4f} | Acc:
{acc*100:.2f}%")

```

Con questo codice dovresti ottenere circa **99% di accuracy** — un miglioramento netto rispetto al 97–98% dell'MLP, con meno parametri.

9.5 Le architetture storiche: da LeNet a ResNet

Capire come si sono evolute le architetture CNN ti dà due cose: intuizione su perché certe scelte funzionano, e vocabolario per leggere la letteratura e il codice che troverai in produzione.

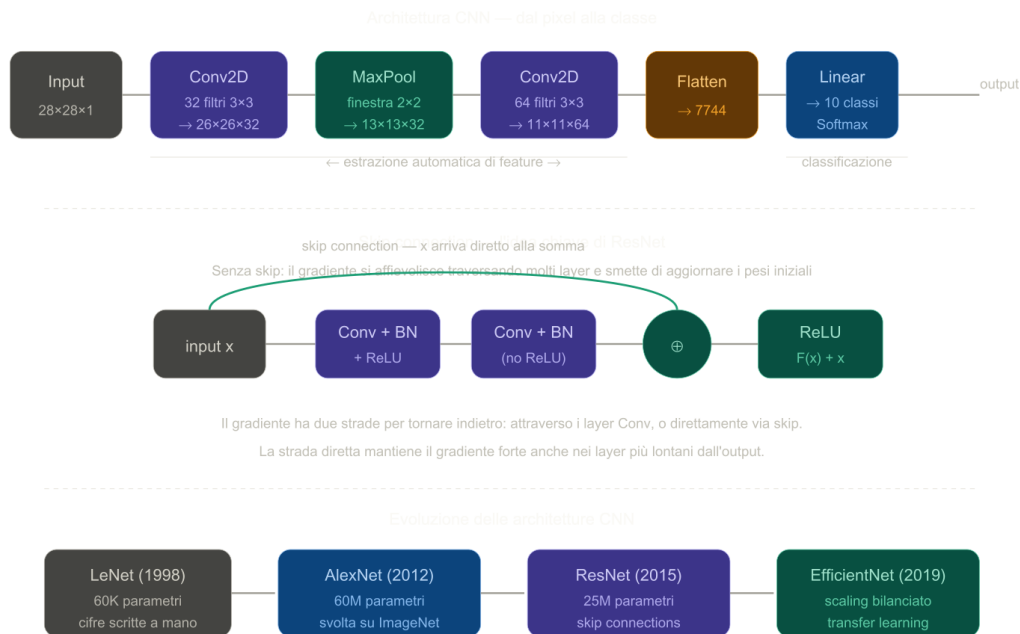


Figura 9.2 — L'architettura CNN completa con conv → pooling → flatten → fully connected, e le skip connections di ResNet.

La Figura 9.2 mostra la linea evolutiva. Ecco cosa ha portato ogni salto.

LeNet (1998) — la prima CNN pratica, progettata da Yann LeCun per riconoscere cifre scritte a mano. Due layer convoluzionali, due di pooling, due fully connected. Soli 60.000 parametri. Funzionava, ma i computer dell'epoca erano troppo lenti per scalarlo a immagini grandi.

AlexNet (2012) — il momento che ha cambiato tutto. Prima CNN ad usare GPU sistematicamente, dropout per la regolarizzazione, e ReLU invece di funzioni di attivazione più lente. Vince ImageNet con un margine enorme (errore dal 26% al 15%) e convince il mondo accademico che il deep learning funziona davvero. 60 milioni di parametri.

ResNet (2015) — introduce le **skip connections** (Figura 9.2, sezione centrale): un collegamento diretto che porta l'input di un blocco direttamente all'output, saltando i layer intermedi. Questo permette di addestrare reti molto più profonde (50, 101, 152 layer) senza il problema del gradiente che svanisce. ResNet-50 è ancora oggi uno dei modelli più usati come backbone per il transfer learning.

EfficientNet (2019) — invece di aggiungere semplicemente più layer, scala in modo bilanciato profondità, larghezza e risoluzione dell'input. Risultato: accuracy comparabile a reti molto più grandi, con molti meno parametri. È la scelta moderna per il transfer learning quando le risorse computazionali sono limitate.

Le skip connections di ResNet — perché funzionano

Il problema delle reti molto profonde è il **vanishing gradient**: durante la backpropagation, il gradiente deve attraversare decine di layer. Ad ogni layer si moltiplica per valori piccoli, e dopo 50 layer è praticamente zero — i pesi dei primi layer smettono di aggiornarsi.

La skip connection risolve questo con un'osservazione semplice: invece di imparare la trasformazione $F(x)$, il blocco impara il **residuo** $F(x) = H(x) - x$, dove $H(x)$ è la trasformazione desiderata. L'output del blocco è $F(x) + x$.

Perché aiuta? Perché il gradiente ora ha due strade per tornare indietro: attraverso i layer convoluzionali, o direttamente via skip connection. La strada diretta mantiene il gradiente forte anche nei layer più lontani dall'output. È come avere una corsia preferenziale per il segnale di errore.

```
class BloccoResiduale(nn.Module):
    """Un blocco residuale come in ResNet."""

    def __init__(self, canali):
        super().__init__()
        self.conv1 = nn.Conv2d(canali, canali, kernel_size=3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(canali) # Batch Normalization: normalizza i
        valori # tra un layer e l'altro, stabilizza
        il training
        self.conv2 = nn.Conv2d(canali, canali, kernel_size=3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(canali)
        self.relu = nn.ReLU()

    def forward(self, x):
        residuo = x # salva l'input

        out = self.relu(self.bn1(self.conv1(x))) # primo layer
        out = self.bn2(self.conv2(out)) # secondo layer (niente ReLU
        ancora)

        out = out + residuo # somma il residuo all'output
        out = self.relu(out) # ReLU dopo la somma
        return out

# Verifica
blocco = BloccoResiduale(canali=64)
x = torch.randn(4, 64, 14, 14)
print(blocco(x).shape) # torch.Size([4, 64, 14, 14]) - stessa dimensione
```

9.6 Transfer Learning: non partire da zero

Addestrare una CNN su ImageNet da zero richiede settimane su cluster di GPU e milioni di immagini. Per la maggior parte dei progetti reali, non è né necessario né pratico.

Il **transfer learning** sfrutta un modello già addestrato su un task generico (tipicamente ImageNet con 1000 classi) e lo adatta al tuo task specifico. Funziona perché i layer iniziali imparano feature generali — bordi, texture, forme — che sono utili per quasi qualsiasi task visivo, non solo per quello originale.

La strategia più comune è **freeze + fine-tune**:

6. Prendi un modello pre-addestrato (es. ResNet50).
7. Congela i pesi di tutti i layer tranne l'ultimo — questi non si aggiornano durante il training.
8. Sostituisci l'ultimo layer con uno nuovo che ha il numero di classi del tuo task.
9. Addestra solo il nuovo layer (o gli ultimi pochi layer) sui tuoi dati.

```
import torchvision.models as models
```

```

# Carica ResNet18 pre-addestrato su ImageNet
# weights="DEFAULT" usa i pesi migliori disponibili
modello_pretrained = models.resnet18(weights="DEFAULT")

# Congela tutti i parametri – non si aggiornano durante il training
for param in modello_pretrained.parameters():
    param.requires_grad = False

# Sostituisci l'ultimo layer (fc) con uno per il tuo numero di classi
# ResNet18 ha fc: Linear(512, 1000) – cambiamo 1000 con il nostro numero di classi
num_classi = 10 # es. Fashion-MNIST
modello_pretrained.fc = nn.Linear(512, num_classi)
# Solo questo layer ha requires_grad=True (default per i layer nuovi)

modello_pretrained = modello_pretrained.to(device)

# Verifica quanti parametri si aggiornano
addestrabili = sum(p.numel() for p in modello_pretrained.parameters() if
p.requires_grad)
totali = sum(p.numel() for p in modello_pretrained.parameters())
print(f"Parametri addestrabili: {addestrabili:,} / {totali:,}")
# Qualcosa come: 5,130 / 11,181,642
# Stiamo addestrando meno dello 0.05% dei parametri!

```

Quando usare il transfer learning? Quasi sempre, quando lavori su immagini. Anche con pochi dati (qualche centinaio di immagini), un modello pre-addestrato con solo il classification head riaddestrato supera spesso una CNN addestrata da zero su migliaia di immagini.

Data augmentation: creare dati artificiali

Il transfer learning risolve il problema dei pochi parametri da addestrare. Ma se hai anche pochi dati, il modello può ancora overfittare sulle poche immagini di training. La **data augmentation** trasforma casualmente le immagini di training ad ogni epoca, aumentando artificialmente la varietà dei dati che il modello vede.

```

from torchvision import transforms

# Trasformazioni per il training – casuali ad ogni immagine
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5), # riflette orizzontalmente con prob.
    50%
    transforms.RandomRotation(degrees=15), # ruota di ±15 gradi
    transforms.ColorJitter(brightness=0.2, # varia luminosità, contrasto,
                           contrast=0.2, # saturazione e tonalità
                           saturation=0.2),
    transforms.RandomCrop(32, padding=4), # ritaglia con padding
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Per il test set: niente augmentation, solo normalizzazione
# (vuoi predizioni stabili, non casuali)
transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

```


Regola pratica: applica la data augmentation solo al training set, mai al validation o test set. Sul test set vuoi misurare quanto il modello generalizza, non quanto è robusto alle trasformazioni.

9.7 Esercizio guidato — classificatore con ResNet pre-addestrato

Costruiamo un classificatore di immagini usando ResNet18 pre-addestrato su un dataset reale. Useremo CIFAR-10: 60.000 immagini a colori 32×32 in 10 classi (aereo, auto, uccello, gatto, cervo, cane, rana, cavallo, nave, camion).

```
# =====
# Transfer learning con ResNet18 su CIFAR-10
# Esegui su Google Colab con GPU T4 attiva
# =====

import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
from torchvision import transforms, datasets
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Dispositivo: {device}")

# --- Dati ---
# CIFAR-10 ha immagini 32×32 a colori (3 canali RGB)
# ResNet si aspetta input almeno 224×224, quindi facciamo resize
transform_train = transforms.Compose([
    transforms.Resize(64),                # scala a 64×64
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(64, padding=8),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), # media per canale R, G, B
                        (0.2470, 0.2435, 0.2616)) # std per canale R, G, B
])

transform_test = transforms.Compose([
    transforms.Resize(64),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                        (0.2470, 0.2435, 0.2616))
])

train_data = datasets.CIFAR10("./data", train=True, download=True,
transform=transform_train)
test_data = datasets.CIFAR10("./data", train=False, download=True,
transform=transform_test)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True, num_workers=2)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False, num_workers=2)

classi =
["aereo", "auto", "uccello", "gatto", "cervo", "cane", "rana", "cavallo", "nave", "camion"]

# --- Modello: ResNet18 pre-addestrato ---
```

```

modello = models.resnet18(weights="DEFAULT")

# Congela tutti i layer
for param in modello.parameters():
    param.requires_grad = False

# Sostituisci il classification head con uno per 10 classi
# In ResNet18 il layer finale si chiama 'fc'
modello.fc = nn.Sequential(
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 10)
)
modello = modello.to(device)

addestrabili = sum(p.numel() for p in modello.parameters() if p.requires_grad)
totali       = sum(p.numel() for p in modello.parameters())
print(f"Parametri addestrabili: {addestrabili:,} / {totali:,}")

# --- Training ---
criterio      = nn.CrossEntropyLoss()
ottimizzatore = torch.optim.Adam(modello.parameters(), lr=0.001)

# Learning rate scheduler: dimezza il lr se la loss smette di migliorare
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    ottimizzatore, mode='min', factor=0.5, patience=3
)

storia_loss, storia_acc = [], []

for epoca in range(15):
    modello.train()
    loss_ep = 0
    for X, y in train_loader:
        X, y = X.to(device), y.to(device)
        ottimizzatore.zero_grad()
        loss = criterio(modello(X), y)
        loss.backward()
        ottimizzatore.step()
        loss_ep += loss.item()

    modello.eval()
    corretti = 0
    loss_val = 0
    with torch.no_grad():
        for X, y in test_loader:
            X, y = X.to(device), y.to(device)
            out = modello(X)
            loss_val += criterio(out, y).item()
            corretti += (out.argmax(1) == y).sum().item()

    acc = corretti / len(test_data)
    loss_val = loss_val / len(test_loader)
    scheduler.step(loss_val) # aggiorna il learning rate se necessario

    storia_loss.append(loss_ep / len(train_loader))
    storia_acc.append(acc)

```

```

    print(f"Epoca {epoca+1:2d} | Loss train: {storia_loss[-1]:.4f} | Acc test:
{acc*100:.2f}%")

# --- Grafici ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(storia_loss, color="coral")
ax1.set_title("Loss di training"); ax1.set_xlabel("Epoca")

ax2.plot([a*100 for a in storia_acc], color="steelblue")
ax2.set_title("Accuracy sul test set"); ax2.set_xlabel("Epoca")
ax2.set_ylabel("Accuracy (%)")
plt.tight_layout(); plt.show()

# --- Visualizza predizioni con confidenza ---
modello.eval()
X_batch, y_batch = next(iter(test_loader))
with torch.no_grad():
    logits = modello(X_batch.to(device))
    probs = torch.softmax(logits, dim=1).cpu()
    preds = probs.argmax(1)

fig, assi = plt.subplots(2, 5, figsize=(14, 6))
for i, ax in enumerate(assi.flat):
    # denormalizza l'immagine per visualizzarla
    img = X_batch[i].permute(1, 2, 0).numpy()
    img = img * (0.2470, 0.2435, 0.2616) + (0.4914, 0.4822, 0.4465)
    img = img.clip(0, 1)
    ax.imshow(img)
    vero = classi[y_batch[i]]
    pred = classi[preds[i]]
    conf = probs[i][preds[i]].item() * 100
    colore = "green" if vero == pred else "red"
    ax.set_title(f"{pred} ({conf:.0f}%) \nvero: {vero}", color=colore, fontsize=9)
    ax.axis("off")
plt.suptitle("Predizioni con confidenza (verde=corretto, rosso=sbagliato)")
plt.tight_layout(); plt.show()

```

Con questo setup dovresti ottenere circa **85–88% di accuracy** su CIFAR-10 addestrando solo il classification head. Una CNN addestrata da zero sullo stesso dataset raggiungerebbe risultati simili ma richiederebbe molte più epoche e un dataset più grande.

Riepilogo del Capitolo 9

Hai imparato:

- **Perché le CNN superano gli MLP sulle immagini:** sfruttano la struttura spaziale, riconoscono pattern indipendentemente dalla posizione, e usano molti meno parametri.
- **La convoluzione:** un kernel scorre sull'immagine calcolando una feature map. Stride, padding e numero di filtri controllano le dimensioni dell'output.
- **Il MaxPooling:** riduce le dimensioni spaziali prendendo il massimo in finestre 2×2, introducendo invarianza alla posizione.
- **L'architettura CNN:** conv → ReLU → pooling, ripetuto più volte, poi flatten e layer fully connected per la classificazione.

- **Le skip connections di ResNet:** permettono di addestrare reti molto profonde dando al gradiente una strada diretta per tornare indietro.
- **Il transfer learning:** usa un modello pre-addestrato su ImageNet, congela i layer esistenti, sostituisce solo il classification head. Funziona bene anche con pochi dati.
- **La data augmentation:** trasforma casualmente le immagini di training per ridurre l'overfitting, mai sul test set.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un classificatore per un dataset di immagini reali usando transfer learning, e confrontarlo con una CNN addestrata da zero.

Traccia:

Userai STL-10 (`datasets.STL10`): 500 immagini di training per classe, 10 classi, 96×96 pixel a colori. È pensato apposta per il transfer learning — il training set piccolo rende difficile addestrare da zero.

10. Costruisci due modelli: (a) ResNet18 pre-addestrato con classification head sostituito, (b) la CNN definita nella sezione 9.4 (adattata a 3 canali e 96×96 input). Addestra entrambi per 15 epoche e confronta le curve di accuracy.
11. Per il modello (a): prova lo **sblocco progressivo** — dopo 10 epoche di training del solo head, sblocca anche gli ultimi due layer di ResNet18 (`layer4`) e continua ad addestrare con un learning rate 10 volte più piccolo (`lr=0.0001`). L'accuracy migliora? Di quanto?

```
# Sblocco del layer4 dopo le prime 10 epoche
for param in modello.layer4.parameters():
    param.requires_grad = True
ottimizzatore = torch.optim.Adam(
    filter(lambda p: p.requires_grad, modello.parameters()), lr=0.0001
)
```

12. Aggiungi al grafico finale una terza curva che mostra la differenza di accuracy tra i due modelli epoca per epoca. In quale epoca il modello pre-addestrato raggiunge l'accuracy massima del modello da zero?
13. **Domanda di interpretazione:** guarda le immagini classificate erroneamente dal modello pre-addestrato. Riesci a identificare un pattern — tipi di immagini che risultano sistematicamente difficili? Quali classi vengono confuse più spesso?

Consegna: il notebook Colab con i grafici comparativi, le curve di sblocco progressivo, e le risposte scritte ai punti 3 e 4.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Sequenze — RNN, LSTM e il problema del contesto lungo. Scopriremo perché i dati che hanno un ordine temporale richiedono architetture diverse, e perché alla fine anche queste sono state superate dal Transformer.

Capitolo 10

Sequenze: RNN, LSTM e il problema del contesto lungo

"Un MLP guarda ogni input come se fosse indipendente dal precedente. Una RNN ricorda — o almeno ci prova."

10.1 Perché i dati sequenziali sono diversi

Nei capitoli precedenti ogni input era indipendente dagli altri. Un'immagine MNIST non dipende dall'immagine prima di lei. Ma moltissimi dati reali hanno un ordine temporale in cui ogni elemento dipende da quelli precedenti.

Una frase: "Il gatto mangiò il pesce perché aveva fame". Il significato di "aveva" dipende dall'intera frase prima. Senza contesto, non sai chi aveva fame.

Una serie temporale: la temperatura di domani dipende da quella degli ultimi giorni. Un valore isolato non basta.

Un file audio: un fonema dipende dai fonemi che lo precedono. Isolare 20 millisecondi di audio e classificarli senza contesto è quasi impossibile.

Questi tre casi hanno una struttura comune: l'ordine conta, e il contesto passato è informazione rilevante. I modelli che abbiamo visto finora non hanno modo di rappresentare questo. Serve un'architettura che abbia **memoria**.

10.2 La RNN: l'idea della memoria ricorrente

L'idea alla base delle **Reti Neurali Ricorrenti** (RNN) è semplice: aggiungi al modello uno stato interno — chiamato **hidden state** — che viene aggiornato ad ogni passo e passato al passo successivo. In questo modo il modello "ricorda" cosa ha visto prima.

Formalmente, ad ogni passo temporale t la RNN calcola:

$$h(t) = \tanh(W_h \cdot h(t-1) + W_x \cdot x(t) + b)$$

Dove $h(t)$ è il nuovo hidden state, $h(t-1)$ è quello del passo precedente, $x(t)$ è l'input corrente, e W_h , W_x , b sono parametri appresi durante il training. Lo stesso set di parametri viene usato ad ogni passo — è questo che la rende "ricorrente".

Analogia: immagina di leggere una frase parola per parola tenendo in mano un foglio di appunti. Ad ogni parola aggiorni i tuoi appunti — aggiungi informazioni nuove, cancelli quelle vecchie irrilevanti. Alla fine hai un riassunto dell'intera frase. La RNN funziona così: $h(t)$ sono i tuoi appunti dopo aver letto fino alla parola t .

compresi tra -1 e 1. Dopo 5–6 moltiplicazioni di numeri piccoli, il gradiente si avvicina a zero: il peso del passo 1 riceve un segnale di aggiornamento quasi nullo. La rete smette di imparare le dipendenze a lungo raggio.

Questo problema si chiama **vanishing gradient** — il gradiente svanisce traversando molti passi temporali. L'effetto opposto, **exploding gradient**, può anche verificarsi (il gradiente cresce invece di ridursi), ma è più facile da gestire con il gradient clipping.

```
# Gradient clipping — taglia i gradienti se superano una soglia
# Si usa comunemente con RNN e LSTM
torch.nn.utils.clip_grad_norm_(modello.parameters(), max_norm=1.0)
# Va chiamato dopo loss.backward() e prima di ottimizzatore.step()
```

Il vanishing gradient è il motivo per cui le RNN semplici funzionano bene su sequenze corte (fino a ~20 passi) ma falliscono su sequenze lunghe. Ed è esattamente il problema che LSTM risolve.

10.4 LSTM: porte che controllano la memoria

La **Long Short-Term Memory** (LSTM), proposta da Hochreiter e Schmidhuber nel 1997, risolve il vanishing gradient con un'idea elegante: aggiunge un secondo canale di memoria — il **cell state** — che scorre attraverso la sequenza quasi indisturbato, e introduce tre **porte** che controllano cosa memorizzare, cosa dimenticare e cosa leggere.

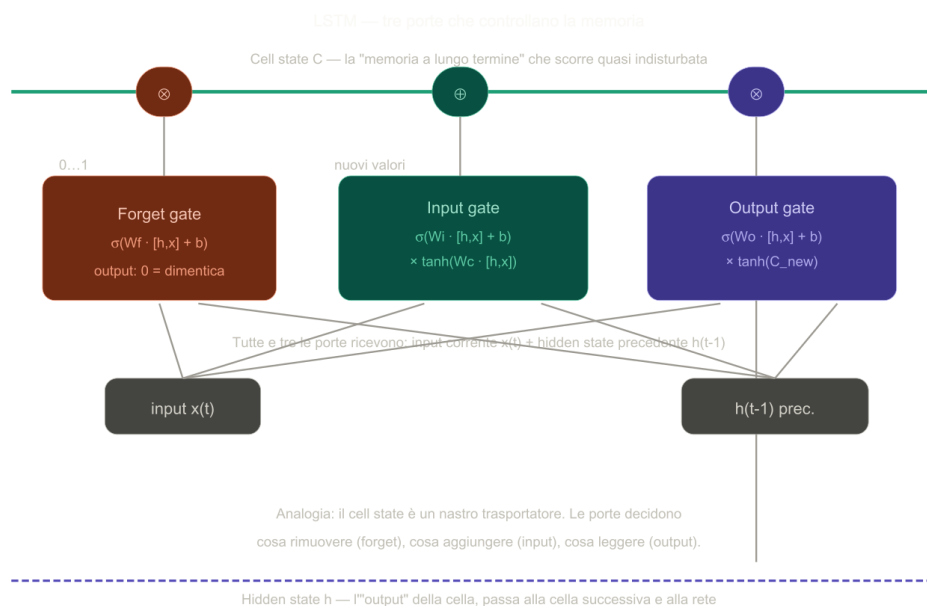


Figura 10.2 — Le porte dell'LSTM — forget, input, output.

Come mostra la Figura 10.2, il cell state è come un nastro trasportatore che attraversa tutta la sequenza con poche modifiche. Le porte sono piccole reti neurali che producono valori tra 0 e 1, usati come "quanto lasciare passare".

Forget gate — decide cosa cancellare dal cell state. Produce un valore tra 0 (dimentica tutto) e 1 (ricorda tutto) per ogni elemento del cell state, basandosi sull'input corrente e sull'hidden state precedente.

Input gate — decide cosa scrivere nel cell state. Calcola quali nuovi valori aggiungere e quanto peso dargli.

Output gate — decide cosa leggere dal cell state per produrre l'hidden state di output.

***Analogia:** il cell state è una lavagna. La forget gate è una spugna che cancella le parti vecchie irrilevanti. L'input gate è un pennarello che scrive le nuove informazioni. L'output gate è uno schermo che decide cosa mostrare all'esterno.*

Perché questo risolve il vanishing gradient? Perché il cell state riceve le informazioni attraverso somme (non moltiplicazioni successive di piccoli numeri). Il gradiente può fluire attraverso il cell state su molti passi senza affievolirsi.

```
# nn.LSTM ha la stessa interfaccia di nn.RNN ma restituisce anche il cell state
lstm = nn.LSTM(
    input_size=10,
    hidden_size=64,
    num_layers=2,
    batch_first=True,
    dropout=0.2
)

x = torch.randn(8, 50, 10)    # sequenze più lunghe: 50 passi
h0 = torch.zeros(2, 8, 64)    # hidden state iniziale
c0 = torch.zeros(2, 8, 64)    # cell state iniziale — novità rispetto a RNN

output, (h_n, c_n) = lstm(x, (h0, c0))
print(output.shape)           # torch.Size([8, 50, 64]) — output ad ogni passo
print(h_n.shape)              # torch.Size([2, 8, 64]) — hidden state finale
print(c_n.shape)              # torch.Size([2, 8, 64]) — cell state finale
```

10.5 GRU: una versione semplificata di LSTM

La **Gated Recurrent Unit** (GRU), proposta da Cho et al. nel 2014, è una versione semplificata dell'LSTM che unisce forget gate e input gate in un unico **update gate** ed elimina il cell state separato, usando solo l'hidden state.

Il risultato è un modello con meno parametri, più veloce da addestrare, che spesso raggiunge performance comparabili all'LSTM — soprattutto su dataset di dimensioni medie.

```
gru = nn.GRU(
    input_size=10,
    hidden_size=64,
    num_layers=2,
    batch_first=True,
    dropout=0.2
)

x = torch.randn(8, 50, 10)
h0 = torch.zeros(2, 8, 64)

output, h_n = gru(x, h0)    # come RNN: niente cell state separato
print(output.shape)         # torch.Size([8, 50, 64])
```


Quando usare LSTM vs GRU in pratica? Inizia con GRU — è più veloce e spesso sufficiente. Passa a LSTM se il task richiede di memorizzare dipendenze su sequenze molto lunghe (>200 passi) o se stai lavorando su un problema dove l'architettura è ben studiata in letteratura con LSTM.

10.6 RNN bidirezionali

In molti task — analisi del sentiment, NER, traduzione — il contesto futuro è importante quanto quello passato. La parola "banca" in "vado in banca" ha un significato diverso da "banca" in "mi siedo sulla banca del fiume". Per disambiguare, può servire guardare le parole successive.

Una **RNN bidirezionale** processa la sequenza in entrambe le direzioni e concatena i due hidden state. Raddoppia i parametri ma spesso migliora significativamente le performance su task dove il contesto completo è disponibile al momento dell'inferenza.

```
lstm_bidi = nn.LSTM(
    input_size=10,
    hidden_size=64,
    num_layers=2,
    batch_first=True,
    bidirectional=True  # unico parametro che cambia
)

x = torch.randn(8, 50, 10)
h0 = torch.zeros(4, 8, 64)  # num_layers * 2 direzioni = 4
c0 = torch.zeros(4, 8, 64)

output, (h_n, c_n) = lstm_bidi(x, (h0, c0))
print(output.shape)  # torch.Size([8, 50, 128])
                    # 128 = 64 forward + 64 backward concatenati
```

Attenzione: la bidirezionalità funziona solo quando hai l'intera sequenza disponibile in anticipo. Per la generazione di testo o le previsioni in tempo reale — dove il futuro non è ancora disponibile — devi usare una RNN unidirezionale.

10.7 Predizione di serie temporale con LSTM

Costruiamo un esempio completo: predire i valori futuri di una serie temporale sinusoidale con rumore, un pattern classico per capire le capacità di memorizzazione di LSTM.

L'idea è: dato una finestra di `seq_len` valori passati, predici il valore successivo.

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt

# --- Generazione dati ---
np.random.seed(42)
torch.manual_seed(42)
```

```

t      = np.linspace(0, 8 * np.pi, 1000)
segnale = np.sin(t) + 0.5 * np.sin(3 * t) + 0.1 * np.random.randn(len(t))
# Combinazione di due sinusoidi con rumore - abbastanza complessa da richiedere
memoria

# Normalizzazione: porta i valori tra 0 e 1
segnale_min, segnale_max = segnale.min(), segnale.max()
segnale_norm = (segnale - segnale_min) / (segnale_max - segnale_min)

# --- Creazione del dataset con finestre scorrevoli ---
def crea_sequenze(dati, seq_len):
    """Trasforma una serie temporale in coppie (input_window, valore_target)."""
    X, y = [], []
    for i in range(len(dati) - seq_len):
        X.append(dati[i : i + seq_len])      # finestra di seq_len valori
        y.append(dati[i + seq_len])          # valore successivo da predire
    return torch.tensor(X, dtype=torch.float32), torch.tensor(y, dtype=torch.float32)

SEQ_LEN = 30      # usa 30 valori passati per predire il successivo
X, y      = crea_sequenze(segnale_norm, SEQ_LEN)
X          = X.unsqueeze(-1)    # aggiunge dimensione feature: (N, 30) → (N, 30, 1)

# Split train/test: 80% training, 20% test
split      = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

from torch.utils.data import TensorDataset, DataLoader
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32,
                           shuffle=True)
test_loader  = DataLoader(TensorDataset(X_test, y_test), batch_size=32,
                           shuffle=False)

print(f"Training: {len(X_train)} sequenze | Test: {len(X_test)} sequenze")

# --- Architettura ---
class LSTMPredictor(nn.Module):

    def __init__(self, input_size=1, hidden_size=64, num_layers=2, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size = input_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            batch_first = True,
            dropout     = dropout
        )
        # Usiamo solo l'output dell'ultimo passo temporale per la predizione
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        # x: (batch, seq_len, 1)
        lstm_out, _ = self.lstm(x)
        # lstm_out: (batch, seq_len, hidden_size)
        # Prendiamo solo l'ultimo passo: contiene il contesto di tutta la sequenza

```

```

        ultimo = lstm_out[:, -1, :] # (batch, hidden_size)
        return self.fc(ultimo).squeeze(-1) # (batch,)

device    = "cuda" if torch.cuda.is_available() else "cpu"
modello   = LSTMPredictor().to(device)
print(f"Parametri: {sum(p.numel() for p in modello.parameters()):,}")

# --- Training ---
criterio   = nn.MSELoss() # Mean Squared Error: standard per regressione
ottimizzatore = torch.optim.Adam(modello.parameters(), lr=0.001)
scheduler   = torch.optim.lr_scheduler.ReduceLROnPlateau(
    ottimizzatore, patience=5, factor=0.5
)

storia_loss_train = []
storia_loss_test  = []

for epoca in range(50):
    # Training
    modello.train()
    loss_tr = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        ottimizzatore.zero_grad()
        pred = modello(xb)
        loss = criterio(pred, yb)
        loss.backward()
        # Gradient clipping: evita l'exploding gradient
        torch.nn.utils.clip_grad_norm_(modello.parameters(), max_norm=1.0)
        ottimizzatore.step()
        loss_tr += loss.item()

    # Valutazione
    modello.eval()
    loss_te = 0
    with torch.no_grad():
        for xb, yb in test_loader:
            xb, yb = xb.to(device), yb.to(device)
            loss_te += criterio(modello(xb), yb).item()

    loss_tr /= len(train_loader)
    loss_te /= len(test_loader)
    scheduler.step(loss_te)

    storia_loss_train.append(loss_tr)
    storia_loss_test.append(loss_te)

    if (epoca + 1) % 10 == 0:
        print(f"Epoca {epoca+1:3d} | Loss train: {loss_tr:.5f} | Loss test: {loss_te:.5f}")

# --- Visualizzazione: predizioni vs valori reali ---
modello.eval()
predizioni_list = []
valori_reali    = []

```

```

with torch.no_grad():
    for xb, yb in test_loader:
        pred = modello(xb.to(device)).cpu().numpy()
        predizioni_list.extend(pred)
        valori_reali.extend(yb.numpy())

# De-normalizza per tornare ai valori originali
pred_denorm = np.array(predizioni_list) * (segnale_max - segnale_min) + segnale_min
real_denorm = np.array(valori_reali) * (segnale_max - segnale_min) + segnale_min

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))

ax1.plot(storia_loss_train, label="Training", color="coral")
ax1.plot(storia_loss_test, label="Test", color="steelblue")
ax1.set_title("Curve di loss"); ax1.set_xlabel("Epoca"); ax1.set_ylabel("MSE")
ax1.legend()

ax2.plot(real_denorm[:200], label="Valori reali", color="steelblue", alpha=0.8)
ax2.plot(pred_denorm[:200], label="Predizioni LSTM", color="coral", alpha=0.8)
ax2.set_title("Predizioni vs valori reali (primi 200 passi del test set)")
ax2.set_xlabel("Passo temporale"); ax2.legend()

plt.tight_layout()
plt.show()

# Calcola MAE (Mean Absolute Error) - più interpretabile del MSE
mae = np.mean(np.abs(pred_denorm - real_denorm))
print(f"MAE sul test set: {mae:.4f}")

```

10.8 Quando usare ancora RNN e LSTM nel 2026

Il Transformer (Capitolo 11) ha superato le RNN e LSTM su quasi tutti i task legati al linguaggio naturale. Ma "superato" non significa "reso inutili". Esistono casi in cui le architetture ricorrenti sono ancora la scelta giusta.

Streaming e inferenza in tempo reale. Un Transformer processa l'intera sequenza in parallelo, ma questo richiede che la sequenza sia completa. Una RNN/LSTM elabora un elemento alla volta e produce output immediatamente — fondamentale per applicazioni real-time come il riconoscimento vocale su dispositivi embedded o la predizione di sensori industriali.

Sequenze molto lunghe con memoria limitata. Un Transformer ha costo computazionale quadratico nella lunghezza della sequenza (ogni token guarda tutti gli altri). Su sequenze di migliaia di passi, questo diventa proibitivo. Una LSTM ha costo lineare.

Dataset piccoli. I Transformer hanno bisogno di grandi quantità di dati per generalizzare bene. Su dataset con poche migliaia di esempi, una LSTM bidirezionale con dropout può superare un piccolo Transformer.

Serie temporali univariate semplici. Per la predizione di singole serie temporali (temperatura, consumo energetico, stock price) con dipendenze temporali moderate, una LSTM rimane una baseline solida e interpretabile.

Riepilogo del Capitolo 10

Hai imparato:

- **Perché le sequenze richiedono architetture diverse:** l'ordine conta, il contesto passato è informazione rilevante, e un MLP non ha modo di rappresentarlo.
- **La RNN:** aggiunge un hidden state che scorre tra i passi temporali, permettendo alla rete di "ricordare". Stessi parametri ad ogni passo.
- **Il vanishing gradient:** durante la backpropagation nel tempo, il gradiente si affievolisce attraverso molti passi e i token lontani smettono di influenzare l'apprendimento.
- **L'LSTM:** introduce cell state e tre porte (forget, input, output) che permettono alla memoria di fluire quasi indisturbata su sequenze lunghe. Risolve il vanishing gradient attraverso somme invece di moltiplicazioni successive.
- **La GRU:** versione semplificata dell'LSTM, meno parametri, spesso performance comparabili. Buona scelta di default.
- **RNN bidirezionali:** processano la sequenza in entrambe le direzioni. Utili quando il contesto futuro è disponibile, non adatte per la generazione in tempo reale.
- **Quando usarle ancora:** streaming real-time, sequenze molto lunghe, dataset piccoli, serie temporali semplici.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un classificatore di sentiment su recensioni di film usando una LSTM, e confrontarlo con una GRU bidirezionale.

Traccia:

Userai il dataset IMDB (50.000 recensioni di film, etichettate come positive o negative), disponibile direttamente da HuggingFace Datasets.

```
# Installazione e caricamento
# !pip install datasets
from datasets import load_dataset
dataset = load_dataset("imdb")
# dataset["train"] ha 25.000 esempi, dataset["test"] altri 25.000
# ogni esempio ha: "text" (la recensione) e "label" (0=negativo, 1=positivo)
```

14. Prepara i dati: costruisci un vocabolario a partire dalle parole più frequenti (tieni le 20.000 più comuni), tokenizza le recensioni e paddale a lunghezza fissa di 256 token. Usa `torch.nn.Embedding` per convertire gli indici in vettori densi di dimensione 128.
15. Addestra due modelli per 5 epoche e confronta accuracy e velocità di training: (a) LSTM a 2 layer, hidden size 128; (b) GRU bidirezionale a 2 layer, hidden size 64.
16. Per la classificazione usa solo l'hidden state dell'ultimo passo (o la media degli hidden state di tutti i passi — prova entrambe e confronta). Quale strategia dà risultati migliori?
17. Trova 3 esempi nel test set dove i due modelli danno predizioni diverse. Le recensioni sono genuinamente ambigue, o uno dei due modelli sbaglia chiaramente?
18. **Domanda di riflessione:** prova ad aumentare la lunghezza delle sequenze da 256 a 512 token. Cosa succede al tempo di training? Spiega perché in 2–3 righe, collegandoti al concetto di vanishing gradient e alla lunghezza delle sequenze.

Consegna: il notebook Colab con le curve comparative di loss e accuracy dei due modelli, gli esempi di predizioni diverse, e le risposte scritte ai punti 3, 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Il Transformer e il meccanismo di attenzione — l'architettura che ha reso obsolete le RNN per il linguaggio, permettendo ad ogni parola di "guardare" tutte le altre contemporaneamente.

Capitolo 11

Il Transformer e il Meccanismo di Attenzione

"Le RNN leggono le parole una alla volta, come un lettore lento che può dimenticare l'inizio della frase. Il Transformer le legge tutte insieme, come se potesse tenere l'intera pagina sott'occhio in un colpo solo."

11.1 Il problema che il Transformer risolve

Alla fine del Capitolo 10 hai visto che le LSTM migliorano la memoria delle RNN, ma non la eliminano del tutto: su sequenze molto lunghe il segnale si affievolisce, e il modello fa fatica a collegare parti distanti del testo.

C'è però un secondo problema, altrettanto importante: la **serialità**. Una RNN elabora i token uno alla volta — il passo 2 non può iniziare prima che il passo 1 sia finito. Questo rende impossibile parallelizzare il training su GPU. Con sequenze di migliaia di token e dataset da miliardi di parole, i tempi diventano proibitivi.

Il **Transformer**, proposto nel paper *"Attention Is All You Need"* (Vaswani et al., 2017), risolve entrambi i problemi con una sola idea: **elimina completamente la ricorrenza** e sostituisce la memoria sequenziale con il meccanismo di attenzione. Ogni token può guardare direttamente tutti gli altri in parallelo — non c'è più un "ordine" obbligato di elaborazione.

Il risultato è un'architettura che si addestra molto più velocemente, scala meglio con più dati e hardware, e generalizza meglio su sequenze lunghe. Non è un'esagerazione dire che quasi tutto quello che oggi chiamiamo "IA" — GPT, Claude, Gemini, LLaMA — è un Transformer.

11.2 Il meccanismo di attenzione: ogni token guarda tutti gli altri

L'idea centrale è questa: invece di passare un hidden state da un passo all'altro (come la RNN), calcoliamo direttamente quanto ogni token deve "prestare attenzione" a ogni altro token.

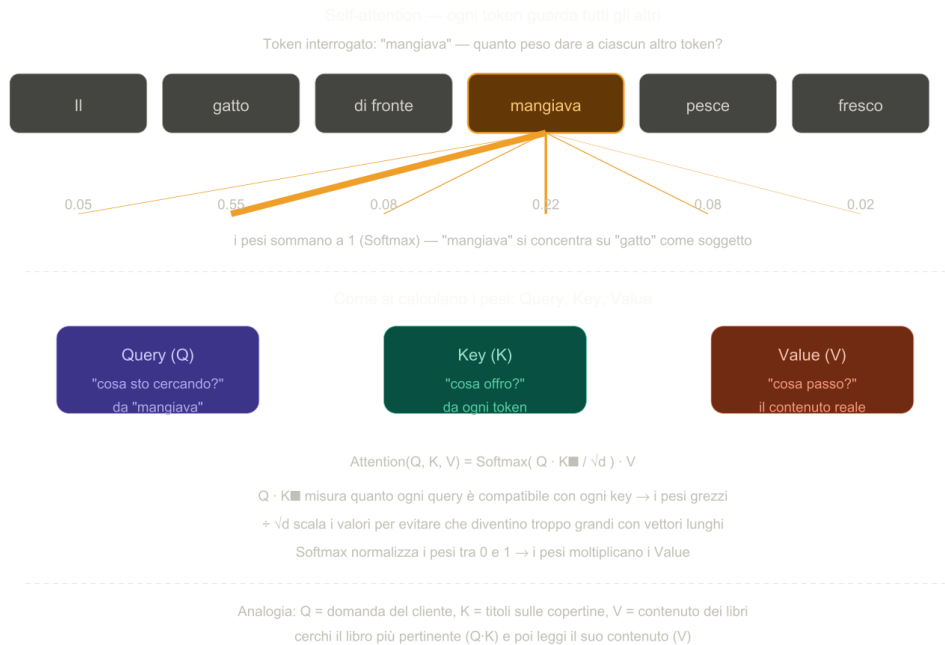


Figura 11.1 — Attenzione, architettura completa, e positional encoding.

Guarda la Figura 11.1. La parola "mangiava" deve capire chi è il suo soggetto. Il meccanismo di attenzione le permette di guardare tutti gli altri token contemporaneamente e assegnare loro dei pesi: peso alto a "gatto" (il soggetto), peso basso a "di fronte" (una clausola relativa irrilevante per il significato del verbo).

Questi pesi non sono fissi: vengono calcolati dinamicamente a partire dal contenuto dei token stessi. Questo è il punto chiave: la stessa parola "banca" riceverà pesi diversi in "vado in banca a prelevare" e in "mi siedo sulla banca del fiume", perché il contesto degli altri token è diverso.

Query, Key, Value: l'analogia della biblioteca

Per calcolare i pesi di attenzione, ogni token produce tre vettori distinti — **Query**, **Key** e **Value** — ciascuno ottenuto moltiplicando l'embedding del token per una matrice di pesi appresa durante il training.

Analogia: immagina una biblioteca. Entri con una domanda in mente (la tua **Query**). Ogni libro ha un titolo sulla copertina (**Key**) che descrive di cosa parla. Confronti la tua domanda con tutti i titoli, trovi i libri più pertinenti, e ne leggi il contenuto (**Value**). I pesi di attenzione sono quanto ogni libro è rilevante per la tua domanda.

La formula matematica è:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) \cdot V$$

Intuitivamente: $Q \cdot K^T$ calcola la compatibilità tra la query di ogni token e le key di tutti gli altri. Dividiamo per $\sqrt{d}$ (dove d è la dimensione dei vettori) per evitare che i prodotti diventino troppo grandi e il Softmax saturi verso valori estremi. Il Softmax trasforma i punteggi in pesi tra 0 e 1 che sommano a 1. Infine moltiplichiamo per i Value: il risultato è una media pesata dei Value di tutti i token.

```
import torch
import torch.nn.functional as F
import math
```



```

def attenzione(Q, K, V):
    """
    Self-attention dal paper originale.
    Q, K, V: tensori di shape (batch, n_token, d_model)
    """
    d = Q.shape[-1] # dimensione dei vettori

    # Punteggi grezzi: quanto ogni query è compatibile con ogni key
    scores = Q @ K.transpose(-2, -1) # (batch, n_token, n_token)
    scores = scores / math.sqrt(d) # scala per evitare saturazione del Softmax

    # Converti in pesi che sommano a 1
    pesi = F.softmax(scores, dim=-1) # (batch, n_token, n_token)

    # Media pesata dei Value
    output = pesi @ V # (batch, n_token, d_model)
    return output, pesi

# Esempio: 1 frase di 5 token, vettori di dim 64
batch, n_tok, d = 1, 5, 64
Q = torch.randn(batch, n_tok, d)
K = torch.randn(batch, n_tok, d)
V = torch.randn(batch, n_tok, d)

output, pesi = attenzione(Q, K, V)
print(output.shape) # torch.Size([1, 5, 64])
print(pesi.shape) # torch.Size([1, 5, 5]) — matrice 5x5 dei pesi di attenzione
print(pesi[0].sum(dim=-1)) # tensor([1., 1., 1., 1., 1.]) — ogni riga somma a 1

```

11.3 Multi-Head Attention: specializzazione parallela

Una singola testa di attenzione impara un tipo di relazione alla volta. Ma le parole in una frase hanno più tipi di relazioni simultanee: soggetto-verbo, aggettivo-sostantivo, pronomi-antecedente, e così via.

La **Multi-Head Attention** risolve questo eseguendo h teste di attenzione in parallelo, ognuna con le proprie matrici Q, K, V — quindi ognuna che impara a prestare attenzione a relazioni diverse. I loro output vengono poi concatenati e proiettati in un unico vettore.

Analogia: invece di leggere un testo con un solo criterio ("rilevanza grammaticale"), hai otto lettori specializzati che leggono contemporaneamente — uno per le relazioni soggetto-verbo, uno per le relazioni spaziali, uno per le co-referenze, e così via. Alla fine confronti le loro osservazioni.

```

import torch.nn as nn

class MultiHeadAttention(nn.Module):

    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0, "d_model deve essere divisibile per n_heads"

```

```

self.d_model = d_model
self.n_heads = n_heads
self.d_head = d_model // n_heads # dimensione di ogni testa

# Proiezioni lineari per Q, K, V e output
# Un'unica matrice per tutte le teste - più efficiente
self.W_q = nn.Linear(d_model, d_model, bias=False)
self.W_k = nn.Linear(d_model, d_model, bias=False)
self.W_v = nn.Linear(d_model, d_model, bias=False)
self.W_o = nn.Linear(d_model, d_model, bias=False)

def split_heads(self, x):
    """Divide il tensore in n_heads teste."""
    B, T, D = x.shape
    # Riorganizza: (B, T, D) → (B, n_heads, T, d_head)
    x = x.reshape(B, T, self.n_heads, self.d_head)
    return x.transpose(1, 2)

def forward(self, x, mask=None):
    B, T, D = x.shape

    # Proiezioni
    Q = self.split_heads(self.W_q(x)) # (B, n_heads, T, d_head)
    K = self.split_heads(self.W_k(x))
    V = self.split_heads(self.W_v(x))

    # Attenzione su ogni testa
    scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_head)

    # Maschera causale: il token i non può guardare i token j > i
    # Usata nel decoder per la generazione autoregressiva
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    pesi = F.softmax(scores, dim=-1)
    output = pesi @ V # (B, n_heads, T, d_head)

    # Ricombina le teste: (B, n_heads, T, d_head) → (B, T, D)
    output = output.transpose(1, 2).reshape(B, T, D)

    return self.W_o(output)

# Verifica
mha = MultiHeadAttention(d_model=256, n_heads=8)
x = torch.randn(4, 20, 256) # batch=4, seq=20, d_model=256
print(mha(x).shape) # torch.Size([4, 20, 256])

```

11.4 Positional Encoding: come il Transformer sa l'ordine delle parole

L'attenzione è una somma pesata: non ha nozione di ordine. Se rimescolassi i token della frase "Il gatto mangiò il topo" in ordine casuale, il meccanismo di attenzione produrrebbe lo stesso output (con pesi diversi, ma senza sapere quale parola veniva prima).

Per risolvere questo, si aggiunge al vettore di embedding di ogni token un **positional encoding** — un vettore che codifica la posizione del token nella sequenza. Il modello impara a usare questa informazione di posizione insieme al contenuto del token.

Il paper originale usa una combinazione di funzioni seno e coseno a frequenze diverse:

```
PE(pos, 2i)    = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1)  = cos(pos / 10000^(2i/d_model))
```

Intuitivamente: ogni posizione produce un pattern unico di valori seno/coseno — come un codice binario, ma continuo. La scelta di frequenze diverse permette al modello di distinguere sia posizioni vicine (frequenze alte) sia posizioni molto distanti (frequenze basse).

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        # Costruisce la matrice di positional encoding una volta sola
        PE = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        # Frequenze decrescenti per le diverse dimensioni
        div = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )

        PE[:, 0::2] = torch.sin(pos * div)    # dimensioni pari: seno
        PE[:, 1::2] = torch.cos(pos * div)    # dimensioni dispari: coseno

        # Registra come buffer: fa parte del modello ma non si addestra
        self.register_buffer('PE', PE.unsqueeze(0))    # (1, max_len, d_model)

    def forward(self, x):
        # Somma il positional encoding all'embedding di input
        x = x + self.PE[:, :x.size(1)]
        return self.dropout(x)
```

11.5 Il Feed-Forward Layer e la Layer Normalization

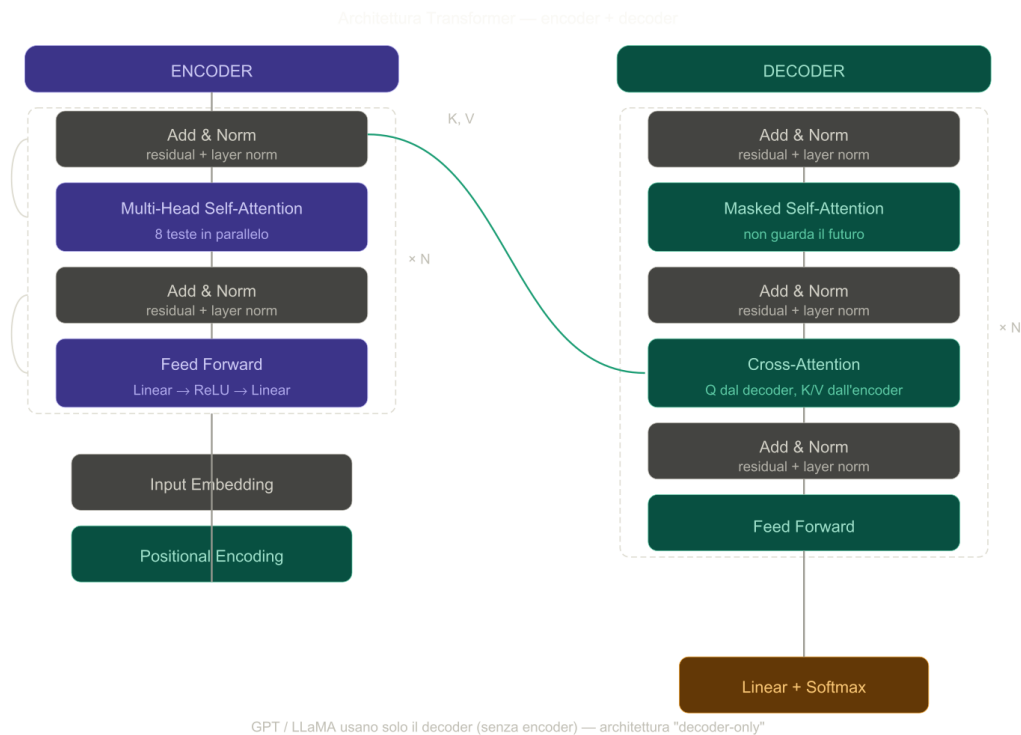


Figura 11.2 — GPT / LLaMA usano solo il decoder (senza encoder) — architettura "decoder-only"

Oltre all'attenzione, ogni blocco Transformer contiene altri due componenti, visibili nella Figura 11.2.

Feed-Forward Network (FFN): dopo l'attenzione, ogni token passa attraverso una piccola rete fully connected indipendente (la stessa per tutti i token, ma applicata a ciascuno separatamente). È composta da due layer lineari con una ReLU (o GELU nei modelli moderni) in mezzo, con una dimensione interna tipicamente 4 volte quella di `d_model`. L'FFN aggiunge capacità di trasformazione non-lineare che l'attenzione, essendo una somma pesata, non può fornire.

Layer Normalization e connessioni residue: come in ResNet, ogni sotto-layer (attenzione e FFN) è avvolto in una connessione residuale (`output = LayerNorm(x + sotto_layer(x))`). Questo stabilizza il training su reti molto profonde e permette al gradiente di fluire direttamente dall'output all'input senza attraversare tutti i layer.

```
class BloccoTransformer(nn.Module):
    """Un singolo blocco encoder del Transformer."""

    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attenzione = MultiHeadAttention(d_model, n_heads)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),  # GELU: più morbida di ReLU, preferita nei
            # moderni LLM
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout)
        )
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
```

```

self.dropout = nn.Dropout(dropout)

def forward(self, x, mask=None):
    # Sub-layer 1: attenzione con connessione residuale
    x = self.norm1(x + self.dropout(self.attenzione(x, mask)))
    # Sub-layer 2: feed-forward con connessione residuale
    x = self.norm2(x + self.ff(x))
    return x

```

11.6 L'architettura completa: encoder, decoder, decoder-only

Come mostra la Figura 11.2, il Transformer originale ha due parti.

L'encoder processa l'intera sequenza di input in parallelo. Ogni token può guardare tutti gli altri liberamente (attenzione bidirezionale). L'output è una sequenza di vettori ricchi di contesto — uno per ogni token di input. Usato per task di comprensione: classificazione, NER, question answering su testo dato.

Il decoder genera l'output token per token. Ha due tipi di attenzione: una **masked self-attention** (ogni token può guardare solo i token precedenti, non quelli futuri — non puoi vedere la risposta prima di generarla) e una **cross-attention** dove le Query vengono dal decoder e le Key/Value dall'encoder (il decoder "interroga" la rappresentazione dell'input prodotta dall'encoder). Usato per task di generazione: traduzione, riassunto, risposta a domande.

I modelli moderni come GPT, Claude e LLaMA usano solo il **decoder** (architettura *decoder-only*), senza encoder. Il motivo è semplice: un singolo decoder può fare tutto — sia comprendere l'input (che viene incluso nel contesto) sia generare l'output. È più semplice da scalare e funziona sorprendentemente bene su quasi tutti i task.

```

class Transformer(nn.Module):
    """Encoder-only Transformer per classificazione di testo."""

    def __init__(self, vocab_size, d_model=128, n_heads=4,
                  n_layers=3, d_ff=512, max_len=256, n_classi=2, dropout=0.1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_enc = PositionalEncoding(d_model, max_len, dropout)
        self.layers = nn.ModuleList([
            BloccoTransformer(d_model, n_heads, d_ff, dropout)
            for _ in range(n_layers)
        ])
        self.norm = nn.LayerNorm(d_model)
        # Usa il token [CLS] (posizione 0) per la classificazione
        self.classifier = nn.Linear(d_model, n_classi)

    def forward(self, x):
        # x: (batch, seq_len) - indici dei token
        out = self.pos_enc(self.embedding(x)) # embedding + posizione
        for layer in self.layers:
            out = layer(out)
        out = self.norm(out)
        # Prende il primo token come rappresentazione dell'intera sequenza
        return self.classifier(out[:, 0, :])

```

```
# Verifica
modello = Transformer(vocab_size=10000, d_model=128, n_heads=4, n_layers=3,
n_classi=2)
x = torch.randint(0, 10000, (8, 64)) # batch=8, sequenze di 64 token
print(modello(x).shape) # torch.Size([8, 2]) - 2 classi
totale = sum(p.numel() for p in modello.parameters())
print(f"Parametri: {totale:,}") # ~2.5M
```

11.7 Perché il Transformer ha vinto su tutto

Il successo del Transformer non è stato immediato. Quando è stato proposto nel 2017 era pensato per la traduzione automatica. Ma nel giro di pochi anni ha sostituito le RNN in quasi ogni dominio. Perché?

Parallelismo totale durante il training. Poiché non c'è ricorrenza, tutti i token vengono elaborati simultaneamente. Una GPU con migliaia di core può sfruttare appieno questa parallelizzazione. Il training è ordini di grandezza più veloce rispetto alle LSTM su dataset grandi.

Attenzione globale senza degradazione. Ogni token ha accesso diretto a ogni altro token con un singolo passo di calcolo, indipendentemente dalla distanza. Non c'è vanishing gradient nel senso delle RNN — le dipendenze a lungo raggio vengono catturate con la stessa facilità di quelle locali.

Scalabilità. Aggiungere più layer, più teste di attenzione, più dimensioni del modello migliora sistematicamente le performance, purché ci siano abbastanza dati. Le *scaling laws* (Capitolo 12) quantificano esattamente questa relazione. Le RNN non scalano allo stesso modo.

Flessibilità. Lo stesso blocco funziona per testo, immagini (Vision Transformer), audio, molecole, codice. Non serve ridisegnare l'architettura per ogni nuovo tipo di dato.

11.8 Esercizio guidato — mini-Transformer per classificazione di testo

Costruiamo un Transformer da zero per la classificazione di sentiment su IMDB, usando l'architettura definita nella sezione 11.6.

```
# =====
# Mini-Transformer per classificazione sentiment IMDB
# Esegui su Google Colab con GPU T4 attiva
# =====
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
from torch.utils.data import DataLoader, Dataset

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Dispositivo: {device}")

# --- Dati: IMDB via torchtext ---
# !pip install torchtext portalocker -q
```

```

from torchtext.datasets import IMDB
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator

tokenizer = get_tokenizer("basic_english")

# Costruisci il vocabolario sul training set
def yield_tokens(data_iter):
    for _, testo in data_iter:
        yield tokenizer(testo)

train_iter = IMDB(split="train")
vocab = build_vocab_from_iterator(
    yield_tokens(train_iter),
    specials=["<pad>", "<unk>"],
    max_tokens=20000
)
vocab.set_default_index(vocab["<unk>"])
print(f"Vocabolario: {len(vocab):,} token")

# Dataset con padding a lunghezza fissa
MAX_LEN = 256

class IMDBDataset(Dataset):
    def __init__(self, split):
        self.dati = list(IMDB(split=split))

    def __len__(self):
        return len(self.dati)

    def __getitem__(self, idx):
        label, testo = self.dati[idx]
        # Tokenizza e converti in indici
        token_ids = vocab(tokenizer(testo))[:MAX_LEN]
        # Padding a MAX_LEN
        padding = [0] * (MAX_LEN - len(token_ids))
        token_ids = token_ids + padding
        label_int = 1 if label == "pos" else 0
        return torch.tensor(token_ids, dtype=torch.long), torch.tensor(label_int)

train_ds = IMDBDataset("train")
test_ds = IMDBDataset("test")
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
print(f"Training: {len(train_ds)} | Test: {len(test_ds)}")

# --- Modello: riusa le classi definite nel capitolo ---
# (incolla qui MultiHeadAttention, PositionalEncoding,
# BloccoTransformer e Transformer dalle sezioni precedenti)

modello = Transformer(
    vocab_size = len(vocab),
    d_model = 128,
    n_heads = 4,
    n_layers = 3,
    d_ff = 512,
    max_len = MAX_LEN,
    n_classi = 2,

```

```

        dropout      = 0.1
    ).to(device)

    totale = sum(p.numel() for p in modello.parameters())
    print(f"Parametri: {totale:,}")

    # --- Training ---
    criterio      = nn.CrossEntropyLoss()
    ottimizzatore = torch.optim.AdamW(modello.parameters(), lr=3e-4, weight_decay=0.01)
    # AdamW: Adam con weight decay - standard per i Transformer

    # Warmup + cosine decay: il lr cresce linearmente per le prime N epoche
    # poi decresce. Questo stabilizza il training dei Transformer.
    def get_lr(step, d_model=128, warmup=4000):
        """Learning rate schedule dal paper originale."""
        step = max(step, 1)
        return d_model**(-0.5) * min(step**(-0.5), step * warmup**(-1.5))

    scheduler = torch.optim.lr_scheduler.LambdaLR(ottimizzatore, get_lr)

    storia_acc = []
    passo      = 0

    for epoca in range(10):
        modello.train()
        loss_ep, corr_ep = 0, 0

        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            ottimizzatore.zero_grad()
            logits = modello(X)
            loss    = criterio(logits, y)
            loss.backward()
            # Gradient clipping: utile anche per i Transformer durante il warmup
            torch.nn.utils.clip_grad_norm_(modello.parameters(), 1.0)
            ottimizzatore.step()
            scheduler.step()
            loss_ep += loss.item()
            corr_ep += (logits.argmax(1) == y).sum().item()
            passo   += 1

        modello.eval()
        corretti = 0
        with torch.no_grad():
            for X, y in test_loader:
                X, y = X.to(device), y.to(device)
                corretti += (modello(X).argmax(1) == y).sum().item()

        acc = corretti / len(test_ds)
        storia_acc.append(acc)
        print(f"Epoca {epoca+1:2d} | "
              f"Loss: {loss_ep/len(train_loader):.4f} | "
              f"Acc train: {corr_ep/len(train_ds)*100:.1f}% | "
              f"Acc test: {acc*100:.2f}%")

```

Con questa architettura da ~2.5M di parametri dovresti ottenere circa **87–89% di accuracy** su IMDB dopo 10 epoche — paragonabile alla LSTM del Capitolo 10, ma con training più veloce grazie al parallelismo.

Riepilogo del Capitolo 11

Hai imparato:

- **Perché il Transformer supera le RNN:** elaborazione parallela di tutti i token, attenzione globale senza degradazione del gradiente, scalabilità eccellente con più dati e parametri.
- **Il meccanismo di self-attention:** ogni token calcola pesi di attenzione verso tutti gli altri. I pesi dipendono dal contenuto, non dalla posizione — stessa parola, contesti diversi, pesi diversi.
- **Query, Key, Value:** Q = cosa cerco, K = cosa offro, V = cosa passo. La compatibilità $Q \cdot K$ determina quanto del V di ogni token viene incluso nell'output.
- **Multi-Head Attention:** h teste in parallelo, ognuna che impara a prestare attenzione a relazioni diverse. Output concatenati e riproiettati.
- **Positional Encoding:** vettori seno/coseno sommati agli embedding per introdurre informazione di posizione senza ricorrenza.
- **Architettura completa:** encoder (attenzione bidirezionale) + decoder (masked self-attention + cross-attention). I modelli moderni come GPT usano solo il decoder.
- **Perché ha vinto:** parallelismo totale, attenzione globale, scalabilità, flessibilità cross-domain.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: implementare la **causal mask** per trasformare l'encoder in un decoder autoregressivo, e usarlo per generare testo carattere per carattere.

Traccia:

La generazione autoregressiva funziona così: dato un contesto di token, il modello predice il token successivo. Il token predetto viene aggiunto al contesto, e si ripete. La **causal mask** garantisce che durante il training il token alla posizione i possa guardare solo le posizioni $0 \dots i-1$, non il futuro.

19. Costruisci la causal mask e verifica che funzioni:

```
def causal_mask(seq_len, device):
    """Matrice triangolare inferiore: 1 dove possiamo guardare, 0 dove no."""
    mask = torch.tril(torch.ones(seq_len, seq_len, device=device))
    return mask.unsqueeze(0).unsqueeze(0)  # (1, 1, seq_len, seq_len)

# Verifica visiva: deve essere triangolare inferiore
print(causal_mask(5, "cpu").squeeze())
```

20. Addestra il Transformer (con causal mask attiva) a predire il carattere successivo nel testo di *La Divina Commedia* (scaricabile da Project Gutenberg). Usa un vocabolario a livello di carattere — molto più semplice da costruire e sufficiente per capire la generazione.

```
# Scarica il testo
import urllib.request
url = "https://www.gutenberg.org/cache/epub/1012/pg1012.txt"
testo = urllib.request.urlopen(url).read().decode("utf-8")
# Costruisci il vocabolario: ogni carattere unico
chars = sorted(set(testo))
```

```
ch2idx = {c: i for i, c in enumerate(chars)}  
idx2ch = {i: c for i, c in enumerate(chars)}
```

21. Scrivi la funzione di generazione. Dato un testo iniziale (seed), genera i 200 caratteri successivi con **temperature sampling**: dividi i logits per un parametro T prima del Softmax. Con $T=0.5$ il testo è più prevedibile, con $T=1.5$ è più creativo e caotico.

```
def genera(modello, seed, n_caratteri, temperatura=1.0):  
    modello.eval()  
    # ... da completare
```

22. Sperimenta con temperature diverse (0.3, 0.8, 1.2) e riporta esempi di testo generato. A quale temperatura il testo è più coerente? Quando inizia a diventare incomprensibile?
23. **Domanda di riflessione:** confronta il testo generato dopo 5 epoche e dopo 20 epoche. Il modello ha imparato strutture più lunghe (es. parole complete invece di sillabe)? Questo ti dice qualcosa su cosa impara nei layer iniziali vs. quelli finali?

Consegna: il notebook con le curve di training, almeno 3 esempi di testo generato a temperature diverse, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Large Language Models — scopriremo come un Transformer addestrato su trilioni di token diventa un assistente capace di ragionare, scrivere codice e rispondere a domande in linguaggio naturale.

Capitolo 12

Large Language Models (LLM)

"Un LLM non sa cosa significa una parola nel senso in cui lo sai tu. Sa che quella parola compare spesso vicino ad altre parole specifiche — e da questa semplice osservazione, ripetuta su trilioni di esempi, emerge qualcosa che assomiglia alla comprensione."

12.1 Cosa rende un LLM diverso dai modelli visti finora

Nei capitoli precedenti hai costruito modelli che imparano a classificare immagini, predire serie temporali, analizzare il sentiment. Tutti hanno in comune una caratteristica: vengono addestrati per un task specifico, con un dataset specifico, verso un obiettivo preciso.

Un **Large Language Model** nasce da un'idea radicalmente diversa: addestrare un unico modello enorme su una quantità di testo così vasta da costringere il modello ad acquisire, come effetto collaterale, una rappresentazione profonda del linguaggio, della conoscenza del mondo e della struttura del ragionamento.

Il task di addestramento sembra banale: predire il token successivo. Data la sequenza "Il sole tramonta a", il modello deve assegnare probabilità alte a "ovest", "ponente", "occidente" e probabilità basse a "martedì" o "cane". Ripetuto su trilioni di token, questo task banale costringe il modello a imparare grammatica, fatti storici, ragionamento logico, stile letterario — tutto ciò che serve per predire bene il token successivo in qualsiasi contesto.

12.2 Tokenizzazione BPE: come il testo diventa numeri

Prima che il testo entri in un LLM, deve essere convertito in numeri. Questo processo si chiama **tokenizzazione**, e la tecnica standard si chiama **Byte Pair Encoding (BPE)**.

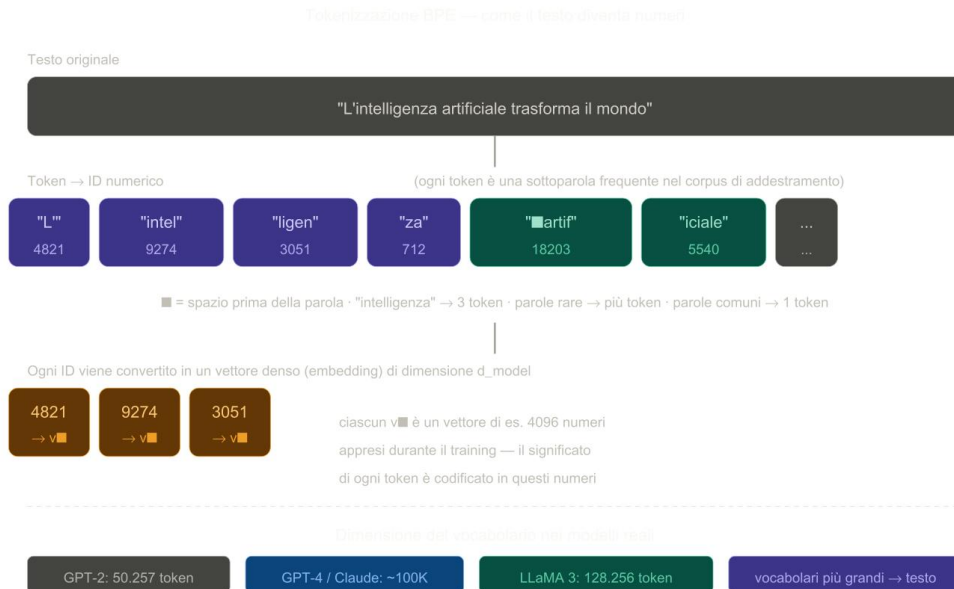


Figura 12.1 — Tokenizzazione BPE — come il testo diventa numeri.

Come mostra la Figura 12.1, i token non sono né caratteri né parole intere: sono **sottoparole** frequenti nel corpus di addestramento. La parola "intelligenza" diventa tre token: "intel" + "ligen" + "za". La parola "il" è un solo token. Una parola rara come "anticostituzionalmente" potrebbe diventare 5 o 6 token.

Perché non usare le parole come token? Perché il vocabolario delle parole italiane (e di tutte le lingue) è enorme e cresce continuamente. Con un vocabolario a parole, ogni parola nuova o mal ortografata sarebbe sconosciuta al modello. BPE usa un vocabolario fisso di 50.000–130.000 sottoparole che copre qualsiasi testo immaginabile.

Ecco come si usa in pratica il tokenizzatore di un modello reale:

```
# pip install transformers
from transformers import AutoTokenizer

# Carica il tokenizzatore di LLaMA 3 (o qualsiasi modello HuggingFace)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")
# Nota: per i modelli gated (come LLaMA) serve accettare la licenza su HuggingFace.
# Per i test puoi usare "gpt2" che è pubblico.

tokenizer_gpt2 = AutoTokenizer.from_pretrained("gpt2")

testo = "L'intelligenza artificiale trasforma il mondo"

# Tokenizza
token_ids = tokenizer_gpt2.encode(testo)
token_str = tokenizer_gpt2.convert_ids_to_tokens(token_ids)

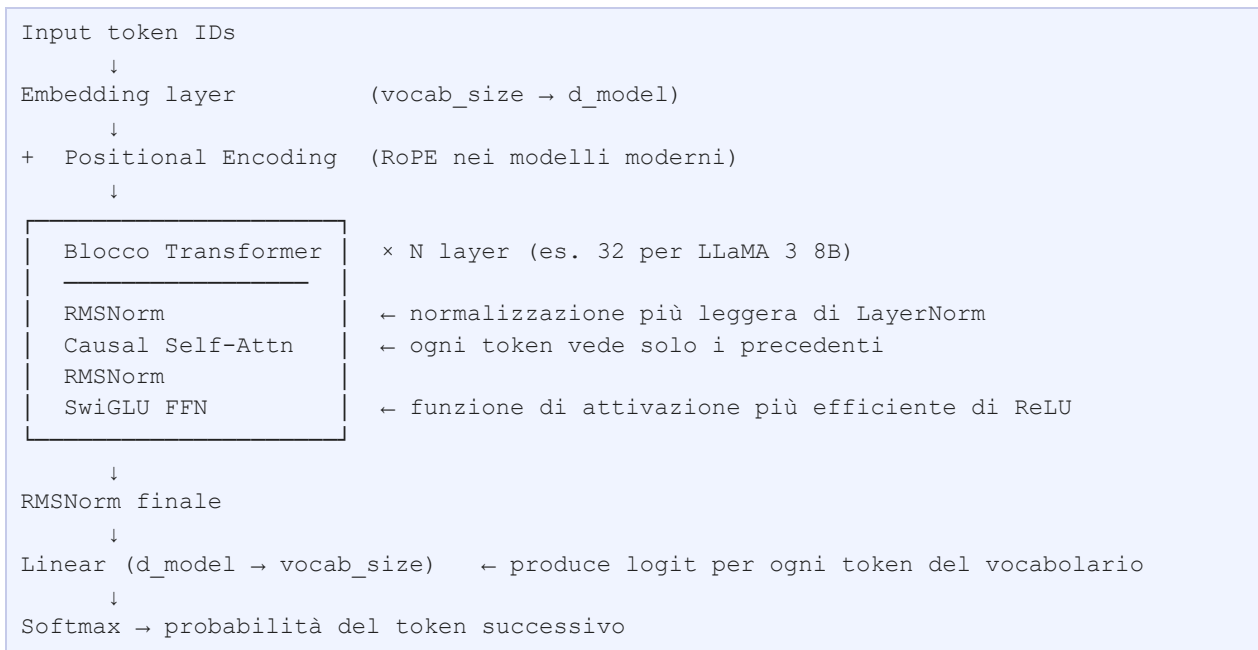
print("Token IDs:", token_ids)
print("Token stringhe:", token_str)
print(f"Caratteri: {len(testo)} → Token: {len(token_ids)}")

# Ritorna al testo originale
testo_ricostruito = tokenizer_gpt2.decode(token_ids)
print("Testo ricostruito:", testo_ricostruito)
```

Una cosa importante da capire sui token: **il contesto di un LLM si misura in token, non in parole o caratteri**. Un modello con "context window" di 128.000 token può tenere in memoria circa 100.000 parole — più o meno l'equivalente di un romanzo. I token sono anche l'unità con cui si misura il costo delle API: ogni chiamata ti viene addebitata in base ai token di input + output.

12.3 Architettura interna: GPT e LLaMA step by step

Un LLM moderno come GPT o LLaMA è un Transformer **decoder-only** (come hai visto nel Capitolo 11) con alcune modifiche rispetto all'architettura originale del 2017.



Le tre differenze principali rispetto al Transformer originale nei modelli moderni:

RMSNorm invece di LayerNorm. Più semplice e leggermente più veloce. Normalizza i vettori dividendo per la loro norma quadratica media, senza calcolare media e varianza.

RoPE (Rotary Positional Embedding). Invece di sommare un vettore di posizione fisso all'embedding, ruota i vettori Q e K nell'operazione di attenzione in modo dipendente dalla posizione relativa. Generalizza meglio a sequenze più lunghe di quelle viste durante il training.

SwiGLU. Una funzione di attivazione per il Feed-Forward Network che combina una moltiplicazione elemento per elemento con una funzione Swish. Empiricamente migliora la qualità del modello rispetto a GELU e ReLU a parità di parametri.

12.4 Pre-training: cosa impara un modello su trilioni di token

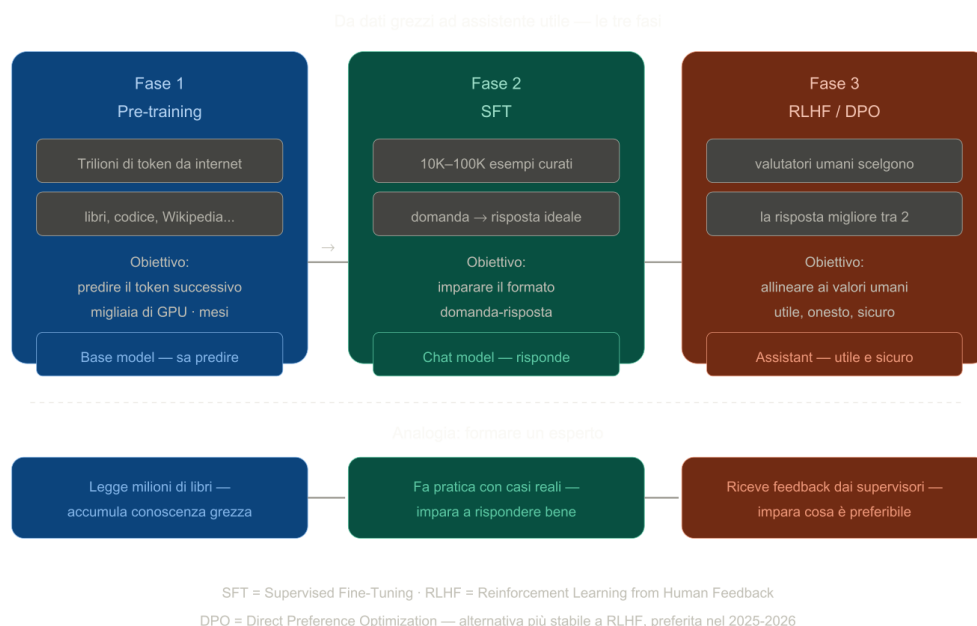


Figura 12.2 — Da dati grezzi ad assistente utile — le tre fasi.

La prima fase — e di gran lunga la più costosa — è il **pre-training**. Come mostra la Figura 12.2, il modello viene addestrato su un corpus enorme (Common Crawl, libri, Wikipedia, codice GitHub, paper scientifici, e molto altro) con un unico obiettivo: predire il token successivo.

Il costo è reale: addestrare LLaMA 3 8B ha richiesto circa 1.3 milioni di GPU-ore su hardware specializzato. Per un'azienda come Anthropic o OpenAI, un singolo run di pre-training di un modello grande può costare decine di milioni di dollari.

Durante il pre-training il modello non riceve alcuna istruzione su cosa fare — impara semplicemente a modellare la distribuzione del testo. Eppure, emergono capacità sorprendenti:

- Sa rispondere a domande perché ha visto milioni di testi in formato Q&A.
- Sa scrivere codice perché ha visto miliardi di righe di codice commentato.
- Sa ragionare per passi perché ha visto spiegazioni matematiche e scientifiche.
- Sa tradurre perché ha visto testi multilingua e traduzioni affiancate.

Tutte queste capacità non vengono insegnate esplicitamente — emergono come conseguenza del task di predizione su dati sufficientemente diversi e abbondanti.

12.5 Da modello linguistico ad assistente: SFT, RLHF, DPO

Un modello appena pre-addestrato sa completare il testo, ma non sa rispondere a domande nel modo in cui ti aspetti. Se scrivi "Come si fa la pasta al pomodoro?", potrebbe rispondere con un'altra domanda, o con un elenco di ingredienti senza contesto, o con qualcosa di completamente diverso — perché il suo training lo ha abituato a *continuare* il testo, non a *rispondere*.

Le fasi successive trasformano il base model in un assistente. Come illustra la Figura 12.2:

SFT — Supervised Fine-Tuning. Il modello viene fine-tuned su un dataset di esempi curati manualmente: coppie (domanda, risposta ideale) scritte o selezionate da esseri umani. Il dataset è piccolo rispetto al pre-training (da 10.000 a qualche milione di esempi), ma ogni esempio è di alta qualità. L'obiettivo è insegnare al modello il formato e lo stile di una conversazione utile.

RLHF — Reinforcement Learning from Human Feedback. Valutatori umani confrontano due risposte generate dal modello per la stessa domanda e scelgono quale preferiscono. Da queste preferenze si addestra un **reward model** — un classificatore che impara a predire quale risposta un umano preferirebbe. Il LLM viene poi ottimizzato via reinforcement learning per massimizzare il reward del reward model.

DPO — Direct Preference Optimization. Un'alternativa a RLHF più stabile e più semplice da implementare, diventata la tecnica preferita nel 2024–2026. Invece di addestrare un reward model separato, ottimizza direttamente il LLM sulle coppie di preferenze umane con una loss derivata matematicamente dalla formulazione RL. Stessa qualità, meno instabilità numerica.

12.6 Scaling laws e capacità emergenti

Nel 2020, Kaplan et al. (OpenAI) hanno pubblicato uno dei risultati più importanti nella storia degli LLM: le **scaling laws**. La scoperta è che la qualità di un LLM (misurata come cross-entropy loss sul testo di test) segue una legge di potenza prevedibile in funzione di tre variabili: numero di parametri, quantità di dati di training, e quantità di compute (FLOP) usata.

In pratica: **raddoppiare i parametri, i dati o il compute riduce la loss in modo prevedibile**. Questo significa che prima ancora di addestrare un modello si può stimare quanto sarà buono — e quanto costerà addestrarlo.

La versione aggiornata di Hoffmann et al. (2022), nota come **Chinchilla**, ha mostrato che molti modelli erano "compute-optimal" con troppe poche dati: per un modello da 70B di parametri, il training ottimale richiede circa 1.4 trilioni di token, non i 300 miliardi usati in precedenza.

Accanto alle scaling laws, c'è il fenomeno delle **capacità emergenti**: alcune abilità non migliorano gradualmente con la dimensione del modello, ma compaiono quasi di colpo superata una certa soglia di scala. Modelli piccoli non fanno aritmetica a più cifre; superati certi miliardi di parametri, la capacità appare quasi bruscamente. Lo stesso per il ragionamento multi-step, la traduzione di lingue rare, la comprensione del codice complesso.

Questo fenomeno non è ancora completamente compreso, ma ha una conseguenza pratica importante: non puoi sempre prevedere quali capacità avrà un modello più grande prima di addestrarlo.

12.7 Panoramica modelli 2026: open source vs. closed

Il panorama dei modelli è cambiato radicalmente dal 2022 ad oggi. Ecco come orientarsi.

Modelli closed (API-only): non puoi scaricare i pesi, accedi solo via API.

Modello	Provider	Punti di forza	----- ----- -----	Claude 3.5 / 3.7 Sonnet
Anthropic	Ragionamento, codice, sicurezza, contesto lungo	GPT-4o	OpenAI	Multimodale,

ecosistema tools maturo | | Gemini 1.5 / 2.0 Pro | Google | Contesto fino a 1M token, integrazione Google |

Modelli open (pesi scaricabili): puoi scaricare i pesi, eseguire in locale, fine-tunare.

| Modello | Parametri | Quando usarlo | |-----|-----|-----| | LLaMA 3.1 8B | 8B | Default: buon bilanciamento qualità/costo, gira su GPU consumer | | LLaMA 3.1 70B | 70B | Quando servono capacità paragonabili ai closed su hardware server | | Mistral 7B / Mixtral 8x7B | 7B / 47B | Attivi | Veloce, efficiente, ottimo per deployment su hardware limitato | | Qwen 2.5 72B | 72B | Eccellente su matematica e codice, forte supporto multilingua | | Phi-3 / Phi-4 | 3.8B–14B | Small models di qualità sorprendente, ottimi per device edge |

Regola pratica per scegliere:

Inizia con un modello closed via API (Claude o GPT-4o) per prototipare — è più veloce. Quando il prototipo funziona, valuta se il costo dell'API a scala giustifica il passaggio a un modello open in auto-hosting. Se hai requisiti di privacy o latenza, parti direttamente dall'open.

12.8 Esercizio guidato — costruisci il tuo chatbot con l'API Anthropic

Costruiamo un chatbot funzionante che mantiene la cronologia della conversazione, ha un system prompt personalizzabile e gestisce i limiti di token in modo robusto.

```
# pip install anthropic
import anthropic
import os

# --- Setup client ---
# Imposta la variabile d'ambiente con la tua API key
# In Colab: usa i Secrets (icona della chiave a sinistra) per non esporre la key nel codice
client = anthropic.Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

# --- Configurazione del chatbot ---
SYSTEM_PROMPT = """Sei un assistente esperto di machine learning e deep learning.
Rispondi in italiano, in modo chiaro e con esempi pratici quando utile.
Quando non sei sicuro di qualcosa, dillo esplicitamente invece di inventare."""

MAX_TOKENS_RISPOSTA = 1024
MODELLO = "claude-sonnet-4-20250514"
MAX_STORIA = 20 # mantieni al massimo 20 messaggi in memoria

def tronca_storia(storia, max_msg=MAX_STORIA):
    """
    Mantieni solo gli ultimi max_msg messaggi per non superare il context limit.
    Conserva sempre il primo messaggio se è dell'utente (contesto iniziale).
    """
    if len(storia) > max_msg:
        storia = storia[-max_msg:]
    return storia
```



```

def chatbot(storia, messaggio_utente):
    """
    Invia un messaggio e ricevi la risposta, aggiornando la storia.
    Ritorna (risposta, storia_aggiornata).
    """
    # Aggiungi il messaggio dell'utente alla storia
    storia.append({"role": "user", "content": messaggio_utente})

    try:
        risposta = client.messages.create(
            model = MODELLO,
            max_tokens = MAX_TOKENS_RISPOSTA,
            system = SYSTEM_PROMPT,
            messages = storia
        )

        testo_risposta = risposta.content[0].text

        # Aggiungi la risposta alla storia
        storia.append({"role": "assistant", "content": testo_risposta})

        # Tronca se necessario
        storia = tronca_storia(storia)

        return testo_risposta, storia

    except anthropic.APIError as e:
        # Gestione errori comuni
        print(f"Errore API: {e}")
        # Rimuovi l'ultimo messaggio utente per non corrompere la storia
        storia.pop()
        return None, storia

# --- Loop conversazione ---
def avvia_chatbot():
    storia = []
    print("Chatbot ML pronto. Scrivi 'esci' per terminare.\n")
    print("-" * 50)

    while True:
        domanda = input("Tu: ").strip()

        if domanda.lower() in ("esci", "exit", "quit"):
            print("Arrivederci!")
            break

        if not domanda:
            continue

        risposta, storia = chatbot(storia, domanda)

        if risposta:
            print(f"\nAssistente: {risposta}\n")
            print(f"[Messaggi in memoria: {len(storia)}]")
            print("-" * 50)
        else:

```

```
print("Errore nella risposta. Riprova.\n")
```

```
avvia_chatbot()
```

Variante: chatbot con cambio di personalità a runtime

```
# Puoi cambiare il comportamento del chatbot cambiando il system prompt
# senza riaddestrare nulla – è uno dei vantaggi principali degli LLM

PERSONALITA = {
    "docente": """Sei un professore universitario di informatica.
                Spiega i concetti con rigore accademico e riferimenti teorici.""",

    "semplice": """Sei un amico esperto di tecnologia.
                  Spiega tutto in modo semplice, con analogie della vita quotidiana.
                  Evita il gergo tecnico quando possibile.""",

    "socratico": """Rispondi sempre con domande che guidano l'utente a trovare
                   la risposta da solo, invece di darla direttamente."""
}

def chatbot_con_persona(storia, messaggio, persona="docente"):
    system = PERSONALITA.get(persona, PERSONALITA["docente"])
    storia.append({"role": "user", "content": messaggio})
    risposta = client.messages.create(
        model=MODELLO, max_tokens=MAX_TOKENS_RISPOSTA,
        system=system, messages=storia
    )
    testo = risposta.content[0].text
    storia.append({"role": "assistant", "content": testo})
    return testo, storia

# Esempio di uso
storia = []
risposta, storia = chatbot_con_persona(
    storia,
    "Cos'è il gradient descent?",
    persona="semplice"
)
print(risposta)
```

Riepilogo del Capitolo 12

Hai imparato:

- **Cosa distingue un LLM:** addestrato su trilioni di token con l'obiettivo di predire il token successivo, da cui emergono capacità generali di linguaggio, conoscenza e ragionamento.
- **Tokenizzazione BPE:** il testo viene spezzato in sottoparole frequenti. Il contesto si misura in token, non in parole.
- **Architettura interna:** Transformer decoder-only con RMSNorm, RoPE per il positional encoding e SwiGLU come funzione di attivazione.

- **Pre-training:** la fase più costosa — predizione del token successivo su enormi corpus eterogenei.
- **SFT → RLHF/DPO:** trasformano il base model in un assistente utile, onesto e sicuro tramite esempi curati e preferenze umane.
- **Scaling laws:** qualità e costo scalano in modo prevedibile con parametri, dati e compute. Le capacità emergenti compaiono oltre certe soglie di scala.
- **Panoramica 2026:** come scegliere tra modelli closed (Claude, GPT-4o) e open (LLaMA 3, Mistral, Qwen) in base al task, al budget e ai requisiti di privacy.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un chatbot specializzato con memoria persistente e valutazione automatica della qualità delle risposte.

Traccia:

24. Costruisci un chatbot esperto in un dominio a tua scelta (cucina, storia, diritto, medicina veterinaria — qualsiasi cosa ti interessi). Scrivi un system prompt dettagliato che definisca: il ruolo del modello, il tono, le limitazioni (cosa non deve rispondere), e il formato preferito delle risposte.
25. Implementa la **memoria persistente**: salva la cronologia su file JSON in modo che la conversazione continui tra una sessione e l'altra.

```
import json, os

def carica_storia(file="storia_chat.json"):
    if os.path.exists(file):
        with open(file) as f:
            return json.load(f)
    return []

def salva_storia(storia, file="storia_chat.json"):
    with open(file, "w") as f:
        json.dump(storia, f, ensure_ascii=False, indent=2)
```

26. Implementa un **valutatore automatico**: dopo ogni risposta, fai una seconda chiamata all'API con un prompt che chiede al modello di valutare la propria risposta su tre dimensioni (accuratezza, completezza, chiarezza) con un punteggio da 1 a 5. Visualizza i punteggi all'utente.
27. Dopo 20 conversazioni, analizza i dati: quali domande hanno ricevuto i punteggi più bassi? Il system prompt può essere migliorato per gestire quelle tipologie di domande meglio?
28. **Domanda di riflessione:** prova a modificare il system prompt aggiungendo la frase "Pensa step by step prima di rispondere". I punteggi medi cambiano? Questo è un esempio di chain-of-thought prompting — lo approfondiremo nel prossimo capitolo.

Consegna: il notebook con il chatbot funzionante, la storia delle conversazioni in JSON, i grafici dei punteggi di qualità nel tempo, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Prompt Engineering Avanzato — impareremo a comunicare con i modelli in modo preciso e sistematico, trasformando le istruzioni in linguaggio naturale in uno strumento di controllo affidabile.

Capitolo 13

Prompt Engineering Avanzato

"Un LLM è come uno strumento di precisione: funziona bene se sai come tenerlo. Il prompt engineering è imparare a tenerlo."

13.1 Cosa significa davvero "fare prompt engineering"

Nel capitolo precedente hai costruito un chatbot che funziona. Ma "funziona" è vago. Funziona per il caso che hai testato a mano — ma funziona in modo affidabile su cento input diversi? Risponde sempre nel formato che ti serve? Non produce mai output che rompono il sistema a valle?

Il **prompt engineering** è la disciplina di progettare istruzioni per LLM che producano output utili, nel formato corretto, in modo affidabile e misurabile. Non è scrivere istruzioni creative sperando che funzionino: è un ciclo iterativo di progettazione, test e misurazione — esattamente come il ciclo di sviluppo del software.



Figura 13.1 — La gerarchia delle tecniche di prompting.

La Figura 13.1 mostra la gerarchia delle tecniche: ogni livello aggiunge contesto al prompt, aumenta il numero di token usati e migliora la qualità su task complessi. La regola pratica è usare sempre il minimo necessario — zero-shot se funziona, few-shot solo se zero-shot non è sufficiente, chain-of-thought solo per i task che richiedono ragionamento esplicito.

13.2 Zero-shot, few-shot e chain-of-thought

Zero-shot

L'approccio più semplice: descrivi il task e basta. Funziona sorprendentemente bene per task ben definiti su modelli grandi.

```
import anthropic
client = anthropic.Anthropic()

# Zero-shot: solo l'istruzione
risposta = client.messages.create(
    model = "claude-sonnet-4-20250514",
    max_tokens = 256,
    messages = [{
        "role": "user",
        "content": "Classifica il sentiment di questa recensione come POSITIVO, NEGATIVO o NEUTRO.\n\nRecensione: 'Il prodotto è arrivato in anticipo ma la confezione era danneggiata.'\n\nRispondi con una sola parola."
    }]
)
print(risposta.content[0].text) # NEGATIVO (o NEUTRO, dipende dal modello)
```

Few-shot

Aggiungi 2–5 esempi input→output nel prompt. Utile quando vuoi guidare il formato esatto della risposta, o quando il task ha una definizione non ovvia.

```
prompt_few_shot = """Classifica il sentiment di ogni recensione.
Rispondi esattamente nel formato: [ETICHETTA] – [motivazione in 5 parole]

Esempi:
Recensione: "Qualità eccellente, arrivato in 24 ore."
Risposta: [POSITIVO] – qualità e consegna ottimi

Recensione: "Non funziona, assistenza ignorante."
Risposta: [NEGATIVO] – prodotto rotto, assistenza assente

Recensione: "Come da descrizione, niente di speciale."
Risposta: [NEUTRO] – corrisponde alle aspettative, anonimo

Ora classifica:
Recensione: "Bella grafica ma il gameplay diventa ripetitivo dopo 2 ore."
Risposta: """

risposta = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=64,
    messages=[{"role": "user", "content": prompt_few_shot}]
)
print(risposta.content[0].text)
# [NEGATIVO] – bella estetica, gameplay delude rapidamente
```

Il few-shot non migliora la "comprensione" del modello — il modello capisce già il task. Migliora la **consistenza del formato**: gli esempi mostrano esattamente come deve essere strutturata la risposta.

Chain-of-Thought (CoT)

Per task che richiedono ragionamento multi-step — matematica, logica, analisi — chiedere al modello di "pensare passo per passo" prima di rispondere migliora significativamente la qualità. Il modello non sta "pensando" davvero: produce token che rappresentano passaggi intermedi, e questi token diventano contesto per i token successivi, guidando verso la risposta corretta.

```
# Senza CoT — il modello risponde direttamente, spesso sbagliando
prompt_diretto = """Un treno parte da Milano alle 8:00 e arriva a Roma alle 12:30.
Un secondo treno parte da Roma alle 9:15 e arriva a Milano alle 13:45.
A che ora si incrociano, assumendo che la distanza sia percorsa a velocità
costante?"""

# Con CoT — guidiamo il ragionamento esplicito
prompt_cot = """Un treno parte da Milano alle 8:00 e arriva a Roma alle 12:30.
Un secondo treno parte da Roma alle 9:15 e arriva a Milano alle 13:45.
A che ora si incrociano, assumendo che la distanza sia percorsa a velocità costante?

Ragiona passo per passo:
1. Calcola la durata del viaggio di ciascun treno.
2. Calcola la velocità relativa (in frazione della distanza per ora).
3. Imposta l'equazione per trovare il momento dell'incontro.
4. Rispondi."""

risposta = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=512,
    messages=[{"role": "user", "content": prompt_cot}]
)
print(risposta.content[0].text)
```

Quando CoT aiuta davvero: su task di ragionamento matematico, logico o multi-step. Su task semplici di classificazione o generazione, CoT aggiunge token inutili senza migliorare la qualità — e aumenta il costo.

13.3 Il system prompt: istruire il modello in modo robusto

Il **system prompt** è il posto giusto per definire il comportamento permanente del modello: ruolo, tono, limitazioni, formato di output, e istruzioni che devono valere per tutta la conversazione. A differenza dei messaggi utente, il system prompt non è visibile all'utente finale e viene applicato a ogni risposta.

Un system prompt efficace risponde a queste domande:

```
SYSTEM_PROMPT_STRUTTURATO = """
# Ruolo
Sei un assistente per l'analisi di contratti legali in italiano.
Hai competenze di diritto contrattuale italiano ma non sei un avvocato —
ricordalo sempre nelle risposte quando dai interpretazioni.

# Cosa fare
- Identifica le clausole rilevanti per la domanda dell'utente.
- Cita sempre il numero di articolo o clausola quando possibile.
- Spiega in linguaggio semplice cosa significa ogni clausola.
```

```
# Cosa NON fare
- Non dare pareri legali definitivi.
- Non inventare clausole non presenti nel testo.
- Non rispondere a domande fuori dal testo del contratto fornito.

# Formato delle risposte
Struttura ogni risposta così:
1. Clausola rilevante (con riferimento)
2. Significato in linguaggio semplice
3. Eventuali implicazioni pratiche
4. Raccomandazione: consulta un avvocato se [condizione]

# Tono
Professionale ma accessibile. Usa frasi brevi. Evita latinismi.
"""
```

Alcune regole pratiche per system prompt robusti:

Sii esplicito sul formato, non solo sul contenuto. "Rispondi in JSON" è meglio di "rispondi in modo strutturato". "Usa massimo 3 paragrafi" è meglio di "rispondi brevemente".

Definisci i casi limite. Cosa deve fare il modello se l'utente chiede qualcosa fuori scope? "Se la domanda non riguarda contratti legali, rispondi: 'Posso aiutarti solo con l'analisi del contratto fornito.'" è molto più affidabile che sperare che il modello lo intuisca.

Usa struttura markdown nel system prompt. I modelli moderni sono addestrati su enormi quantità di testo markdown — usare intestazioni e liste nel system prompt migliora la comprensione del prompt stesso.

13.4 Structured output: ottenere JSON garantito

Uno dei casi d'uso più comuni in produzione è estrarre informazioni strutturate dal testo libero e passarle a un sistema a valle (database, API, UI). Il problema è che i modelli tendono a produrre JSON "quasi corretto" — con virgole mancanti, chiavi tra virgolette singole, o testo fuori dal JSON.

Ci sono tre tecniche, in ordine di affidabilità crescente.

Tecnica 1 — istruzione nel prompt. Semplice ma inaffidabile su input complessi.

```
prompt = """Estrai le informazioni dal seguente testo e restituisci SOLO JSON valido,
senza testo prima o dopo, senza markdown, senza backtick.

Schema: {"nome": string, "email": string, "azienda": string, "ruolo": string}

Testo: "Ciao, sono Marco Bianchi, CTO di TechSrl. Puoi scrivermi a marco@techsrl.it"

JSON: """
```

Tecnica 2 — prefill della risposta. Forzi il modello a iniziare la risposta con `{`, così non può produrre testo introduttivo.

```
risposta = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=512,
    messages=[
        {"role": "user", "content": prompt},
```



```

        {"role": "assistant", "content": "{}"} # prefill: forza l'inizio con {
    ]
)
# La risposta inizia già con { - concatena
json_str = "{" + risposta.content[0].text
import json
dati = json.loads(json_str)
print(dati)

```

Tecnica 3 — tool use / function calling. Il modo più affidabile: definisci lo schema come strumento e il modello è vincolato a rispettarlo. L'API garantisce che l'output sia JSON valido con lo schema corretto.

```

strumenti = [{
    "name": "estrai_contatto",
    "description": "Estrae informazioni di contatto da un testo",
    "input_schema": {
        "type": "object",
        "properties": {
            "nome": {"type": "string", "description": "Nome completo"},
            "email": {"type": "string", "description": "Indirizzo email"},
            "azienda": {"type": "string", "description": "Nome dell'azienda"},
            "ruolo": {"type": "string", "description": "Ruolo/posizione"}
        },
        "required": ["nome", "email"]
    }
}]

risposta = client.messages.create(
    model = "claude-sonnet-4-20250514",
    max_tokens = 512,
    tools = strumenti,
    messages = [{"role": "user", "content":
        "Ciao, sono Marco Bianchi, CTO di TechSrl. Puoi scrivermi a
marco@techsrl.it"}]
)

# Trova il blocco tool_use nella risposta
for blocco in risposta.content:
    if blocco.type == "tool_use":
        dati = blocco.input # dizionario Python già validato
        print(dati)
        # {'nome': 'Marco Bianchi', 'email': 'marco@techsrl.it',
        #  'azienda': 'TechSrl', 'ruolo': 'CTO'}

```

13.5 Tree of Thoughts e Self-Consistency per task complessi

Per task dove una singola catena di ragionamento non è sufficiente, esistono tecniche che esplorano più percorsi in parallelo.

Self-Consistency

Genera la stessa risposta più volte con temperatura > 0 (così ogni run è diverso) e prendi la risposta più frequente. Funziona bene su task con risposta discreta (classificazione, risposta numerica, scelta multipla).

```

import collections

def self_consistency(domanda, n_campioni=5, temperatura=0.7):
    """
    Genera n_campioni risposte e restituisce quella più frequente.
    Utile per task dove la risposta è discreta (es. classificazione).
    """
    risposte = []
    for _ in range(n_campioni):
        r = client.messages.create(
            model = "claude-sonnet-4-20250514",
            max_tokens = 256,
            # Nota: l'API Anthropic non espone temperature nel parametro standard,
            # ma il principio è lo stesso – generazioni multiple danno output diversi
            messages = [{"role": "user", "content": domanda}]
        )
        risposte.append(r.content[0].text.strip())

    # Conta le occorrenze e restituisci la più frequente
    conteggio = collections.Counter(risposte)
    piu_frequente, quante_volte = conteggio.most_common(1)[0]
    confidenza = quante_volte / n_campioni
    return piu_frequente, confidenza, conteggio

risposta, conf, tutte = self_consistency(
    "Questo triangolo con lati 3, 4, 5 è rettangolo? Rispondi solo SÌ o NO."
)
print(f"Risposta: {risposta} (confidenza: {conf:.0%})")
print(f"Distribuzione: {dict(tutte)}")

```

Tree of Thoughts (ToT)

Invece di generare una singola catena di pensiero, esplori più "rami" di ragionamento e valuti quale è più promettente prima di continuare. È computazionalmente costoso (molte chiamate API) ma utile per problemi di pianificazione o ricerca.

```

def tree_of_thoughts(problema, n_rami=3, profondita=2):
    """
    Implementazione semplificata di ToT.
    Ad ogni passo genera n_rami approcci diversi, poi chiede al modello
    di valutare quale è più promettente e continua solo con quello.
    """
    contesto = problema

    for livello in range(profondita):
        # Genera n_rami approcci diversi
        prompt_genera = f"""Problema: {contesto}

        Proponi {n_rami} approcci DIVERSI per affrontare il prossimo passo.
        Numera gli approcci da 1 a {n_rami}. Sii conciso (max 2 righe per approccio)."""

        r_genera = client.messages.create(
            model="claude-sonnet-4-20250514", max_tokens=512,
            messages=[{"role": "user", "content": prompt_genera}]
        )
        approcci = r_genera.content[0].text

```

```

# Valuta quale approccio è più promettente
prompt_valuta = f"""Problema originale: {problema}

Approcci proposti:
{approcci}

Quale approccio è più promettente per risolvere il problema?
Rispondi con il numero dell'approccio (1, 2 o {n_rami}) e una riga di motivazione."""

r_valuta = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=128,
    messages=[{"role": "user", "content": prompt_valuta}]
)

# Aggiorna il contesto con l'approccio scelto
contesto = f"{contesto}\n\nApproccio scelto al passo
{livello+1}:\n{r_valuta.content[0].text}"
print(f"--- Passo {livello+1} ---\n{r_valuta.content[0].text}\n")

# Genera la soluzione finale
r_finale = client.messages.create(
    model="claude-sonnet-4-20250514", max_tokens=1024,
    messages=[{"role": "user", "content":
        f"{contesto}\n\nOra fornisci la soluzione completa al problema
originale."}]
)
return r_finale.content[0].text

# Esempio: problema di pianificazione
soluzione = tree_of_thoughts(
    "Devo organizzare una migrazione di database in produzione con zero downtime. "
    "Il database ha 50GB di dati e l'applicazione ha 10.000 utenti attivi."
)
print("Soluzione finale:\n", soluzione)

```

13.6 Come testare e misurare la qualità dei prompt

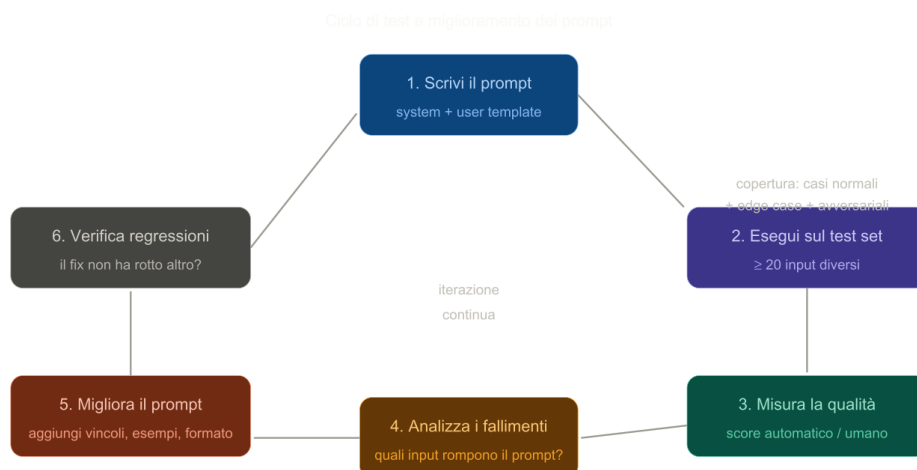


Figura 13.2 — Ciclo di test/misurazione dei prompt.

Un prompt non testato sistematicamente non è un prompt — è un'ipotesi. Come mostra la Figura 13.2, il ciclo di sviluppo affidabile di un prompt è: scrivi → crea test set → esegui in batch → misura → modifica.

Il test set è il componente più importante. Deve contenere:

- **Casi tipici** — input rappresentativi del caso d'uso principale.
- **Casi limite** — input ambigui, insoliti, o che potrebbero confondere il modello.
- **Casi negativi** — input fuori scope per verificare che il modello li gestisca correttamente.

```
import json
from typing import Callable

def valuta_prompt(
    prompt_template: str,
    test_set: list[dict],
    funzione_score: Callable,
    verbose: bool = True
) -> dict:
    """
    Esegue un prompt su un test set e calcola lo score medio.

    Args:
        prompt_template: stringa con {input} come segnaposto
        test_set: lista di {"input": ..., "atteso": ...}
        funzione_score: funzione(output, atteso) -> float tra 0 e 1
    """
    risultati = []

    for esempio in test_set:
        prompt = prompt_template.format(input=esempio["input"])
        risposta = client.messages.create(
            model = "claude-sonnet-4-20250514",
            max_tokens = 256,
            messages = [{"role": "user", "content": prompt}]
        )
        output = risposta.content[0].text.strip()
        score = funzione_score(output, esempio["atteso"])

        risultati.append({
            "input": esempio["input"],
            "atteso": esempio["atteso"],
            "output": output,
            "score": score
        })

    if verbose:
        stato = "✓" if score == 1.0 else "X"
        print(f"{stato} Input: {esempio['input'][:50]}...")
        print(f"  Atteso: {esempio['atteso']} | Output: {output[:60]}")

    score_medio = sum(r["score"] for r in risultati) / len(risultati)
    print(f"\nScore medio: {score_medio:.1%} ({sum(r['score']==1 for r in
    risultati)}/{len(risultati)} corretti)")
```

```

    return {"score_medio": score_medio, "risultati": risultati}

# --- Esempio: valutazione di un classificatore di sentiment ---

prompt_sentiment = """Classifica il sentiment di questa recensione.
Rispondi con UNA SOLA parola: POSITIVO, NEGATIVO o NEUTRO.

Recensione: {input}"""

test_set_sentiment = [
    {"input": "Prodotto fantastico, lo ricomprerò!", "atteso": "POSITIVO"},
    {"input": "Pessima qualità, si è rotto dopo un giorno.", "atteso": "NEGATIVO"},
    {"input": "Nella media, niente di speciale.", "atteso": "NEUTRO"},
    {"input": "Bella confezione ma funziona male.", "atteso": "NEGATIVO"},
    {"input": "Spedizione rapida, qualità adeguata al prezzo.", "atteso": "NEUTRO"},
    {"input": "Supera le aspettative in tutto.", "atteso": "POSITIVO"},
    {"input": "Non male per il prezzo.", "atteso": "NEUTRO"},
    {"input": "Assolutamente inutile, non funziona.", "atteso": "NEGATIVO"},
]

def score_exact_match(output: str, atteso: str) -> float:
    """1.0 se l'output contiene l'etichetta attesa, 0.0 altrimenti."""
    return 1.0 if atteso.upper() in output.upper() else 0.0

risultati = valuta_prompt(prompt_sentiment, test_set_sentiment, score_exact_match)

```

LLM-as-Judge: valutazione automatica di output aperti

Per output non strutturati (testi, spiegazioni, risposte aperte) dove non esiste un "output atteso" preciso, puoi usare un secondo LLM come giudice.

```

def llm_judge(output: str, criteri: str, modello_giudice: str = "claude-sonnet-4-
20250514") -> dict:
    """
    Usa un LLM per valutare la qualità di un output.
    Restituisce un dizionario con score e motivazione.
    """
    prompt_giudice = f"""Valuta la seguente risposta secondo i criteri indicati.

Risposta da valutare:
{output}

Criteri di valutazione:
{criteri}

Rispondi SOLO con JSON valido in questo formato:
{{"score": <intero da 1 a 5>, "motivazione": "<frase breve>"}}"""

    r = client.messages.create(
        model=modello_giudice, max_tokens=256,
        messages=[{"role": "user", "content": prompt_giudice}]
    )

    try:
        valutazione = json.loads(r.content[0].text)
        return valutazione

```

```

except json.JSONDecodeError:
    return {"score": 0, "motivazione": "Errore nel parsing della valutazione"}

# Esempio di uso
risposta_da_valutare = """
Il gradient descent è un algoritmo di ottimizzazione che aggiorna i parametri
del modello nella direzione opposta al gradiente della loss.
Ad ogni passo, i pesi vengono aggiornati di una quantità proporzionale
al learning rate moltiplicato per il gradiente.
"""

criteri = """
1. Accuratezza tecnica (la spiegazione è corretta?)
2. Chiarezza (è comprensibile per un principiante?)
3. Completezza (copre i concetti essenziali senza essere ridondante?)
"""

valutazione = llm_judge(risposta_da_valutare, criteri)
print(f"Score: {valutazione['score']}/5")
print(f"Motivazione: {valutazione['motivazione']}")

```

13.7 Esercizio guidato — evaluator automatico di prompt

Costruiamo un sistema completo per confrontare due versioni di un prompt sullo stesso test set e decidere quale è migliore.

```

# =====
# Prompt A/B testing automatizzato
# =====
import anthropic, json
from dataclasses import dataclass

client = anthropic.Anthropic()

@dataclass
class VersionePrompt:
    nome: str
    template: str # usa {input} come segnaposto

def confronta_prompt(
    versione_a: VersionePrompt,
    versione_b: VersionePrompt,
    test_set: list[dict],
    funzione_score: Callable,
    stampa_dettagli: bool = True
) -> dict:
    """Confronta due prompt sullo stesso test set e dichiara il vincitore."""

    print(f"{'='*60}")
    print(f"Confronto: {versione_a.nome} vs {versione_b.nome}")
    print(f"Test set: {len(test_set)} esempi")
    print(f"{'='*60}\n")

```

```

    risultati_a = valuta_prompt(versione_a.template, test_set, funzione_score,
                                verbose=stampa_dettagli)

    print()
    risultati_b = valuta_prompt(versione_b.template, test_set, funzione_score,
                                verbose=stampa_dettagli)

    score_a = risultati_a["score_medio"]
    score_b = risultati_b["score_medio"]
    delta = score_b - score_a

    print(f"\n{'='*60}")
    print(f"RISULTATI FINALI")
    print(f"  {versione_a.nome}: {score_a:.1%}")
    print(f"  {versione_b.nome}: {score_b:.1%}")
    print(f"  Delta: {delta:+.1%}")

    if abs(delta) < 0.05:
        print("  Verdetto: PARI (differenza < 5%)")
        vincitore = None
    elif delta > 0:
        print(f"  Verdetto: {versione_b.nome} è migliore")
        vincitore = versione_b.nome
    else:
        print(f"  Verdetto: {versione_a.nome} è migliore")
        vincitore = versione_a.nome

    return {"vincitore": vincitore, "score_a": score_a, "score_b": score_b}

# --- Test su classificazione di sentiment ---

prompt_v1 = VersionePrompt(
    nome = "v1 - zero-shot semplice",
    template = "Classifica il sentiment: POSITIVO, NEGATIVO o NEUTRO.\n\nTesto: {input}\n\nRisposta:"
)

prompt_v2 = VersionePrompt(
    nome = "v2 - con definizioni esplicite",
    template = """"Classifica il sentiment del testo seguente.

Definizioni:
- POSITIVO: esprime soddisfazione, approvazione o entusiasmo
- NEGATIVO: esprime insoddisfazione, critica o delusione
- NEUTRO: descrive fatti senza valutazione, o bilancia pro e contro

Rispondi con UNA SOLA parola: POSITIVO, NEGATIVO o NEUTRO.

Testo: {input}
Risposta: """"
)

risultato = confronta_prompt(
    prompt_v1, prompt_v2,
    test_set_sentiment,
    score_exact_match,
    stampa_dettagli=False

```

Riepilogo del Capitolo 13

Hai imparato:

- **La gerarchia dei prompt:** zero-shot → few-shot → chain-of-thought → tecniche avanzate. Usa il minimo necessario.
- **Zero-shot:** solo l'istruzione. Sufficiente per task semplici e ben definiti su modelli grandi.
- **Few-shot:** 2–5 esempi nel prompt. Migliora consistenza del formato, non la comprensione.
- **Chain-of-Thought:** chiedi di ragionare passo per passo. Migliora molto su task di ragionamento, inutile su task semplici.
- **System prompt robusto:** definisce ruolo, tono, formato, casi limite. Sii esplicito, usa struttura markdown.
- **Structured output:** usa il tool use/function calling per JSON garantito, non solo istruzioni testuali.
- **Self-Consistency:** genera più risposte e prendi la più frequente. Utile per task con risposta discreta.
- **Tree of Thoughts:** esplora più rami di ragionamento e valuta quale continuare. Costoso ma potente su task di pianificazione.
- **Ciclo di test:** scrivi test set con casi tipici, limite e negativi. Misura con exact match, LLM-as-Judge o metriche task-specific.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un sistema di prompt engineering sistematico per un task reale, con test set, A/B testing e analisi degli errori.

Traccia:

Scegli uno di questi task (o uno tuo equivalente):

- Estrazione di entità nominate (persone, luoghi, organizzazioni) da articoli di giornale.
 - Classificazione di ticket di supporto in categorie (billing, tecnico, account, altro).
 - Generazione di titoli SEO per articoli di blog dato il testo.
29. Costruisci un test set di almeno 30 esempi con output atteso. Includi almeno 5 casi limite e 5 casi negativi (input fuori scope). Documentalo in un file JSON.
 30. Scrivi tre versioni del prompt (v1 zero-shot, v2 few-shot con 3 esempi, v3 con definizioni esplicite + CoT se rilevante). Testa tutte e tre sullo stesso test set con la funzione `valuta_prompt`.
 31. Analizza gli errori della versione migliore: quali input ha sbagliato? C'è un pattern comune? Scrivi una v4 che tenta di correggere quei casi specifici e verifica se migliora senza peggiorare sui casi già corretti (regressione).
 32. Implementa un LLM-as-Judge per i casi dove non esiste un output atteso preciso (es. valuta la qualità dei titoli generati su una scala 1–5 per pertinenza e attrattiva). Confronta il giudizio automatico con il tuo giudizio manuale su 10 esempi: quanto concordano?

33. **Domanda di riflessione:** dopo aver completato l'esercizio, quale delle tre versioni del prompt ha performato meglio? La versione più complessa era necessariamente la migliore? Cosa ti dice questo sulla relazione tra complessità del prompt e qualità dell'output?

Consegna: il notebook con i test set in JSON, le curve comparative delle tre versioni, l'analisi degli errori, e le risposte scritte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: RAG — Retrieval-Augmented Generation. Impareremo a dare agli LLM accesso a conoscenza privata e aggiornata, senza riaddestrare nulla.

Capitolo 14

RAG: Dare Memoria Esterna agli LLM

"Un LLM sa tutto quello che c'era su internet fino alla data di training. Non sa nulla di quello che hai scritto tu ieri. Il RAG risolve esattamente questo."

14.1 Il problema che il RAG risolve

Gli LLM hanno due limiti strutturali che li rendono inutilizzabili per molti casi d'uso reali.

Conoscenza obsoleta. I modelli hanno una data di cutoff: tutto ciò che è successo dopo non lo sanno. Se costruisci un assistente per la tua azienda, il modello non conosce i tuoi prodotti, le tue policy interne, i tuoi contratti, le decisioni prese la settimana scorsa.

Nessuna conoscenza privata. I dati interni dell'azienda non sono su internet e non sono nel training set. Documentazione tecnica, email, database clienti, report finanziari — il modello non li ha mai visti.

La soluzione ovvia sembrerebbe il fine-tuning: riaddestra il modello sui tuoi dati. Ma il fine-tuning è costoso, richiede tempo, e non scala: ogni volta che aggiungi un documento devi riaddestrare. E soprattutto, i modelli fine-tuned tendono ad allucinare quando gli viene chiesto qualcosa di specifico — confondono conoscenza generale con conoscenza specifica appena appresa.

Il **Retrieval-Augmented Generation (RAG)** risolve il problema in modo più elegante: invece di insegnare al modello i tuoi dati, glieli mostri al momento della domanda. Recupera i documenti rilevanti dalla tua knowledge base, li inserisci nel prompt come contesto, e il modello risponde basandosi su quel contesto — non sulla sua memoria parametrica.

***Analogia:** la differenza tra un medico che cerca di ricordare tutto a memoria e un medico che consulta le cartelle cliniche del paziente prima di rispondere. Il secondo è molto più affidabile, anche se tecnicamente "meno intelligente".*

14.2 La pipeline completa: cinque passi



Figura 14.1 — La pipeline completa end-to-end.

Come mostra la Figura 14.1, il RAG si divide in due fasi distinte con tempistiche diverse.

La **fase di indicizzazione** viene eseguita una volta (o periodicamente quando i documenti cambiano): prendi i tuoi documenti, spezzali in blocchi, converti ogni blocco in un vettore numerico e salvali in un database vettoriale.

La **fase di retrieval e generazione** viene eseguita ad ogni query: converti la domanda dell'utente nello stesso spazio vettoriale, trova i blocchi più simili alla domanda, inseriscili nel prompt come contesto, e passa tutto all'LLM.

I cinque passi nel dettaglio:

- 1. Chunking** — spezzare i documenti in blocchi. La dimensione ottimale è tipicamente 300–600 token con un overlap di 50–100 token tra blocchi consecutivi. L'overlap evita che informazioni rilevanti vengano tagliate esattamente al confine tra due chunk.
- 2. Embedding** — convertire ogni chunk in un vettore numerico usando un modello di embedding. Il vettore cattura il significato semantico del testo: chunk simili per significato saranno vicini nello spazio vettoriale.
- 3. Indicizzazione** — salvare i vettori (con il testo originale associato) in un database vettoriale ottimizzato per la ricerca per similarità.
- 4. Retrieval** — data una query, convertirla nello stesso spazio vettoriale e trovare i top-k chunk più simili usando la similarità coseno.
- 5. Generazione** — costruire un prompt che include i chunk recuperati come contesto e passarlo all'LLM per la risposta finale.

14.3 Embedding e similarità coseno

Il modello di embedding è il componente che rende possibile la ricerca semantica. Prende un testo di qualsiasi lunghezza e produce un vettore denso di dimensione fissa — tipicamente 768, 1024 o 1536 numeri. Vettori di testi semanticamente simili sono vicini nello spazio vettoriale, anche se le parole usate sono diverse.

Come mostra la Figura 14.1, "gatto" e "felino" hanno similarità coseno 0.94 — sono quasi identici semanticamente. "gatto" e "animale" hanno similarità 0.61 — correlati ma non equivalenti. "gatto" e "database" hanno similarità 0.08 — non correlati.

La **similarità coseno** misura l'angolo tra due vettori:

$$\cos(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$$

Vale tra -1 e 1. Più è alto, più i testi sono simili. In pratica per testi in italiano si considera alta una similarità sopra 0.7.

```
import numpy as np

def similarita_coseno(v1: np.ndarray, v2: np.ndarray) -> float:
    """Calcola la similarità coseno tra due vettori."""
    normal = np.linalg.norm(v1)
    norma2 = np.linalg.norm(v2)
    if normal == 0 or norma2 == 0:
        return 0.0
    return float(np.dot(v1, v2) / (normal * norma2))

# Uso con il modello di embedding di Anthropic / OpenAI / HuggingFace
# Esempio con sentence-transformers (open source, funziona in locale)
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

modello_embed = SentenceTransformer("sentence-transformers/paraphrase-multilingual-mpnet-base-v2")
# Questo modello supporta l'italiano nativamente

frasi = [
    "Il gatto dorme sul divano.",
    "Un felino riposa sul sofà.",
    "La banca ha aumentato i tassi di interesse.",
    "Come si addestra una rete neurale?"
]

vettori = modello_embed.encode(frasi) # shape: (4, 768)
print(f"Shape dei vettori: {vettori.shape}")

# Confronta la prima frase con tutte le altre
for i, frase in enumerate(frasi[1:], 1):
    sim = similarita_coseno(vettori[0], vettori[i])
    print(f"Sim(frase_0, frase_{i}): {sim:.3f} | '{frase[:40]}'")
```

14.4 ChromaDB: vector database in locale

ChromaDB è il punto di partenza ideale per il RAG: zero configurazione, gira in locale, API semplice. Per la produzione si usa Pinecone, pgvector o FAISS — ma la logica è identica.

```
# pip install chromadb sentence-transformers
import chromadb
from sentence_transformers import SentenceTransformer

# --- Setup ---
modello_embed = SentenceTransformer("sentence-transformers/paraphrase-multilingual-
mpnet-base-v2")
client_chroma = chromadb.PersistentClient(path="./chroma_db") # salva su disco

# Crea (o carica se esiste già) una collection
# Una collection è un indice per un insieme di documenti
collection = client_chroma.get_or_create_collection(
    name = "knowledge_base",
    metadata = {"hnsw:space": "cosine"} # usa similarità coseno
)

# --- Indicizzazione: aggiungi documenti ---
documenti = [
    "Il machine learning è un sottoinsieme dell'intelligenza artificiale che permette
    ai sistemi di imparare dai dati.",
    "Le reti neurali convoluzionali (CNN) sono particolarmente efficaci per il
    riconoscimento di immagini.",
    "Il meccanismo di attenzione del Transformer permette ad ogni token di guardare
    tutti gli altri contemporaneamente.",
    "Il fine-tuning adatta un modello pre-addestrato a un task specifico usando pochi
    dati aggiuntivi.",
    "Il RAG combina il retrieval di documenti rilevanti con la generazione di testo
    per rispondere a domande.",
    "I Large Language Models vengono addestrati su trilioni di token per predire il
    token successivo.",
    "PyTorch è il framework di deep learning più usato nella ricerca grazie alla sua
    flessibilità.",
    "La backpropagation calcola i gradienti risalendo il grafo computazionale
    dall'output all'input.",
]

ids = [f"doc_{i}" for i in range(len(documenti))]
vettori = modello_embed.encode(documenti).tolist()

collection.add(
    documents = documenti,
    embeddings = vettori,
    ids = ids
)
print(f"Documenti indicizzati: {collection.count()}")

# --- Retrieval: cerca i documenti più rilevanti ---
def cerca(query: str, n_risultati: int = 3) -> list[dict]:
    """Cerca i chunk più simili alla query."""
    vettore_query = modello_embed.encode([query]).tolist()

    risultati = collection.query(
        query_embeddings = vettore_query,
```

```

        n_results          = n_risultati
    )

    chunks = []
    for doc, dist, id_ in zip(
        risultati["documents"][0],
        risultati["distances"][0],
        risultati["ids"][0]
    ):
        chunks.append({
            "testo":      doc,
            "similarita": 1 - dist,    # ChromaDB restituisce distanza, non similarità
            "id":         id_
        })
    return chunks

# Test del retrieval
query = "Come funziona l'attenzione nei Transformer?"
chunks = cerca(query)
print(f"\nQuery: '{query}'")
print(f"Risultati:")
for c in chunks:
    print(f"    [{c['similarita']:.3f}] {c['testo'][:80]}...")

```

14.5 Pipeline RAG completa con LLM

Mettiamo insieme retrieval e generazione in una funzione unica.

```

import anthropic

client_llm = anthropic.Anthropic()

SYSTEM_RAG = """Sei un assistente esperto di intelligenza artificiale.
Rispondi SOLO basandoti sui documenti forniti nel contesto.
Se la risposta non è presente nei documenti, dillo esplicitamente –
non inventare informazioni.
Cita sempre la fonte (l'ID del documento) quando usi informazioni specifiche."""

def rag_query(domanda: str, n_chunk: int = 3) -> str:
    """
    Pipeline RAG completa:
    1. Recupera i chunk più rilevanti
    2. Costruisce il prompt con il contesto
    3. Genera la risposta
    """
    # PASSO 1: Retrieval
    chunks = cerca(domanda, n_risultati=n_chunk)

    if not chunks:
        return "Non ho trovato documenti rilevanti per questa domanda."

    # PASSO 2: Costruisci il contesto
    contesto = "\n\n".join([
        f"[Documento {c['id']} – similarità: {c['similarita']:.2f}]\n{c['testo']}"

```

```

        for c in chunks
    ])

    prompt = f"""Contesto dai documenti:
{contesto}

---
Domanda: {domanda}

Rispondi basandoti esclusivamente sul contesto fornito sopra."""

    # PASSO 3: Generazione
    risposta = client_llm.messages.create(
        model      = "claude-sonnet-4-20250514",
        max_tokens = 512,
        system     = SYSTEM_RAG,
        messages   = [{"role": "user", "content": prompt}]
    )

    return risposta.content[0].text

# Test
domande = [
    "Come funziona il meccanismo di attenzione?",
    "Qual è la differenza tra ML e deep learning?",
    "Cos'è la fotosintesi?",    # fuori scope – il modello deve dirlo
]

for domanda in domande:
    print(f"\nDomanda: {domanda}")
    print(f"Risposta: {rag_query(domanda)}")
    print("-" * 60)

```

14.6 Indicizzare documenti PDF reali

In produzione i documenti sono PDF, Word, pagine web. Ecco come gestirli.

```

# pip install pypdf langchain-text-splitters
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def carica_pdf(percorso: str) -> str:
    """Estrae il testo da un PDF."""
    reader = PdfReader(percorso)
    testo = ""
    for pagina in reader.pages:
        testo += pagina.extract_text() + "\n"
    return testo

def indicizza_pdf(percorso: str, collection_name: str = "documenti"):
    """
    Pipeline completa di indicizzazione per un PDF:
    estrai testo → chunking → embedding → salva in ChromaDB.
    """

```

```

"""
# 1. Estrai il testo
testo = carica_pdf(percorso)
print(f"Testo estratto: {len(testo):,} caratteri")

# 2. Chunking con overlap
# RecursiveCharacterTextSplitter prova a spezzare su paragrafi,
# poi su frasi, poi su parole - rispettando la struttura del testo
splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,      # ~500 token per chunk
    chunk_overlap = 100,   # 100 token di overlap tra chunk adiacenti
    separators = ["\n\n", "\n", ". ", " ", ""]
)
chunks = splitter.split_text(testo)
print(f"Chunks creati: {len(chunks)}")

# 3. Crea collection
coll = client_chroma.get_or_create_collection(
    name = collection_name,
    metadata = {"hnsw:space": "cosine"}
)

# 4. Embedding e indicizzazione in batch
# Processiamo in batch per evitare di sovraccaricare il modello di embedding
BATCH_SIZE = 64
for i in range(0, len(chunks), BATCH_SIZE):
    batch = chunks[i : i + BATCH_SIZE]
    vettori = modello_embed.encode(batch).tolist()
    ids = [f"chunk_{i+j}" for j in range(len(batch))]
    metadati = [{"source": percorso, "chunk_index": i+j} for j in
range(len(batch))]

    coll.add(documents=batch, embeddings=vettori, ids=ids, metadatas=metadati)
    print(f"  Indicizzati chunk {i}-{i+len(batch)-1}")

print(f"Indicizzazione completata. Totale chunk: {coll.count()}")
return coll

# Uso
# coll = indicizza_pdf("mio_documento.pdf", "mia_knowledge_base")

```


14.7 Tecniche avanzate: re-ranking e hybrid search

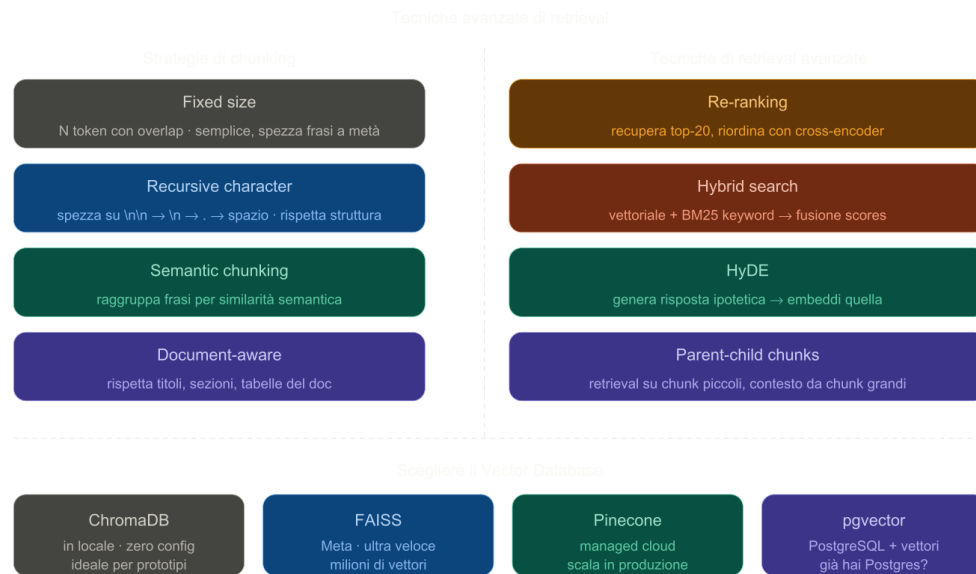


Figura 14.2 — Il confronto tra le tecniche di retrieval.

La Figura 14.2 mostra le tecniche che migliorano la qualità del retrieval oltre la semplice ricerca vettoriale.

Re-ranking

Il retrieval vettoriale recupera i top-k chunk per similarità embedding. Ma l'embedding è una rappresentazione compressa — può perdere sfumature importanti. Il re-ranking usa un modello più preciso (cross-encoder) per riordinare i chunk recuperati.

```
# pip install sentence-transformers
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def retrieval_con_reranking(query: str, n_iniziali: int = 20, n_finali: int = 3):
    """
    Recupera n_iniziali chunk con retrieval vettoriale,
    poi riordina con cross-encoder e tiene solo i migliori n_finali.
    """
    # Retrieval iniziale: ampio
    chunks_iniziali = cerca(query, n_risultati=n_iniziali)

    # Re-ranking: preciso
    coppie = [(query, c["testo"]) for c in chunks_iniziali]
    scores = reranker.predict(coppie)

    # Riordina per score decrescente
    chunks_riordinati = sorted(
        zip(chunks_iniziali, scores),
        key=lambda x: x[1],
        reverse=True
    )

    return [c for c, _ in chunks_riordinati[:n_finali]]
```

Hybrid Search

La ricerca vettoriale è ottima per query semantiche ma può fallire su query molto specifiche con numeri, nomi propri o codici tecnici. BM25 (una variante di TF-IDF) eccelle esattamente in quei casi. Combinare i due dà il meglio di entrambi.

```
# pip install rank-bm25
from rank_bm25 import BM25Okapi

class HybridRetriever:
    """Combina ricerca vettoriale e BM25 per risultati migliori."""

    def __init__(self, documenti: list[str]):
        self.documenti = documenti

        # Indice vettoriale (già in ChromaDB)
        # Indice BM25
        tokenizzati = [doc.lower().split() for doc in documenti]
        self.bm25 = BM25Okapi(tokenizzati)

    def cerca(self, query: str, n: int = 5, peso_vettoriale: float = 0.6) -> list[str]:
        """
        Combina score vettoriali e BM25 con Reciprocal Rank Fusion.
        peso_vettoriale: quanto peso dare alla ricerca semantica (0-1)
        """
        # Score BM25
        tokens_query = query.lower().split()
        scores_bm25 = self.bm25.get_scores(tokens_query)

        # Score vettoriali (da ChromaDB)
        vettore_q = modello_embed.encode([query]).tolist()
        risultati_v = collection.query(query_embeddings=vettore_q,
n_results=len(self.documenti))
        scores_v = np.zeros(len(self.documenti))
        for id_, dist in zip(risultati_v["ids"][0], risultati_v["distances"][0]):
            idx = int(id_.split("_")[1])
            scores_v[idx] = 1 - dist

        # Normalizza entrambi gli score tra 0 e 1
        def normalizza(arr):
            mx = arr.max()
            return arr / mx if mx > 0 else arr

        scores_norm_v = normalizza(scores_v)
        scores_norm_bm25 = normalizza(scores_bm25)

        # Score combinato pesato
        scores_finali = (peso_vettoriale * scores_norm_v +
            (1 - peso_vettoriale) * scores_norm_bm25)

        # Restituisci i top-n documenti
        top_idx = np.argsort(scores_finali)[::-1][:n]
        return [self.documenti[i] for i in top_idx]
```

HyDE — Hypothetical Document Embedding

Invece di embeddare la query direttamente, chiedi all'LLM di generare una risposta ipotetica alla domanda, poi embeddala. L'idea è che una risposta ipotetica è più simile semanticamente ai documenti rilevanti rispetto alla domanda stessa.

```
def hyde_retrieval(query: str, n: int = 3) -> list[dict]:
    """
    HyDE: genera prima una risposta ipotetica, poi la usa per il retrieval.
    """
    # Genera risposta ipotetica
    risposta_ipotetica = client_llm.messages.create(
        model = "claude-sonnet-4-20250514",
        max_tokens = 256,
        messages = [{
            "role": "user",
            "content": f"Scrivi una breve risposta tecnica a questa domanda, "
                       f"come se stessi scrivendo documentazione:\n\n{query}"
        }]
    ).content[0].text

    # Usa la risposta ipotetica per il retrieval
    vettore_ipotetico = modello_embed.encode([risposta_ipotetica]).tolist()
    risultati = collection.query(
        query_embeddings = vettore_ipotetico,
        n_results = n
    )

    return [
        {"testo": doc, "similarita": 1 - dist}
        for doc, dist in zip(risultati["documents"][0], risultati["distances"][0])
    ]
```

14.8 Esercizio guidato — sistema Q&A su documenti PDF

Costruiamo un sistema completo di question answering sui tuoi documenti PDF con ChromaDB e Claude.

```
# =====
# Sistema RAG su PDF — esegui su Google Colab
# =====
# !pip install chromadb sentence-transformers pypdf langchain-text-splitters
# anthropic -q

import os, anthropic, chromadb
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from langchain_text_splitters import RecursiveCharacterTextSplitter

device = "cuda" if __import__("torch").cuda.is_available() else "cpu"
modello_embed = SentenceTransformer(
    "sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
    device=device
)

client_llm = anthropic.Anthropic()
client_chroma = chromadb.PersistentClient(path="./rag_db")
```

```

# --- 1. Carica e indicizza i PDF ---
def indicizza_documenti(percorsi_pdf: list[str], nome_collection: str = "rag") -> chromadb.Collection:
    coll = client_chroma.get_or_create_collection(nome_collection,
                                                    metadata={"hnsw:space": "cosine"})
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)

    for percorso in percorsi_pdf:
        nome_file = os.path.basename(percorso)
        reader = PdfReader(percorso)
        testo = "".join(p.extract_text() or "" for p in reader.pages)
        chunks = splitter.split_text(testo)

        vettori = modello_embed.encode(chunks, batch_size=32,
show_progress_bar=True).tolist()
        ids = [f"{nome_file}_chunk_{i}" for i in range(len(chunks))]
        meta = [{"source": nome_file, "chunk": i} for i in range(len(chunks))]

        coll.add(documents=chunks, embeddings=vettori, ids=ids, metadatas=meta)
        print(f"✓ {nome_file}: {len(chunks)} chunk indicizzati")

    print(f"\nTotale chunk nel database: {coll.count()}")
    return coll

# --- 2. Funzione di retrieval con score threshold ---
def retrieval(coll, query: str, n: int = 4, soglia_sim: float = 0.4) -> list[dict]:
    """Recupera chunk con similarità sopra la soglia."""
    vq = modello_embed.encode([query]).tolist()
    res = coll.query(query_embeddings=vq, n_results=n,
                     include=["documents", "distances", "metadatas"])

    chunks = []
    for doc, dist, meta in zip(res["documents"][0], res["distances"][0],
res["metadatas"][0]):
        sim = 1 - dist
        if sim >= soglia_sim:
            chunks.append({"testo": doc, "sim": sim, "fonte": meta.get("source",
"?")})

    return chunks

# --- 3. Generazione con citazione delle fonti ---
def rispondi(coll, domanda: str) -> str:
    chunks = retrieval(coll, domanda)

    if not chunks:
        return ("Non ho trovato informazioni sufficientemente rilevanti "
                "nei documenti per rispondere a questa domanda.")

    contesto = "\n\n".join(
        f"[Fonte: {c['fonte']} | Rilevanza: {c['sim']:.0%}]\n{c['testo']}"
        for c in chunks
    )

```

```

        risposta = client_llm.messages.create(
            model      = "claude-sonnet-4-20250514",
            max_tokens = 768,
            system      = """Sei un assistente che risponde a domande basandosi
esclusivamente sui documenti forniti. Cita sempre la fonte tra parentesi
quadre quando usi un'informazione specifica. Se la risposta non è nei documenti,
dillo chiaramente senza inventare."""
            messages    = [{"role": "user", "content":
                            f"Documenti di riferimento:\n{contesto}\n\nDomanda:
{domanda}"}]
        ).content[0].text

    return risposta

# --- 4. Interfaccia interattiva ---
def avvia_rag(percorsi_pdf: list[str]):
    print("Indicizzazione documenti...")
    coll = indicizza_documenti(percorsi_pdf)

    print("\nSistema RAG pronto. Scrivi 'esci' per terminare.\n")
    while True:
        domanda = input("Domanda: ").strip()
        if domanda.lower() in ("esci", "exit"):
            break
        if not domanda:
            continue
        print(f"\nRisposta:\n{rispondi(coll, domanda)}\n{'-'*60}\n")

# Esempio di uso - sostituisci con i tuoi PDF
# avvia_rag(["documento1.pdf", "documento2.pdf"])

# Per testare senza PDF, indicizza testo direttamente:
coll_test = client_chroma.get_or_create_collection("test", metadata={"hnsw:space":
"cosine"})
testi_test = [
    "Il RAG (Retrieval-Augmented Generation) combina recupero di documenti e
generazione di testo.",
    "ChromaDB è un database vettoriale open source ottimizzato per applicazioni di
machine learning.",
    "La similarità coseno misura l'angolo tra due vettori nello spazio degli
embedding.",
    "Il chunking spezza i documenti in blocchi di dimensione fissa con overlap per il
retrieval.",
]
vettori_test = modello_embed.encode(testi_test).tolist()
coll_test.add(
    documents    = testi_test,
    embeddings   = vettori_test,
    ids          = [f"t{i}" for i in range(len(testi_test))]
)

print(rispondi(coll_test, "Cos'è il RAG e come funziona?"))

```

Riepilogo del Capitolo 14

Hai imparato:

- **Perché il RAG:** gli LLM non conoscono i tuoi dati privati né le informazioni recenti. Il RAG fornisce contesto rilevante al momento della domanda senza riaddestrare nulla.
- **La pipeline in due fasi:** indicizzazione (offline) — chunking → embedding → vector DB; retrieval e generazione (online) — embed query → top-k retrieval → prompt con contesto → LLM.
- **Embedding e similarità coseno:** il modello di embedding proietta testi semanticamente simili vicini nello spazio vettoriale. La similarità coseno misura la vicinanza.
- **ChromaDB:** database vettoriale in locale, zero configurazione, ideale per prototipi. In produzione: Pinecone, pgvector, FAISS.
- **Chunking:** RecursiveCharacterTextSplitter con `chunk_size ~500` e `overlap ~100` è un buon punto di partenza. Chunk troppo grandi diluiscono il segnale; troppo piccoli perdono il contesto.
- **Re-ranking:** recupera più chunk del necessario, poi riordina con un cross-encoder più preciso.
- **Hybrid search:** combina ricerca vettoriale (semantica) e BM25 (keyword) per il meglio dei due mondi.
- **HyDE:** genera una risposta ipotetica e usala per il retrieval — più simile semanticamente ai documenti che la query grezza.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un sistema RAG completo su una knowledge base reale e misurarne la qualità con metriche oggettive.

Traccia:

Scegli una knowledge base reale: la documentazione ufficiale di PyTorch (scaricabile come PDF dal sito), la Wikipedia italiana su un argomento ampio (es. fisica, storia romana), o qualsiasi corpus di almeno 50 pagine che conosci bene.

34. Indicizza l'intera knowledge base. Prova due configurazioni di chunking — (a) `chunk_size=300, overlap=50` e (b) `chunk_size=700, overlap=150` — e costruisci due collection separate.
35. Crea un test set di 20 domande con risposta nota (che puoi verificare tu stesso sul documento originale). Include almeno 5 domande che richiedono di collegare informazioni da parti diverse del documento.
36. Valuta entrambe le configurazioni sulle 20 domande usando LLM-as-Judge: chiedi a Claude di valutare la risposta su tre criteri — accuratezza, completezza e assenza di allucinazioni — con score 1–5. Quale configurazione di chunking ottiene score migliori?
37. Implementa il re-ranking sulla configurazione migliore. Migliora gli score? Identifica i casi dove il re-ranking ha cambiato l'ordine dei chunk recuperati e il risultato finale è migliorato.
38. **Domanda di riflessione:** analizza le risposte sbagliate o incomplete. Il problema era nel retrieval (i chunk giusti non erano stati recuperati) o nella generazione (i chunk giusti c'erano ma il modello non li ha usati bene)? Come potresti diagnosticare sistematicamente questa differenza?

Consegna: il notebook con le due collection indicizzate, il test set in JSON, i grafici comparativi degli score, l'analisi dei casi con e senza re-ranking, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Agenti IA — quando un LLM smette di rispondere e inizia ad agire nel mondo.

Capitolo 15

Agenti IA

"Un LLM che risponde a domande è uno strumento. Un LLM che pianifica azioni, le esegue e osserva i risultati è un agente. La differenza non è tecnica — è di responsabilità."

15.1 Dal modello passivo al sistema autonomo

Fino a questo punto hai usato gli LLM in modo passivo: dai un input, ricevi un output, sei tu a decidere cosa farne. Questo funziona bene per task di generazione e analisi, ma non per task che richiedono di agire nel mondo: cercare informazioni aggiornate, leggere e scrivere file, chiamare API esterne, eseguire codice.

Un **agente IA** è un sistema in cui l'LLM non si limita a rispondere, ma pianifica una sequenza di azioni, le esegue usando strumenti esterni, osserva i risultati e aggiusta il piano di conseguenza — in un ciclo che si ripete finché il task non è completato.

La differenza rispetto a un chatbot normale è fondamentale: un chatbot risponde. Un agente agisce. E un sistema che agisce ha conseguenze nel mondo reale — file modificati, email inviate, API chiamate, codice eseguito. Questo porta responsabilità che un semplice chatbot non ha.

***Analogia:** un assistente umano che risponde alle tue domande via email è un chatbot. Un assistente che ha accesso al tuo computer, può fare ricerche, aprire programmi e modificare documenti per conto tuo è un agente. Il secondo è molto più potente — e richiede molta più fiducia.*

15.2 Tool use e function calling

Il meccanismo che permette a un LLM di usare strumenti si chiama **tool use** (nell'API Anthropic) o **function calling** (nell'API OpenAI). Il funzionamento è semplice:

39. Definisci gli strumenti disponibili con il loro schema (nome, descrizione, parametri).
40. Mandi la richiesta al modello insieme alla lista degli strumenti.
41. Il modello decide se usare un tool e quale, e restituisce una chiamata strutturata invece di testo libero.
42. Tu esegui il tool con i parametri forniti e rimandi il risultato al modello.
43. Il modello genera la risposta finale basandosi sul risultato.

```
import anthropic, json

client = anthropic.Anthropic()

# --- Definizione degli strumenti ---
strumenti = [
    {
```



```

    "name": "cerca_meteo",
    "description": "Recupera le condizioni meteo attuali per una città.",
    "input_schema": {
        "type": "object",
        "properties": {
            "citta": {
                "type": "string",
                "description": "Nome della città, es. 'Milano' o 'Roma'"
            },
            "unita": {
                "type": "string",
                "enum": ["celsius", "fahrenheit"],
                "description": "Unità di temperatura"
            }
        },
        "required": ["citta"]
    }
},
{
    "name": "calcola",
    "description": "Esegue calcoli matematici. Usa questo invece di fare calcoli mentali.",
    "input_schema": {
        "type": "object",
        "properties": {
            "espressione": {
                "type": "string",
                "description": "Espressione matematica da valutare, es. '(15 * 8) / 3 + 12'"
            }
        },
        "required": ["espressione"]
    }
}
]

```

```

# --- Funzioni Python che i tool chiamano davvero ---
def cerca_meteo(citta: str, unita: str = "celsius") -> dict:
    """Simula una chiamata a un'API meteo."""
    # In produzione qui chiameresti OpenWeatherMap, WeatherAPI, ecc.
    dati_finti = {
        "Milano": {"temp": 18, "condizione": "nuvoloso", "umidita": 72},
        "Roma": {"temp": 24, "condizione": "soleggiato", "umidita": 45},
        "Napoli": {"temp": 26, "condizione": "soleggiato", "umidita": 60},
    }
    dati = dati_finti.get(citta, {"temp": 20, "condizione": "variabile", "umidita": 55})
    if unita == "fahrenheit":
        dati["temp"] = dati["temp"] * 9/5 + 32
    dati["unita"] = unita
    dati["citta"] = citta
    return dati

def calcola(espressione: str) -> dict:
    """Valuta un'espressione matematica in modo sicuro."""
    try:
        # ast.literal_eval è sicuro - non esegue codice arbitrario
        # Per espressioni più complesse si usa numexpr o sympy

```

```

        risultato = eval(espressione, {"__builtins__": {}},
                          {"abs": abs, "round": round, "min": min, "max": max})
    return {"risultato": risultato, "espressione": espressione}
except Exception as e:
    return {"errore": str(e), "espressione": espressione}

# Mappa nome_tool → funzione Python
TOOL_MAP = {
    "cerca_meteo": cerca_meteo,
    "calcola":     calcola,
}

# --- Loop agente: gestisce chiamate tool in automatico ---
def esegui_agente(domanda: str, max_turni: int = 10) -> str:
    """
    Loop completo: invia la domanda, gestisce le chiamate ai tool,
    e restituisce la risposta finale.
    max_turni: limite di sicurezza per evitare loop infiniti.
    """
    messaggi = [{"role": "user", "content": domanda}]

    for turno in range(max_turni):
        risposta = client.messages.create(
            model      = "claude-sonnet-4-20250514",
            max_tokens = 1024,
            tools      = strumenti,
            messages   = messaggi
        )

        # Se il modello ha finito (nessun tool da chiamare)
        if risposta.stop_reason == "end_turn":
            testo_finale = "".join(
                b.text for b in risposta.content if hasattr(b, "text")
            )
            return testo_finale

        # Se il modello vuole usare uno o più tool
        if risposta.stop_reason == "tool_use":
            # Aggiungi la risposta del modello alla storia
            messaggi.append({"role": "assistant", "content": risposta.content})

            # Esegui tutti i tool richiesti
            risultati_tool = []
            for blocco in risposta.content:
                if blocco.type != "tool_use":
                    continue

                nome      = blocco.name
                params    = blocco.input
                print(f" → Tool: {nome} ({params})")

                # Esegui il tool
                if nome in TOOL_MAP:
                    risultato = TOOL_MAP[nome](*params)
                else:
                    risultato = {"errore": f"Tool '{nome}' non trovato"}

```

```

print(f" ← Risultato: {risultato}")

risultati_tool.append({
    "type": "tool_result",
    "tool_use_id": blocco.id,
    "content": json.dumps(risultato, ensure_ascii=False)
})

# Rimanda i risultati al modello
messaggi.append({"role": "user", "content": risultati_tool})

return "Limite di turni raggiunto senza risposta finale."

# Test
print("=== Test agente ===\n")
risposta = esegui_agente(
    "Che tempo fa a Milano e Roma? Qual è la differenza di temperatura? "
    "Calcola anche il 20% della temperatura di Roma in gradi Celsius."
)
print(f"\nRisposta finale:\n{risposta}")

```

15.3 Il ciclo ReAct: Ragionamento → Azione → Osservazione

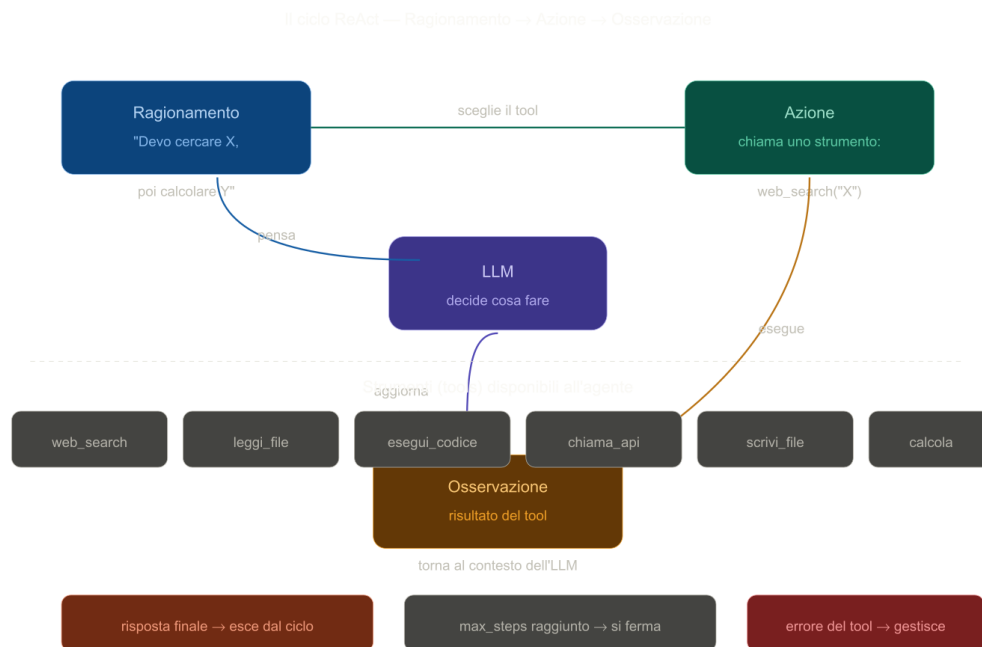


Figura 15.1 — Il ciclo ReAct.

Come mostra la Figura 15.1, il pattern **ReAct** (Reasoning + Acting) struttura il comportamento dell'agente in tre fasi che si ripetono:

Ragionamento — il modello "pensa ad alta voce" prima di agire, esplicitando il proprio piano. Questo non è solo utile per il debug: il testo del ragionamento diventa parte del contesto e guida le azioni successive.

Azione — il modello sceglie quale tool usare e con quali parametri. La scelta è guidata dal ragionamento del passo precedente.

Osservazione — il risultato del tool viene aggiunto al contesto. Il modello lo legge e decide se il task è completato o se serve un'altra iterazione.

Il ciclo si interrompe quando il modello produce una risposta finale invece di chiamare un tool, o quando si raggiunge il limite di iterazioni impostato.

```
# Esempio di trace ReAct — cosa vede il modello internamente
trace_esempio = """
Turno 1:
  Ragionamento: "L'utente chiede il meteo a Milano e Roma e la differenza.
                  Devo prima recuperare entrambe le temperature, poi calcolare."
  Azione: cerca_meteo(citta="Milano")
  Osservazione: {"temp": 18, "condizione": "nuvoloso", "citta": "Milano"}

Turno 2:
  Ragionamento: "Ho Milano (18°C). Ora recupero Roma."
  Azione: cerca_meteo(citta="Roma")
  Osservazione: {"temp": 24, "condizione": "soleggiato", "citta": "Roma"}

Turno 3:
  Ragionamento: "Ho entrambe: 18°C e 24°C. La differenza è 24-18=6°C.
                  Uso il tool calcola per confermare e per il 20% di 24."
  Azione: calcola(espressione="24 - 18")
  Osservazione: {"risultato": 6}

Turno 4:
  Azione: calcola(espressione="24 * 0.20")
  Osservazione: {"risultato": 4.8}

Turno 5:
  Ragionamento: "Ho tutti i dati. Posso rispondere."
  Risposta finale: "A Milano ci sono 18°C (nuvoloso), a Roma 24°C (soleggiato).
                   La differenza è 6°C. Il 20% della temperatura di Roma è 4.8°C."
"""
```

15.4 Memoria degli agenti

Un'agente senza memoria è un agente che non impara dai propri passi precedenti e non ricorda le conversazioni passate. Esistono tre tipi di memoria, con caratteristiche diverse.

Memoria in-context — tutto ciò che è nella finestra di contesto corrente: la conversazione, i risultati dei tool, il ragionamento intermedio. È la memoria più immediata, ma ha un limite (la context window) e sparisce alla fine della sessione.

Memoria esterna — dati persistenti salvati fuori dall'LLM: un database, un file JSON, un vector store. L'agente può leggerla e scriverla tramite tool appositi. Persiste tra sessioni e può contenere quantità arbitrarie di informazioni.

Memoria episodica — un log strutturato delle azioni passate e dei loro risultati, usato per migliorare le decisioni future. In pratica: l'agente ricorda "l'ultima volta che ho provato X è fallito, provo Y".

```

import json, os
from datetime import datetime

class MemoriaAgente:
    """
    Memoria esterna persistente per un agente.
    Salva fatti, preferenze e storico delle azioni su disco.
    """

    def __init__(self, percorso: str = "memoria_agente.json"):
        self.percorso = percorso
        self.dati = self._carica()

    def _carica(self) -> dict:
        if os.path.exists(self.percorso):
            with open(self.percorso) as f:
                return json.load(f)
        return {"fatti": {}, "preferenze": {}, "storico": []}

    def _salva(self):
        with open(self.percorso, "w") as f:
            json.dump(self.dati, f, ensure_ascii=False, indent=2)

    def ricorda_fatto(self, chiave: str, valore):
        """Salva un fatto per uso futuro."""
        self.dati["fatti"][chiave] = {
            "valore": valore,
            "aggiornato": datetime.now().isoformat()
        }
        self._salva()

    def recupera_fatto(self, chiave: str):
        """Recupera un fatto salvato in precedenza."""
        entry = self.dati["fatti"].get(chiave)
        return entry["valore"] if entry else None

    def registra_azione(self, tool: str, params: dict, risultato: dict, successo:
bool):
        """Aggiunge un'azione allo storico episodico."""
        self.dati["storico"].append({
            "timestamp": datetime.now().isoformat(),
            "tool": tool,
            "params": params,
            "risultato": risultato,
            "successo": successo
        })
        # Mantieni solo le ultime 100 azioni
        self.dati["storico"] = self.dati["storico"][-100:]
        self._salva()

    def storico_recente(self, n: int = 5) -> list:
        """Restituisce le ultime n azioni per il contesto dell'agente."""
        return self.dati["storico"][-n:]

    def contesto_memoria(self) -> str:
        """Formatta la memoria come testo da inserire nel prompt."""
        fatti = self.dati["fatti"]
        if not fatti:

```

```

        return "Nessun fatto memorizzato."
    righe = [f"- {k}: {v['valore']}" for k, v in fatti.items()]
    return "Fatti noti:\n" + "\n".join(righe)

# Uso
memoria = MemoriaAgente()
memoria.ricorda_fatto("citta_preferita_utente", "Milano")
memoria.ricorda_fatto("unita_temperatura", "celsius")
print(memoria.contesto_memoria())

```

15.5 Pattern architetturali: ReAct, Plan&Execute, Multi-Agent

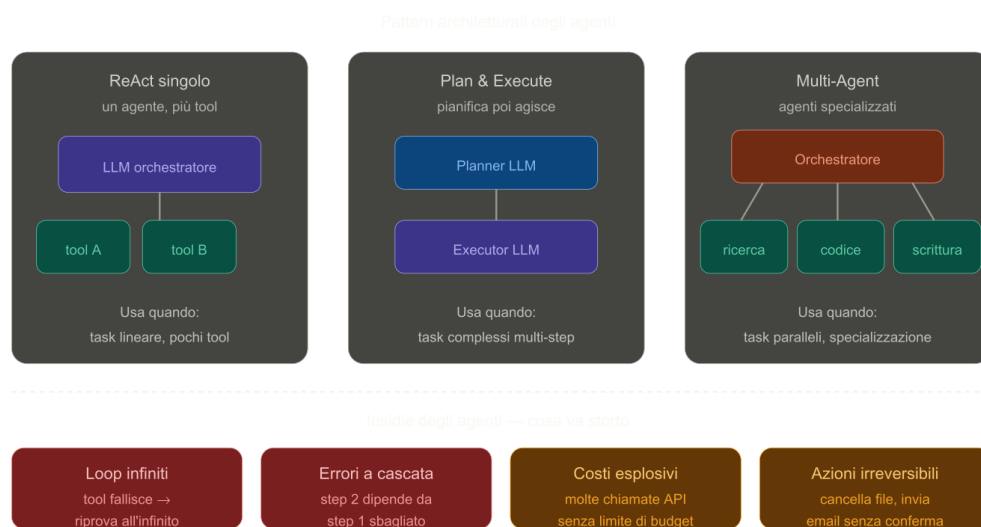


Figura 15.2 — L'architettura multi-agent.

Come mostra la Figura 15.2, non esiste un'unica architettura per gli agenti. La scelta dipende dalla complessità del task.

ReAct singolo — un LLM con accesso a più tool. Ragiona e agisce in un ciclo. Semplice da implementare, sufficiente per la maggior parte dei task lineari. Usa questo come default.

Plan & Execute — un LLM separato pianifica l'intera sequenza di azioni prima di eseguirne una sola. Un secondo LLM (o lo stesso) esegue il piano passo per passo. Utile quando il piano completo deve essere valutato prima di agire — es. task con azioni irreversibili che richiedono approvazione umana.

Multi-Agent — un orchestratore coordina più sotto-agenti specializzati. Ogni sotto-agente ha un set di tool specifico per il suo dominio. Utile per task che richiedono specializzazione parallela — es. un agente che fa ricerche, uno che scrive codice e uno che produce il report finale.

```

# Implementazione Plan & Execute
def agente_plan_execute(task: str) -> str:
    """
    Fase 1: genera un piano strutturato.
    Fase 2: esegue ogni passo del piano.
    """

```

```

"""

# --- Fase 1: pianificazione ---
piano_risposta = client.messages.create(
    model      = "claude-sonnet-4-20250514",
    max_tokens = 512,
    messages   = [{
        "role":    "user",
        "content": f"""Pianifica come completare questo task.
Elenca i passi necessari in formato JSON:
{{"passi": [{{"passo": 1, "azione": "...", "tool": "...", "motivo": "..."}}]}}

Task: {task}

Strumenti disponibili: cerca_meteo, calcola
Rispondi SOLO con JSON valido."""
    }]
).content[0].text

try:
    piano = json.loads(piano_risposta)
    passi = piano.get("passi", [])
except json.JSONDecodeError:
    return "Errore nella pianificazione."

print(f"Piano generato: {len(passi)} passi")
for p in passi:
    print(f"  Passo {p['passo']}: {p['azione']} (tool: {p['tool']})")

# --- Fase 2: esecuzione ---
# In un sistema reale qui potresti mostrare il piano all'utente
# e chiedere conferma prima di procedere (human-in-the-loop)
print("\nEsecuzione del piano...")
return esegui_agente(task)  # riusa l'agente ReAct per l'esecuzione

# Test
risultato = agente_plan_execute(
    "Confronta il meteo di Milano e Napoli e dimmi quale città è più calda oggi."
)
print(f"\nRisultato: {risultato}")

```

15.6 Insidie degli agenti: cosa va storto

La Figura 15.2 mostra le quattro insidie più comuni. Conoscerle prima di deployare un agente può salvarti da problemi seri.

Loop infiniti. Se un tool fallisce sistematicamente e il modello continua a riprovare, puoi trovarti con centinaia di chiamate API e nessun risultato. La soluzione è sempre impostare `max_turni` e gestire gli errori in modo esplicito.

```

# Pattern anti-loop: traccia i tool calls e blocca i duplicati
def esegui_agente_sicuro(domanda: str, max_turni: int = 8) -> str:
    messaggi      = [{"role": "user", "content": domanda}]
    chiamate_viste = []  # traccia (tool, params) già eseguiti

```

```

for turno in range(max_turni):
    risposta = client.messages.create(
        model="claude-sonnet-4-20250514", max_tokens=1024,
        tools=strumenti, messages=messaggi
    )

    if risposta.stop_reason == "end_turn":
        return "".join(b.text for b in risposta.content if hasattr(b, "text"))

    if risposta.stop_reason == "tool_use":
        messaggi.append({"role": "assistant", "content": risposta.content})
        risultati_tool = []

        for blocco in risposta.content:
            if blocco.type != "tool_use":
                continue

            # Controlla se questa chiamata è già stata eseguita
            chiave = (blocco.name, json.dumps(blocco.input, sort_keys=True))
            if chiave in chiamate_viste:
                # Interrompi invece di ripetere all'infinito
                return "Agente bloccato: tool chiamato più volte con gli stessi
parametri."

            chiamate_viste.append(chiave)

            if blocco.name in TOOL_MAP:
                risultato = TOOL_MAP[blocco.name](*blocco.input)
            else:
                risultato = {"errore": f"Tool '{blocco.name}' non disponibile"}

            risultati_tool.append({
                "type": "tool_result",
                "tool_use_id": blocco.id,
                "content": json.dumps(risultato, ensure_ascii=False)
            })

        messaggi.append({"role": "user", "content": risultati_tool})

return f"Task non completato dopo {max_turni} turni."

```

Costi esplosivi. Ogni chiamata LLM ha un costo. Un agente con 20 turni su un modello costoso può diventare molto caro. Imposta sempre un budget massimo di token e monitora il consumo.

```

def esegui_con_budget(domanda: str, budget_token: int = 50_000) -> str:
    """Agente con limite esplicito di token totali."""
    token_usati = 0
    messaggi = [{"role": "user", "content": domanda}]

    for _ in range(10):
        risposta = client.messages.create(
            model="claude-sonnet-4-20250514", max_tokens=1024,
            tools=strumenti, messages=messaggi
        )
        token_usati += risposta.usage.input_tokens + risposta.usage.output_tokens

        if token_usati > budget_token:

```



```

        return f"Budget token esaurito ({token_usati} token usati). Task
incompleto."

    if risposta.stop_reason == "end_turn":
        return "".join(b.text for b in risposta.content if hasattr(b, "text"))

    # ... gestione tool_use come sopra
    return "Limite turni raggiunto."

```

Errori a cascata. Se lo step 2 usa l'output dello step 1 che era sbagliato, l'errore si propaga. Valida i risultati intermedi critici prima di procedere.

Azioni irreversibili. Un agente che può inviare email, modificare database o chiamare API esterne deve avere un meccanismo di conferma per le azioni che non possono essere annullate. Il **human-in-the-loop** è il pattern corretto: fai approvare le azioni critiche dall'utente prima di eseguirle.

```

def richiedi_conferma(azione: str, dettagli: dict) -> bool:
    """Chiede conferma all'utente per azioni irreversibili."""
    print(f"\n⚠ L'agente vuole eseguire: {azione}")
    print(f"  Dettagli: {json.dumps(dettagli, ensure_ascii=False, indent=2)}")
    risposta = input("  Confermi? (s/n): ").strip().lower()
    return risposta in ("s", "si", "si", "yes", "y")

```

15.7 Esercizio guidato — agente che cerca informazioni sul web

Costruiamo un agente che usa la ricerca web reale per rispondere a domande aggiornate, con gestione degli errori e memoria della sessione.

```

# =====
# Agente con ricerca web — esegui su Google Colab
# pip install anthropic duckduckgo-search
# =====
import anthropic, json
from duckduckgo_search import DDGS

client = anthropic.Anthropic()

# --- Tool: ricerca web reale con DuckDuckGo ---
def cerca_web(query: str, max_risultati: int = 3) -> dict:
    """Cerca sul web usando DuckDuckGo (gratuito, no API key)."""
    try:
        with DDGS() as ddgs:
            risultati = list(ddgs.text(query, max_results=max_risultati))
        return {
            "query": query,
            "risultati": [
                {"titolo": r["title"], "snippet": r["body"], "url": r["href"]}
                for r in risultati
            ]
        }
    except Exception as e:
        return {"errore": str(e), "query": query}

```

```

def estrai_url(url: str, max_caratteri: int = 2000) -> dict:
    """Scarica e restituisce il testo di una pagina web."""
    try:
        import urllib.request
        req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
        with urllib.request.urlopen(req, timeout=5) as r:
            html = r.read().decode("utf-8", errors="ignore")
        # Rimuovi i tag HTML in modo grezzo
        import re
        testo = re.sub(r"<[^>]+>", " ", html)
        testo = re.sub(r"\s+", " ", testo).strip()
        return {"url": url, "testo": testo[:max_caratteri]}
    except Exception as e:
        return {"errore": str(e), "url": url}

STRUMENTI_WEB = [
    {
        "name": "cerca_web",
        "description": "Cerca informazioni aggiornate sul web. Usa per domande su eventi recenti, dati attuali o qualsiasi informazione che potrebbe essere cambiata.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "Termini di ricerca"},
                "max_risultati": {"type": "integer", "description": "Numero di risultati (1-5)", "default": 3}
            },
            "required": ["query"]
        }
    },
    {
        "name": "estrai_url",
        "description": "Legge il contenuto di una pagina web specifica. Usa dopo cerca_web per approfondire un risultato.",
        "input_schema": {
            "type": "object",
            "properties": {
                "url": {"type": "string", "description": "URL completo della pagina"}
            },
            "required": ["url"]
        }
    }
]

TOOL_MAP_WEB = {"cerca_web": cerca_web, "estrai_url": estrai_url}

SYSTEM_AGENTE = """Sei un assistente di ricerca che usa strumenti web per trovare informazioni aggiornate. Quando cerchi informazioni:
1. Usa cerca_web per trovare fonti rilevanti.
2. Usa estrai_url per approfondire le fonti più promettenti.
3. Cita sempre le fonti (URL) nelle risposte.
4. Se non trovi informazioni affidabili, dillo esplicitamente.
Rispondi in italiano."""

def agente_web(domanda: str, max_turni: int = 8) -> str:
    messaggi = [{"role": "user", "content": domanda}]

```

```

for turno in range(max_turni):
    risposta = client.messages.create(
        model      = "claude-sonnet-4-20250514",
        max_tokens = 2048,
        system     = SYSTEM_AGENTE,
        tools      = STRUMENTI_WEB,
        messages   = messaggi
    )

    if risposta.stop_reason == "end_turn":
        return "".join(b.text for b in risposta.content if hasattr(b, "text"))

    if risposta.stop_reason == "tool_use":
        messaggi.append({"role": "assistant", "content": risposta.content})
        risultati = []

        for blocco in risposta.content:
            if blocco.type != "tool_use":
                continue
            print(f"  [turno {turno+1}] Tool: {blocco.name} -
{json.dumps(blocco.input)[:80]}")
            risultato = TOOL_MAP_WEB[blocco.name](**blocco.input)
            risultati.append({
                "type":      "tool_result",
                "tool_use_id": blocco.id,
                "content":    json.dumps(risultato, ensure_ascii=False)
            })

        messaggi.append({"role": "user", "content": risultati})

    return "Limite turni raggiunto."

# Test
print(agente_web("Qual è il modello di intelligenza artificiale più recente di
Anthropic?"))

```

Riepilogo del Capitolo 15

Hai imparato:

- **Cos'è un agente:** un LLM che pianifica azioni, le esegue con tool esterni, osserva i risultati e itera — non solo risponde.
- **Tool use:** definisci strumenti con schema JSON, il modello decide quando usarli, tu li esegui e rimandi il risultato.
- **Il ciclo ReAct:** Ragionamento → Azione → Osservazione, ripetuto finché il task è completato o si raggiunge il limite.
- **Memoria:** in-context (finestra attuale), esterna (database persistente), episodica (log delle azioni passate).
- **Pattern architetturali:** ReAct singolo (default), Plan&Execute (task con azioni irreversibili), Multi-Agent (specializzazione parallela).

- **Insidie:** loop infiniti (→ `max_turni` e traccia duplicati), costi esplosivi (→ budget token), errori a cascata (→ valida output intermedi), azioni irreversibili (→ human-in-the-loop).

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un agente di ricerca e analisi che combina ricerca web, elaborazione dati e generazione di report strutturati.

Traccia:

Costruisci un agente "analista di mercato" che, dato il nome di un'azienda tech, produce un report strutturato con: panoramica dell'azienda, ultimi sviluppi (news recenti), principali prodotti/servizi, e una valutazione sintetica delle prospettive basata sulle informazioni trovate.

44. Definisci almeno quattro tool: `cerca_web`, `estrai_url`, `salva_nota(chiave, testo)` (per salvare informazioni intermedie in memoria), e `genera_report(sezioni)` (che formatta le note raccolte in un report markdown).
45. Implementa la memoria della sessione: l'agente deve poter accedere alle note salvate nei turni precedenti. Questo simula il ragionamento a più passi dove le informazioni si accumulano.
46. Aggiungi un meccanismo di retry con backoff per i tool di rete: se `estrai_url` fallisce, riprova dopo 1 secondo, poi dopo 2, poi rinuncia e passa alla fonte successiva.
47. Testa l'agente su tre aziende diverse. Quanto sono coerenti i report? Quali sezioni risultano più difficili da popolare con le sole informazioni disponibili online?
48. **Domanda di riflessione:** il tuo agente ha mai prodotto informazioni false o inventate come se fossero fatti? In quale parte del ciclo è avvenuto (ricerca, estrazione, sintesi)? Come potresti aggiungere un passo di verifica delle fonti prima di includere un'informazione nel report finale?

Consegna: il notebook con i tre report generati in markdown, il log delle azioni di ogni run, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: MCP — Model Context Protocol. Impareremo come standardizzare le integrazioni tra agenti e strumenti esterni, costruendo il tuo primo server MCP da zero.

Capitolo 16

MCP — Model Context Protocol

"Prima di MCP, ogni integrazione era un'isola: codice custom per connettere Claude a GitHub, altro codice per connettere GPT-4 al tuo database, altro ancora per qualsiasi altra cosa. MCP è il ponte standard che elimina queste isole."

16.1 Il problema che MCP risolve

Nel Capitolo 15 hai costruito un agente con tool personalizzati. Funzionava, ma ogni tool era hardcoded nell'applicazione: per aggiungere un nuovo strumento dovevi modificare il codice dell'agente, ridistribuirlo, e gestire la compatibilità con diversi LLM.

Immagina di voler costruire un assistente che usa Claude Desktop e accede al tuo filesystem, al tuo database PostgreSQL, al tuo account GitHub e ai tuoi file Google Drive. Senza uno standard, devi scrivere quattro integrazioni custom — ognuna con il suo formato, il suo protocollo di comunicazione, la sua gestione degli errori.

Il **Model Context Protocol (MCP)**, proposto da Anthropic nel novembre 2024, risolve questo con un protocollo aperto e standardizzato: qualsiasi server MCP può essere usato da qualsiasi client MCP, indipendentemente dall'LLM sottostante. È lo stesso principio dei driver USB — scrivi il driver una volta, funziona su qualsiasi dispositivo che supporta lo standard.

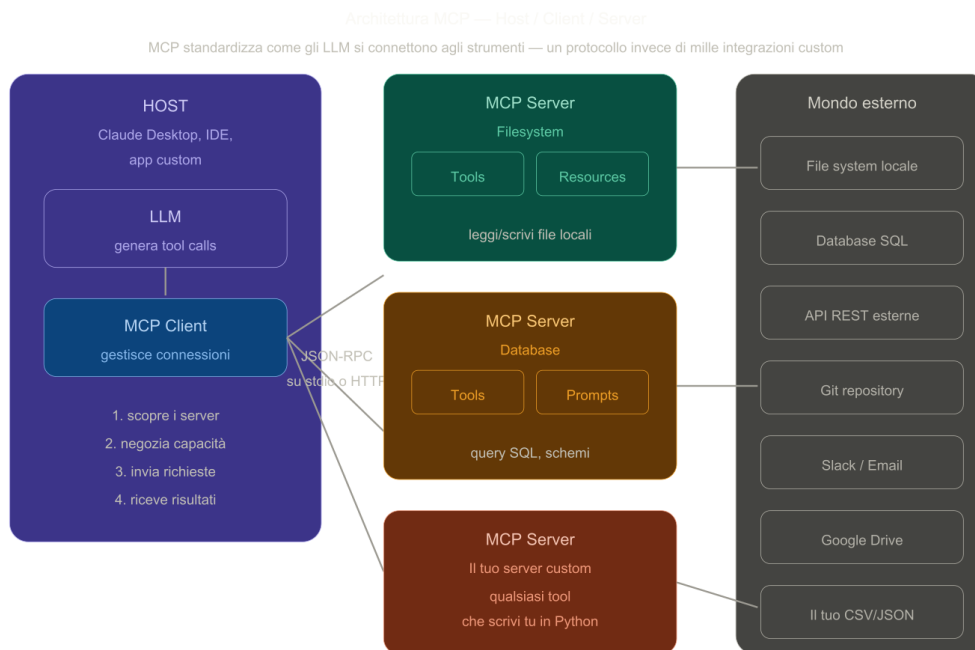


Figura 16.1 — L'architettura Host/Client/Server.

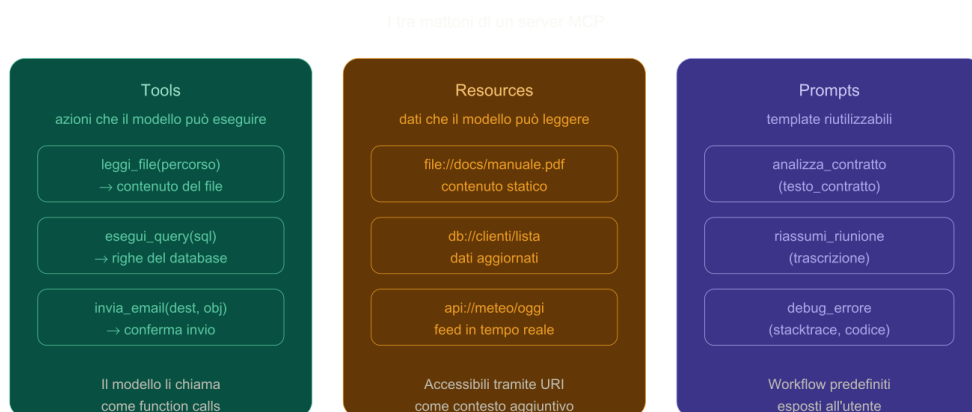
Come mostra la Figura 16.1, l'architettura ha tre componenti:

Host — l'applicazione che l'utente usa direttamente (Claude Desktop, un IDE, la tua app custom). Contiene l'LLM e uno o più client MCP.

Client MCP — componente dentro l'host che gestisce la connessione con i server: scoperta delle capacità, negoziazione del protocollo, invio di richieste e ricezione di risultati.

Server MCP — un processo separato (locale o remoto) che espone strumenti, risorse e prompt. È qui che vive la logica di integrazione con il sistema esterno.

16.2 I tre mattoni: Tools, Resources, Prompts



Un server MCP può esporre uno, due o tutti e tre i mattoni — dipende dal suo scopo
Tools = il modello agisce · Resources = il modello legge · Prompts = l'utente sceglie un workflow

Figura 16.2 — La struttura interna di un server MCP.

Come mostra la Figura 16.2, ogni server MCP può esporre tre tipi di componenti.

Tools — azioni che il modello può eseguire: leggere un file, fare una query SQL, chiamare un'API, inviare un messaggio. Sono l'equivalente dei tool use del Capitolo 15, ma esposti tramite il protocollo standard MCP. Il modello li chiama come function calls e riceve il risultato.

Resources — dati che il modello può leggere come contesto. Sono identificati da URI (come `file:///docs/manuale.pdf` o `db://clienti/schema`) e possono essere statici (un file) o dinamici (una query che si aggiorna). Non sono azioni — il modello non li esegue, li legge.

Prompts — template di workflow predefiniti che l'utente può selezionare dall'interfaccia. Invece di scrivere ogni volta un prompt lungo, l'utente sceglie "Analizza questo contratto" e il server fornisce il prompt strutturato corretto. Utile per workflow aziendali ripetitivi.

16.3 Configurare MCP su Claude Desktop

Claude Desktop supporta MCP nativamente. La configurazione avviene tramite un file JSON che specifica quali server avviare.

Il file di configurazione si trova in:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/tuo_utente/Documenti"
      ]
    },
    "sqlite": {
      "command": "uvx",
      "args": ["mcp-server-sqlite", "--db-path", "/path/to/mydb.db"]
    },
    "mio-server-custom": {
      "command": "python",
      "args": ["/path/to/mio_server_mcp.py"]
    }
  }
}
```

Ogni server ha un `command` (il processo da avviare) e `args` (gli argomenti). Claude Desktop avvia questi processi all'apertura e comunica con loro tramite stdio (standard input/output). Quando riavvii Claude Desktop, i server vengono rilevati automaticamente e i loro tool diventano disponibili nelle conversazioni.

16.4 Il protocollo MCP in dettaglio

MCP usa **JSON-RPC 2.0** come formato di comunicazione — uno standard semplice per chiamate remote. Ogni messaggio è un oggetto JSON con un metodo, parametri e un ID per correlare richieste e risposte.

Il ciclo di vita di una connessione MCP ha tre fasi:

- 1. Inizializzazione** — il client invia `initialize` con la versione del protocollo e le sue capacità. Il server risponde con le proprie capacità. Si stabilisce un "contratto" su cosa entrambi sanno fare.
- 2. Scoperta** — il client chiede la lista dei tool (`tools/list`), delle risorse (`resources/list`) e dei prompt (`prompts/list`). Il server risponde con gli schemi completi di tutto ciò che espone.
- 3. Esecuzione** — il client chiama tool (`tools/call`) o legge risorse (`resources/read`) durante la conversazione. Il server esegue e risponde.

```
# Esempio di messaggio JSON-RPC che il client invia al server
richiesta_list_tools = {
  "jsonrpc": "2.0",
  "id":      1,
  "method":  "tools/list",
  "params":  {}
}

# Risposta del server
risposta_tools = {
  "jsonrpc": "2.0",
  "id":      1,
  "result":  {
```

```

    "tools": [
      {
        "name": "leggi_csv",
        "description": "Legge un file CSV e restituisce le righe come JSON",
        "inputSchema": {
          "type": "object",
          "properties": {
            "percorso": {"type": "string"},
            "max_righe": {"type": "integer", "default": 100}
          },
          "required": ["percorso"]
        }
      }
    ]
  }
}

```

Non devi implementare questo protocollo a mano — la libreria `mcp` di Python lo gestisce completamente. Ma capirne la struttura ti aiuta a debuggare quando qualcosa non funziona.

16.5 Il tuo primo server MCP in Python

La libreria ufficiale `mcp` rende la creazione di un server molto semplice. Il pattern base è: importa `FastMCP`, decora le funzioni con `@mcp.tool()` o `@mcp.resource()`, avvia il server.

```

# pip install mcp
# Salva come: mio_server.py

from mcp.server.fastmcp import FastMCP

# Crea il server con nome e versione
mcp = FastMCP("Il Mio Server", version="1.0.0")

# --- TOOL: azioni che il modello può eseguire ---

@mcp.tool()
def somma(a: float, b: float) -> float:
    """Somma due numeri. Usa questo invece di fare calcoli mentali."""
    return a + b

@mcp.tool()
def leggi_file(percorso: str) -> str:
    """
    Legge il contenuto di un file di testo.

    Args:
        percorso: percorso assoluto o relativo al file
    Returns:
        contenuto del file come stringa
    """
    try:
        with open(percorso, "r", encoding="utf-8") as f:
            return f.read()
    except:
        return ""

```



```

except FileNotFoundError:
    return f"Errore: file '{percorso}' non trovato"
except Exception as e:
    return f"Errore nella lettura: {e}"

@mcp.tool()
def scrivi_file(percorso: str, contenuto: str) -> str:
    """
    Scrive contenuto in un file. Crea il file se non esiste.

    Args:
        percorso: percorso del file da scrivere
        contenuto: testo da scrivere nel file
    Returns:
        messaggio di conferma o errore
    """
    try:
        with open(percorso, "w", encoding="utf-8") as f:
            f.write(contenuto)
        return f"File '{percorso}' scritto correttamente ({len(contenuto)} caratteri)"
    except Exception as e:
        return f"Errore nella scrittura: {e}"

# --- RESOURCE: dati che il modello può leggere come contesto ---

@mcp.resource("config://impostazioni")
def impostazioni_app() -> str:
    """Restituisce le impostazioni correnti dell'applicazione."""
    import json
    config = {
        "versione": "1.0.0",
        "lingua": "it",
        "max_righe": 1000,
        "debug": False
    }
    return json.dumps(config, indent=2)

@mcp.resource("docs://istruzioni")
def istruzioni() -> str:
    """Documentazione interna su come usare questo server."""
    return ""
# Istruzioni d'uso

Questo server espone tool per gestire file di testo.

## Tool disponibili
- somma(a, b): calcola la somma di due numeri
- leggi_file(percorso): legge un file
- scrivi_file(percorso, contenuto): scrive un file

## Note importanti
- I percorsi possono essere relativi alla directory di avvio del server
- Il server non ha accesso a file fuori dalla directory configurata
"""

```

```
# --- PROMPT: workflow predefiniti ---

@mcp.prompt()
def analizza_file(percorso: str) -> str:
    """
    Workflow per analizzare un file di testo.
    Legge il file e produce un'analisi strutturata.
    """
    return f"""Leggi il file '{percorso}' usando il tool leggi_file,
poi produci un'analisi strutturata con:
1. Tipo di contenuto (codice, testo, dati, altro)
2. Lunghezza approssimativa (righe e caratteri)
3. Struttura principale (sezioni, funzioni, colonne, ecc.)
4. Eventuali problemi o anomalie evidenti
5. Un riassunto in 2-3 righe"""

# Avvia il server (modalità stdio per Claude Desktop)
if __name__ == "__main__":
    mcp.run()
```

Per testare il server da riga di comando prima di collegarlo a Claude Desktop:

```
# Installa la libreria
pip install mcp

# Avvia il server in modalità development (mostra log dettagliati)
mcp dev mio_server.py

# Oppure avvialo direttamente
python mio_server.py
```

16.6 Server MCP per esporre dati da CSV

Il caso d'uso più comune per chi inizia: vuoi che Claude possa interrogare i tuoi dati in un file CSV senza doverli copiare ogni volta nel prompt.

```
# Salva come: server_csv.py
import json
import csv
import os
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Analisi CSV", version="1.0.0")

# Directory sicura - il server accede SOLO ai file qui dentro
DIR_DATI = os.path.expanduser("~/dati_mcp")
os.makedirs(DIR_DATI, exist_ok=True)

def percorso_sicuro(nome_file: str) -> str:
    """
```

```

    Costruisce un percorso sicuro dentro DIR_DATI.
    Previene path traversal (es. '../../../etc/passwd').
    """
    percorso = os.path.realpath(os.path.join(DIR_DATI, nome_file))
    if not percorso.startswith(os.path.realpath(DIR_DATI)):
        raise ValueError(f"Accesso negato: '{nome_file}' è fuori dalla directory
dati")
    return percorso

@mcp.tool()
def lista_csv() -> str:
    """Elenca tutti i file CSV disponibili nella directory dati."""
    try:
        files = [f for f in os.listdir(DIR_DATI) if f.endswith(".csv")]
        if not files:
            return f"Nessun file CSV in {DIR_DATI}"
        return json.dumps({"file": files, "directory": DIR_DATI})
    except Exception as e:
        return f"Errore: {e}"

@mcp.tool()
def leggi_csv(nome_file: str, max_righe: int = 50) -> str:
    """
    Legge un file CSV e restituisce le righe come JSON.

    Args:
        nome_file: nome del file CSV (solo nome, non percorso completo)
        max_righe: numero massimo di righe da restituire (default 50)
    Returns:
        JSON con colonne e righe del CSV
    """
    try:
        percorso = percorso_sicuro(nome_file)
        with open(percorso, newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            righe = [row for row in reader]

        totale = len(righe)
        righe = righe[:max_righe]

        return json.dumps({
            "file": nome_file,
            "colonne": list(righe[0].keys()) if righe else [],
            "righe": righe,
            "totale_righe": totale,
            "mostrate": len(righe)
        }, ensure_ascii=False)

    except ValueError as e:
        return f"Errore di sicurezza: {e}"
    except FileNotFoundError:
        return f"File '{nome_file}' non trovato in {DIR_DATI}"
    except Exception as e:
        return f"Errore: {e}"

```

```

@mcp.tool()
def statistiche_csv(nome_file: str, colonna: str) -> str:
    """
    Calcola statistiche di base per una colonna numerica di un CSV.

    Args:
        nome_file: nome del file CSV
        colonna: nome della colonna da analizzare
    Returns:
        JSON con min, max, media, mediana e conteggio
    """
    try:
        percorso = percorso_sicuro(nome_file)
        with open(percorso, newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            valori_raw = [row.get(colonna) for row in reader]

        # Converti in float, ignora valori non numerici
        valori = []
        for v in valori_raw:
            try:
                valori.append(float(v))
            except (TypeError, ValueError):
                pass

        if not valori:
            return f"Nessun valore numerico trovato nella colonna '{colonna}'"

        valori_ord = sorted(valori)
        n = len(valori)
        mediana = (valori_ord[n//2-1] + valori_ord[n//2]) / 2 if n % 2 == 0 else
        valori_ord[n//2]

        return json.dumps({
            "colonna": colonna,
            "conteggio": n,
            "min": min(valori),
            "max": max(valori),
            "media": round(sum(valori) / n, 4),
            "mediana": round(mediana, 4)
        })

    except Exception as e:
        return f"Errore: {e}"

@mcp.tool()
def filtra_csv(nome_file: str, colonna: str, valore: str) -> str:
    """
    Filtra le righe di un CSV dove una colonna contiene un valore specifico.

    Args:
        nome_file: nome del file CSV
        colonna: colonna su cui filtrare
        valore: valore da cercare (confronto stringa, case-insensitive)
    Returns:
        JSON con le righe che corrispondono al filtro
    """

```

```

try:
    percorso = percorso_sicuro(nome_file)
    with open(percorso, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        filtrate = [
            row for row in reader
            if str(row.get(colonna, "")).lower() == valore.lower()
        ]

    return json.dumps({
        "filtro": f"{colonna} = '{valore}'",
        "risultati": len(filtrate),
        "righe": filtrate[:100] # max 100 righe nel risultato
    }, ensure_ascii=False)

except Exception as e:
    return f"Errore: {e}"

# Resource: schema dei CSV disponibili
@mcp.resource("csv://schema")
def schema_disponibili() -> str:
    """Restituisce lo schema (colonne) di tutti i CSV nella directory dati."""
    try:
        schema = {}
        for f in os.listdir(DIR_DATI):
            if not f.endswith(".csv"):
                continue
            percorso = os.path.join(DIR_DATI, f)
            with open(percorso, newline="", encoding="utf-8") as fh:
                reader = csv.DictReader(fh)
                prima = next(reader, None)
                schema[f] = list(prima.keys()) if prima else []
        return json.dumps(schema, ensure_ascii=False, indent=2)
    except Exception as e:
        return f"Errore: {e}"

if __name__ == "__main__":
    print(f"Server CSV avviato. Directory dati: {DIR_DATI}")
    print("Copia i tuoi file .csv in quella directory, poi connetti Claude Desktop.")
    mcp.run()

```

16.7 Aggiungere il server a Claude Desktop

Dopo aver scritto il server, aggiungilo al file di configurazione di Claude Desktop:

```

{
  "mcpServers": {
    "analisi-csv": {
      "command": "python",
      "args": ["/percorso/assoluto/server_csv.py"]
    }
  }
}

```

Riavvia Claude Desktop. Nella barra degli strumenti vedrai apparire i tool del tuo server. Da quel momento puoi scrivere nella chat cose come:

```
Elenca i CSV disponibili, poi analizza le vendite del Q3 nel file vendite_2025.csv.  
Dammi le statistiche della colonna "importo" e filtra per regione="Nord".
```

Claude chiamerà automaticamente `lista_csv()`, `leggi_csv()`, `statistiche_csv()` e `filtra_csv()` in sequenza, senza che tu debba fare nulla di più.

16.8 Server MCP esistenti e come sceglierli

Non devi costruire un server per ogni integrazione — molti esistono già. Il repository ufficiale `modelcontextprotocol/servers` su GitHub mantiene un elenco aggiornato. Alcuni dei più usati:

Server	Cosa fa	Install	----- ----- -----	@modelcontextprotocol/server-filesystem
Accesso al filesystem locale		<code>npx -y @modelcontextprotocol/server-filesystem /path</code>		<code>mcp-server-sqlite</code>
Query su database SQLite		<code>uvx mcp-server-sqlite --db-path db.sqlite</code>		@modelcontextprotocol/server-github
Repository, PR, issues GitHub				Richiede token GitHub
@modelcontextprotocol/server-postgres	Query su PostgreSQL			Richiede connection string
mcp-server-fetch	Scarica pagine web	<code>uvx mcp-server-fetch</code>		@modelcontextprotocol/server-memory
Memoria persistente key-value		<code>npx -y @modelcontextprotocol/server-memory</code>		

Come scegliere: inizia dai server ufficiali per le integrazioni standard. Costruisci un server custom solo quando hai dati o sistemi proprietari che nessun server esistente copre. Prima di costruire, cerca su GitHub — la community cresce velocemente.

Riepilogo del Capitolo 16

Hai imparato:

- **Il problema di MCP:** le integrazioni custom tra LLM e strumenti non scalano. MCP è il protocollo standard che permette a qualsiasi server di funzionare con qualsiasi client.
- **Architettura Host/Client/Server:** l'host contiene l'LLM e il client MCP; i server girano come processi separati e comunicano tramite JSON-RPC su stdio o HTTP.
- **I tre mattoni:** Tools (azioni eseguibili), Resources (dati leggibili tramite URI), Prompts (workflow predefiniti per l'utente).
- **Configurazione Claude Desktop:** un file JSON specifica quali server avviare; i tool diventano disponibili automaticamente nelle conversazioni.
- **FastMCP in Python:** decoratori `@mcp.tool()`, `@mcp.resource()`, `@mcp.prompt()` per esporre funzioni al protocollo con minimo boilerplate.
- **Sicurezza:** valida sempre i percorsi con `os.path.realpath` per prevenire path traversal; limita l'accesso alla directory minima necessaria.
- **Server esistenti:** usa quelli ufficiali per integrazioni standard; costruisci custom solo per sistemi proprietari.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: costruire un server MCP completo che espone dati da un sistema reale di tua scelta, con tool, resource e almeno un prompt.

Traccia:

Scegli uno di questi scenari (o un tuo equivalente):

- Server per analisi di log applicativi (file `.log` con timestamp, livello, messaggio).
 - Server per gestione di un catalogo prodotti (CSV con nome, prezzo, categoria, stock).
 - Server per note personali (file Markdown in una directory).
49. Costruisci il server con almeno quattro tool, una resource e un prompt. I tool devono coprire operazioni CRUD: crea, leggi, aggiorna, cerca.
50. Aggiungi **validazione degli input** robusta: ogni tool deve gestire esplicitamente tutti i casi di errore possibili (file non trovato, colonna inesistente, formato non valido) e restituire messaggi di errore informativi invece di eccezioni non gestite.
51. Aggiungi un meccanismo di **audit log**: ogni chiamata ai tool che modifica dati (scrittura, aggiornamento, cancellazione) deve essere registrata in un file di log con timestamp, nome del tool, parametri usati e risultato.
52. Connetti il server a Claude Desktop e conduci una conversazione di almeno 10 turni dove usi tutti i tool. Salva il log delle chiamate.
53. **Domanda di riflessione:** durante la sessione, Claude ha mai chiamato un tool con parametri errati o in un ordine non ottimale? Come potresti migliorare le descrizioni dei tool (`@mcp.tool()` accetta una docstring) per guidare meglio il modello verso l'uso corretto?

Consegna: il file `server.py` completo, il file di configurazione `claude_desktop_config.json`, il log della sessione di 10 turni, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Prossimo capitolo: Hardware e CUDA — capiremo perché la GPU è indispensabile, come funziona la memoria della GPU e come ottimizzare l'uso della VRAM quando i modelli diventano grandi.

Capitolo 17

Hardware e CUDA

"Puoi capire tutto il deep learning senza capire l'hardware. Ma non puoi ottimizzare un modello reale senza sapere dove finiscono i dati e perché certi calcoli sono lenti."

17.1 Perché la GPU è indispensabile

Nel Capitolo 8 hai usato `torch.cuda.is_available()` senza pensarci troppo. È il momento di capire perché questa riga di codice cambia così tanto.

Una **CPU** moderna ha 8–64 core. Ogni core è potente e generico — sa fare qualsiasi operazione in modo efficiente, gestisce branch complessi, è ottimizzata per la latenza (finire un compito velocemente).

Una **GPU** moderna ha migliaia di core piccoli. La RTX 4090 ne ha 16.384. Ogni core è molto più semplice di un core CPU, ma insieme possono eseguire migliaia di operazioni matematiche in parallelo. La GPU è ottimizzata per il throughput — fare molte cose contemporaneamente.

Il training di una rete neurale è quasi interamente fatto di moltiplicazioni di matrici. Moltiplicare una matrice 4096×4096 per un'altra significa fare $4096^3 \approx 68$ miliardi di moltiplicazioni e addizioni. Su CPU, sequenzialmente, questo richiede secondi. Su GPU, in parallelo su migliaia di core, richiede millisecondi.

***Analogia:** la CPU è un maestro artigiano — fa tutto benissimo ma uno alla volta. La GPU è una fabbrica con migliaia di operai specializzati — ogni singolo operaio è meno capace del maestro, ma insieme producono molto di più nell'unità di tempo.*

17.2 CUDA, CUDA Core e Tensor Core

CUDA (Compute Unified Device Architecture) è la piattaforma di calcolo parallelo di NVIDIA che permette di scrivere codice che gira sulla GPU. PyTorch usa CUDA internamente — quando scrivi `tensor.to("cuda")`, stai spostando i dati nella memoria della GPU e tutte le operazioni successive useranno CUDA.

Dentro una GPU NVIDIA ci sono due tipi di unità di calcolo rilevanti per il deep learning.

CUDA Core — l'unità base. Esegue operazioni su un singolo numero alla volta (FP32, FP16, INT32). È il "lavoratore generico" della GPU.

Tensor Core — introdotti con l'architettura Volta (2017), sono specializzati per moltiplicazioni di matrici piccole (tipicamente 4×4 o 16×16) con precisione mista. Eseguono un'operazione del tipo $D = A \times B + C$ in un singolo ciclo di clock, dove A, B, C, D sono matrici piccole. Su un H100, i Tensor Core arrivano a 3.958 TFLOPS in BF16 — decine di volte più veloci dei CUDA Core per le operazioni di deep learning.

PyTorch sfrutta automaticamente i Tensor Core quando usi BF16 o FP16 e le dimensioni delle matrici sono multiple di 16. Non devi fare nulla di speciale — ma devi usare il formato numerico corretto.

```
import torch
import time

device = "cuda"
N = 4096

# FP32 — usa CUDA Core
a_fp32 = torch.randn(N, N, device=device, dtype=torch.float32)
b_fp32 = torch.randn(N, N, device=device, dtype=torch.float32)

torch.cuda.synchronize()
t0 = time.time()
for _ in range(100):
    _ = a_fp32 @ b_fp32
torch.cuda.synchronize()
print(f"FP32: {(time.time()-t0)*10:.1f} ms per operazione")

# BF16 — usa Tensor Core (molto più veloce)
a_bf16 = a_fp32.to(torch.bfloat16)
b_bf16 = b_fp32.to(torch.bfloat16)

torch.cuda.synchronize()
t0 = time.time()
for _ in range(100):
    _ = a_bf16 @ b_bf16
torch.cuda.synchronize()
print(f"BF16: {(time.time()-t0)*10:.1f} ms per operazione")
# Su una T4: BF16 è tipicamente 2-4x più veloce di FP32
```

Nota: `torch.cuda.synchronize()` è necessario per misurare tempi accurati. Le operazioni GPU sono asincrone — senza `synchronize` il timer misurerebbe solo il tempo di invio delle istruzioni, non di esecuzione.

17.3 La gerarchia di memoria GPU

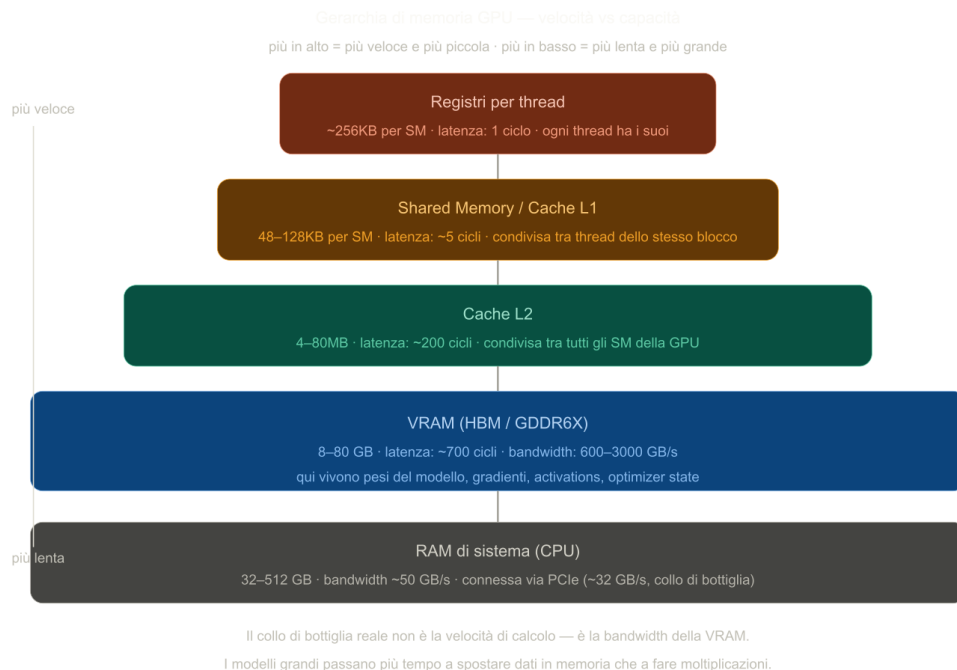


Figura 17.1 — La gerarchia di memoria della GPU.

Come mostra la Figura 17.1, la memoria di una GPU ha più livelli con caratteristiche molto diverse. Capire questa gerarchia spiega perché certi pattern di codice sono veloci e altri lenti.

Registri — la memoria più veloce e più piccola. Ogni thread CUDA ha i suoi registri privati. Latenza di 1 ciclo. Usati automaticamente dal compilatore CUDA per le variabili locali.

Shared Memory / Cache L1 — condivisa tra i thread dello stesso blocco CUDA. Latenza ~5 cicli. Fondamentale per ottimizzazioni manuali in CUDA C, ma PyTorch la gestisce automaticamente.

Cache L2 — condivisa tra tutti gli SM (Streaming Multiprocessor) della GPU. Latenza ~200 cicli. Le GPU moderne hanno cache L2 sempre più grandi (fino a 80MB su H100) per ridurre gli accessi alla VRAM.

VRAM — la memoria principale della GPU. Qui vivono i pesi del modello, i gradienti, le activations, lo stato dell'ottimizzatore. Latenza ~700 cicli ma bandwidth altissima (fino a 3.35 TB/s su H100 con HBM3e). È qui che si verifica l'**out of memory** (OOM) tanto temuto.

RAM di sistema — connessa alla GPU tramite PCIe (~64 GB/s bidirezionale). Lentissima rispetto alla VRAM. Spostare dati tra RAM e VRAM è uno dei colli di bottiglia più comuni.

Cosa occupa la VRAM durante il training

Quando addestri un modello, la VRAM deve contenere:

```
VRAM totale ≈ parametri × (  
    2 byte [pesi in BF16]  
    + 2 byte [gradienti in BF16]  
    + 8 byte [stato ottimizzatore Adam: m e v in FP32]  
) + activations (variabile, dipende dal batch size)
```

Per un modello da 7B di parametri:

- Pesì BF16: $7B \times 2 \text{ byte} = 14 \text{ GB}$
- Gradienti BF16: **14 GB**
- Stato Adam FP32: $7B \times 8 \text{ byte} = 56 \text{ GB}$
- **Totale minimo: ~84 GB** — serve un A100 80GB o più GPU in parallelo

Questo spiega perché il fine-tuning completo è impraticabile su hardware consumer, e perché LoRA e la quantizzazione (prossime sezioni) sono così importanti.

17.4 Quantizzazione: più modello con meno VRAM

La **quantizzazione** riduce la precisione numerica dei pesi del modello, diminuendo drasticamente la VRAM necessaria con un impatto limitato sulla qualità.

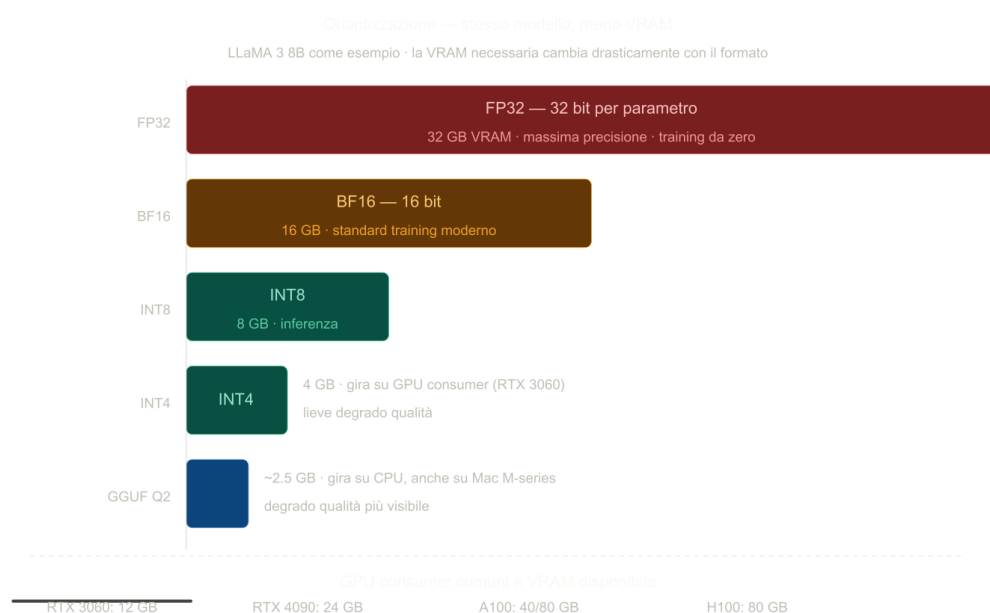


Figura 17.2 — Il confronto tra i formati di quantizzazione.

Come mostra la Figura 17.2, il formato numerico determina quanta VRAM serve per ogni parametro. Un modello da 8B di parametri richiede:

Formato	Bit per param	VRAM (8B param)	Uso tipico
FP32	32	32 GB	Training da zero, massima precisione
BF16	16	16 GB	Training moderno, inferenza di qualità
INT8	8	8 GB	Inferenza, minima perdita di qualità
INT4	4	4 GB	Inferenza su GPU consumer
GGUF Q2	~2.5	~2.5 GB	Inferenza su CPU o Mac

BF16 vs FP16 — entrambi usano 16 bit, ma con distribuzioni diverse. BF16 ha lo stesso range dinamico di FP32 (stessa dimensione dell'esponente) ma meno precisione decimale. FP16 ha più precisione ma range più limitato — può andare in overflow durante il training. Per questo motivo BF16 è il formato standard per il training moderno.

```
import torch

# Confronto tra formati numerici
x_fp32 = torch.tensor(3.14159265358979, dtype=torch.float32)
x_bf16 = x_fp32.to(torch.bfloat16)
x_fp16 = x_fp32.to(torch.float16)
```

```

x_int8 = x_fp32.to(torch.int8)    # troncato a intero!

print(f"FP32:  {x_fp32.item():.10f}  ({x_fp32.element_size()} byte)")
print(f"BF16:  {x_bf16.item():.10f}  ({x_bf16.element_size()} byte)")
print(f"FP16:  {x_fp16.item():.10f}  ({x_fp16.element_size()} byte)")
print(f"INT8:   {x_int8.item()}        ({x_int8.element_size()} byte)")
# FP32:   3.1415927410   (4 byte)
# BF16:   3.1406250000   (2 byte) – perde precisione ma mantiene ordine di grandezza
# FP16:   3.1406250000   (2 byte)
# INT8:    3              (1 byte) – inutile per pesi neurali senza de-quantizzazione

```

Quantizzazione in PyTorch con bitsandbytes

```

# pip install bitsandbytes transformers accelerate
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

# Configurazione per quantizzazione 4-bit (NF4 = NormalFloat4, ottimizzato per pesi
di LLM)
config_4bit = BitsAndBytesConfig(
    load_in_4bit                = True,
    bnb_4bit_compute_dtype      = torch.bfloat16,    # calcoli in BF16
    bnb_4bit_use_double_quant   = True,              # doppia quantizzazione: risparmia
~0.4 GB
    bnb_4bit_quant_type         = "nf4"              # NF4: migliore per distribuzioni
gaussiane
)

# Carica il modello già quantizzato – occupa ~4GB invece di ~16GB
modello = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3-8B",    # richiede accesso su HuggingFace
    quantization_config = config_4bit,
    device_map          = "auto"      # distribuisce automaticamente su GPU
disponibili
)

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")

# Calcola la VRAM effettivamente usata
vram_usata = sum(
    p.numel() * p.element_size()
    for p in modello.parameters()
) / 1e9
print(f"VRAM stimata dai parametri: {vram_usata:.2f} GB")

# Inferenza normale – la quantizzazione è trasparente
input_ids = tokenizer("La capitale d'Italia è", return_tensors="pt").to("cuda")
with torch.no_grad():
    output = modello.generate(**input_ids, max_new_tokens=50)
print(tokenizer.decode(output[0], skip_special_tokens=True))

```

17.5 Scegliere l'hardware: consumer GPU, cloud, locale

La scelta dell'hardware dipende da cosa vuoi fare. Ecco una guida pratica.

Per iniziare (prototipazione, corso): Google Colab con GPU T4 gratuita. Limitata (16GB VRAM, ore di runtime giornaliero) ma sufficiente per tutti gli esercizi di questo volume. Zero costo, zero configurazione.

Per progetti seri su hardware locale:

| GPU | VRAM | Uso consigliato | |----|----|-----| | RTX 3060 12GB | 12 GB | Fine-tuning LoRA di modelli 7B, inferenza INT4 di modelli 13B | | RTX 4090 24GB | 24 GB | Fine-tuning BF16 di modelli 7B, inferenza di modelli 34B quantizzati | | RTX 3090 24GB | 24 GB | Alternativa più economica alla 4090, meno efficiente sui Tensor Core | | 2× RTX 4090 | 48 GB | Fine-tuning BF16 di modelli 13B, inferenza di modelli 70B |

Per training serio o modelli grandi: Cloud GPU — A100 (40/80GB) o H100 (80GB) su AWS, GCP, Azure o provider specializzati (Lambda Labs, RunPod, Vast.ai). Lambda Labs e RunPod sono tipicamente 3–5× più economici dei cloud mainstream per GPU dedicate al ML.

Per Mac con Apple Silicon: I modelli GGUF girano su llama.cpp usando la memoria unificata (RAM e VRAM condivise). Un Mac M3 Pro con 36GB di RAM può eseguire modelli fino a ~34B in qualità Q4. Non è adatto per il training, ma per l'inferenza locale è eccellente.

17.6 Monitorare memoria e utilizzo GPU

Prima di ottimizzare, devi misurare. Ecco gli strumenti essenziali.

```
import torch

def stato_gpu():
    """Stampa l'utilizzo corrente della VRAM."""
    if not torch.cuda.is_available():
        print("Nessuna GPU disponibile")
        return

    props = torch.cuda.get_device_properties(0)
    allocata = torch.cuda.memory_allocated(0) / 1e9
    riservata = torch.cuda.memory_reserved(0) / 1e9
    totale = props.total_memory / 1e9

    print(f"GPU: {props.name}")
    print(f"VRAM allocata: {allocata:.2f} GB")
    print(f"VRAM riservata: {riservata:.2f} GB")    # include la cache PyTorch
    print(f"VRAM totale: {totale:.2f} GB")
    print(f"VRAM libera: {totale - riservata:.2f} GB")

stato_gpu()

# Profiling con torch.profiler — identifica dove il modello passa il tempo
from torch.profiler import profile, ProfilerActivity, record_function

modello_test = torch.nn.Linear(1024, 1024).cuda()
x_test = torch.randn(64, 1024, device="cuda")

with profile(
    activities = [ProfilerActivity.CPU, ProfilerActivity.CUDA],
```

```

        record_shapes = True,
        profile_memory = True
    ) as prof:
        with record_function("forward"):
            y = modello_test(x_test)
        with record_function("backward"):
            y.sum().backward()

# Stampa le operazioni più costose
print(prof.key_averages().table(
    sort_by = "cuda_time_total",
    row_limit = 10
))

```

Da riga di comando, `nvidia-smi` mostra l'utilizzo in tempo reale:

```

# Utilizzo istantaneo
nvidia-smi

# Aggiornamento ogni secondo
nvidia-smi dloop -d 1

# Solo VRAM e utilizzo GPU
nvidia-smi --query-gpu=memory.used,memory.free,utilization.gpu --format=csv -l 1

```

17.7 Esercizio guidato — profila un modello e ottimizza la VRAM

Confrontiamo l'uso di VRAM di uno stesso modello in FP32, BF16 e INT8, e misuriamo l'impatto sulla velocità di inferenza.

```

# =====
# Profiling VRAM e velocità - esegui su Google Colab T4
# =====
import torch
import torch.nn as nn
import time

device = "cuda"

# --- Modello di test: Transformer encoder medio ---
class TransformerTest(nn.Module):
    def __init__(self, d=512, n_heads=8, n_layers=6):
        super().__init__()
        layer = nn.TransformerEncoderLayer(
            d_model=d, nhead=n_heads, dim_feedforward=d*4,
            batch_first=True, norm_first=True # Pre-LN: più stabile
        )
        self.encoder = nn.TransformerEncoder(layer, num_layers=n_layers)
        self.out = nn.Linear(d, 1000)

    def forward(self, x):
        return self.out(self.encoder(x).mean(dim=1))

```

```

def benchmark(modello, dtype_name, batch=32, seq=128, d=512, n_iter=50):
    """Misura VRAM e latenza di inferenza per un dato dtype."""
    torch.cuda.empty_cache()
    torch.cuda.reset_peak_memory_stats()

    modello.eval()
    x = torch.randn(batch, seq, d, device=device,
dtype=next(modello.parameters()).dtype)

    # Warmup
    with torch.no_grad():
        for _ in range(5):
            _ = modello(x)
    torch.cuda.synchronize()

    # Misura latenza
    t0 = time.time()
    with torch.no_grad():
        for _ in range(n_iter):
            _ = modello(x)
    torch.cuda.synchronize()
    latenza_ms = (time.time() - t0) / n_iter * 1000

    vram_peak = torch.cuda.max_memory_allocated() / 1e9

    print(f" {dtype_name:6s} | VRAM picco: {vram_peak:.2f} GB | "
          f"Latenza: {latenza_ms:.1f} ms/batch")
    return vram_peak, latenza_ms

print("=== Benchmark Transformer (d=512, 6 layer, batch=32) ===\n")

# FP32
m_fp32 = TransformerTest().to(device, dtype=torch.float32)
v32, l32 = benchmark(m_fp32, "FP32")

# BF16
m_bf16 = TransformerTest().to(device, dtype=torch.bfloat16)
v16, l16 = benchmark(m_bf16, "BF16")

# FP16
m_fp16 = TransformerTest().to(device, dtype=torch.float16)
v_fp16, l_fp16 = benchmark(m_fp16, "FP16")

# INT8 via torch.quantization
m_int8 = TransformerTest().to(device, dtype=torch.float32)
m_int8 = torch.quantization.quantize_dynamic(
    m_int8.cpu(), {nn.Linear}, dtype=torch.qint8
).to(device)
v8, l8 = benchmark(m_int8, "INT8")

print(f"\nRisparmio VRAM BF16 vs FP32: {(1 - v16/v32)*100:.0f}%")
print(f"Speedup BF16 vs FP32: {l32/l16:.1f}x")

# --- Gradient checkpointing: riduce VRAM delle activations durante training ---
class TransformerConCheckpoint(nn.Module):

```

```

"""Usa gradient checkpointing per ridurre la VRAM delle activations."""
def __init__(self, d=512, n_heads=8, n_layers=6):
    super().__init__()
    layer = nn.TransformerEncoderLayer(
        d_model=d, nhead=n_heads, dim_feedforward=d*4,
        batch_first=True, norm_first=True
    )
    self.encoder = nn.TransformerEncoder(
        layer, num_layers=n_layers,
        enable_nested_tensor=False
    )
    self.out = nn.Linear(d, 1000)
    # Abilita gradient checkpointing:
    # non salva le activations intermediate - le ricalcola durante il backward
    # Costa ~33% in più di tempo ma dimezza la VRAM delle activations
    self.encoder.enable_nested_tensor = False

def forward(self, x):
    from torch.utils.checkpoint import checkpoint_sequential
    x = checkpoint_sequential(self.encoder.layers, 2, x)
    return self.out(x.mean(dim=1))

print("\n=== Gradient checkpointing (training mode) ===")
torch.cuda.empty_cache()
torch.cuda.reset_peak_memory_stats()

m_ck = TransformerConCheckpoint().to(device, dtype=torch.bfloat16)
x = torch.randn(32, 128, 512, device=device, dtype=torch.bfloat16,
requires_grad=True)
y = m_ck(x)
y.sum().backward()
torch.cuda.synchronize()

vram_ck = torch.cuda.max_memory_allocated() / 1e9
print(f" BF16 + checkpoint | VRAM picco: {vram_ck:.2f} GB")
print(f" Risparmio vs BF16 senza checkpoint: {(1 - vram_ck/v16)*100:.0f}%")

```

Riepilogo del Capitolo 17

Hai imparato:

- **Perché la GPU:** migliaia di core piccoli in parallelo vs. decine di core potenti in serie. La GPU vince sulle moltiplicazioni di matrici perché il deep learning è quasi interamente fatto di quelle.
- **CUDA Core vs Tensor Core:** i Tensor Core sono specializzati per moltiplicazioni di matrici in precisione mista e sono ordini di grandezza più veloci. Si attivano automaticamente con BF16/FP16 e dimensioni multiple di 16.
- **Gerarchia di memoria:** registri → shared/L1 → L2 → VRAM → RAM. Il collo di bottiglia reale è la bandwidth della VRAM, non la velocità di calcolo.
- **Cosa occupa la VRAM:** pesi + gradienti + stato ottimizzatore + activations. Per il training con Adam, pianifica ~6 byte per parametro solo per i pesi e l'ottimizzatore.
- **Quantizzazione:** FP32 → BF16 → INT8 → INT4 → GGUF. Ogni dimezzamento dei bit dimezza la VRAM necessaria con impatto progressivamente maggiore sulla qualità.

- **Scegliere l'hardware:** Colab per iniziare, RTX 4090 per lavoro locale serio, cloud A100/H100 per training di modelli grandi.
- **Monitorare:** `torch.cuda.memory_allocated()`, `nvidia-smi`, `torch.profiler` per trovare i colli di bottiglia prima di ottimizzare.

Esercizio Finale — Da svolgere autonomamente

Obiettivo: ottimizzare sistematicamente l'uso di VRAM di un modello reale, documentando il trade-off qualità/memoria per ogni tecnica applicata.

Traccia:

Prendi il classificatore CNN del Capitolo 9 (o il Transformer del Capitolo 11) e applica una serie di ottimizzazioni in sequenza, misurando VRAM e accuracy dopo ogni passo.

54. Stabilisci la baseline: addestra il modello in FP32 per 5 epoche su CIFAR-10. Registra VRAM di picco, tempo per epoca e accuracy finale.

55. Applica le ottimizzazioni una alla volta, misurando l'impatto di ognuna:

```
# Ottimizzazione A: BF16 con automatic mixed precision
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()

for X, y in train_loader:
    ottimizzatore.zero_grad()
    with autocast(dtype=torch.bfloat16): # forward in BF16
        pred = modello(X.to(device))
        loss = criterio(pred, y.to(device))
    scaler.scale(loss).backward() # backward in FP32 per stabilità
    scaler.step(ottimizzatore)
    scaler.update()

# Ottimizzazione B: gradient checkpointing (dopo AMP)
modello.gradient_checkpointing_enable() # su modelli HuggingFace

# Ottimizzazione C: batch size più grande (ora che hai più VRAM libera)
# Ottimizzazione D: gradient accumulation (simula batch grande senza più VRAM)
ACCUM_STEPS = 4
for i, (X, y) in enumerate(train_loader):
    loss = criterio(modello(X.to(device)), y.to(device)) / ACCUM_STEPS
    loss.backward()
    if (i + 1) % ACCUM_STEPS == 0:
        ottimizzatore.step()
        ottimizzatore.zero_grad()
```

56. Costruisci una tabella con i risultati di ogni configurazione: tecnica applicata, VRAM di picco, tempo per epoca, accuracy test. Quale combinazione offre il miglior rapporto VRAM/accuracy?
57. Usa `nvidia-smi dloop -d 1` (in un terminale separato) durante il training per osservare l'utilizzo della GPU nel tempo. L'utilizzo è costantemente alto (>90%) o ci sono periodi di idle? Cosa indica un utilizzo basso?
58. **Domanda di riflessione:** nel training con AMP, il forward pass è in BF16 ma il backward è in FP32. Perché questa asimmetria? Cosa succederebbe se facessimo anche il backward in BF16?

Consegna: il notebook con la tabella comparativa delle ottimizzazioni, i grafici di VRAM nel tempo, e le risposte ai punti 4 e 5.

(Soluzione commentata in Appendice E.)

Progetto di Fine Volume 2

Sei arrivato alla fine del Volume 2. Nei capitoli 8–17 hai costruito una comprensione completa dello stack tecnico del deep learning moderno: da PyTorch ai Transformer, dagli LLM agli agenti, dal RAG all'MCP, fino all'hardware che fa girare tutto.

Il **progetto finale** integra tutto in un sistema reale:

Chatbot RAG con agente integrato e MCP server custom.

Il sistema deve:

59. Indicizzare una knowledge base di documenti PDF (almeno 5 documenti, almeno 50 pagine totali).
60. Rispondere a domande sulla knowledge base usando RAG (Cap. 14).
61. Quando la risposta non è nei documenti, cercare sul web usando un agente ReAct (Cap. 15).
62. Esporre la knowledge base tramite un server MCP (Cap. 16) così che Claude Desktop possa interrogarla direttamente.
63. Usare BF16 e quantizzazione INT8 per ottimizzare l'uso di VRAM se si usa un modello locale (Cap. 17).

La checklist di consegna del progetto:

- Notebook Colab funzionante con RAG + agente
- Server MCP Python collegato a Claude Desktop
- Documentazione delle scelte architetturali (perché RAG invece di fine-tuning? Quale modello di embedding? Quale strategia di chunking?)
- Evaluation: test set di 20 domande con score di qualità usando LLM-as-Judge
- Analisi degli errori: almeno 3 casi dove il sistema ha sbagliato e come potresti migliorarlo

Volume 3 — Produrre in sicurezza: dalla prototipazione alla produzione. Fine-tuning, deploy con Docker, sicurezza contro prompt injection, e come usare l'IA nel lavoro quotidiano.

Appendice A

Glossario Completo

Ogni termine tecnico spiegato in italiano semplice

Come usare questo glossario: i termini sono ordinati alfabeticamente. Quando incontri una parola che non conosci durante la lettura, cerca qui la definizione in linguaggio semplice. I riferimenti tra parentesi indicano il capitolo in cui il termine viene approfondito.

A

Accuracy (Accuratezza) La frazione di predizioni corrette sul totale. Se il modello classifica correttamente 90 email su 100, l'accuracy è 90%. È la metrica più intuitiva, ma può essere fuorviante su dataset sbilanciati: un modello che dice sempre "non spam" ottiene il 99% di accuracy se lo spam è l'1% dei dati, pur essendo completamente inutile. (Cap. 6, 19)

Agente IA Un sistema in cui un LLM non si limita a rispondere a domande, ma pianifica ed esegue sequenze di azioni usando strumenti esterni. A differenza di un chatbot, un agente agisce nel mondo: naviga il web, scrive file, chiama API. La differenza fondamentale è la responsabilità: un chatbot risponde, un agente agisce con conseguenze reali. (Cap. 15)

Adam / AdamW Ottimizzatore che combina il momentum (memoria della direzione) con l'adattamento del learning rate per ogni peso. AdamW corregge un difetto di Adam separando il weight decay dal calcolo del gradiente. Entrambi sono lo standard per addestrare LLM e Transformer moderni. (Cap. 7)

Allucinazione (Hallucination) Quando un LLM genera informazioni false dette con sicurezza, come se fossero fatti verificati. Non è un bug nel codice — è una caratteristica intrinseca dei modelli probabilistici che completano sequenze di token. Rilevare e gestire le allucinazioni è uno dei problemi aperti più importanti dell'IA applicata. (Cap. 19, 23)

ANI — Artificial Narrow Intelligence Tutta l'IA che esiste oggi, senza eccezioni. Un sistema ANI eccelle in un task specifico ma non sa fare nulla al di fuori di esso. GPT-4 scrive codice eccellente ma non può guidare un'auto. Tutto ciò che costruirai in questa guida è ANI. (Cap. 1)

AGI — Artificial General Intelligence Sistema teorico capace di svolgere qualsiasi task cognitivo umano con flessibilità tra domini diversi. Non esiste ancora. (Cap. 1)

Attenzione (Attention) Meccanismo introdotto nel Transformer che permette a ogni token di "guardare" tutti gli altri contemporaneamente e assegnare loro pesi in base alla rilevanza. "Il gatto che mangiava" — il verbo "mangiava" assegna peso alto a "gatto" (il soggetto) e basso al resto. È il motore di tutti gli LLM moderni. (Cap. 11)

AUC-ROC Area Under the Receiver Operating Characteristic curve. Misura la qualità del ranking di un classificatore: quanto bene separa i positivi dai negativi indipendentemente dalla soglia di decisione. Vale tra 0.5 (casuale) e 1.0 (perfetto). (Cap. 6)

AUC-PR Area Under the Precision-Recall curve. Più onesta di AUC-ROC su dataset sbilanciati perché misura la performance direttamente sulla classe minoritaria. Su un dataset con 0.1% di frodi, usa AUC-PR invece di AUC-ROC. (Progetto Vol. 1)

Autograd Sistema di PyTorch che tiene traccia automaticamente di ogni operazione su un tensor e calcola i gradienti in automatico quando chiami `.backward()`. Non devi implementare la backpropagation a mano: PyTorch lo fa per te. (Cap. 8)

B

Backpropagation Algoritmo che calcola i gradienti di tutti i parametri di una rete neurale rispetto alla loss, percorrendo il grafo computazionale al contrario (dall'output verso l'input). Usa la chain rule del calcolo differenziale. In PyTorch: `loss.backward()`. (Cap. 4, 5)

Batch Sottinsieme del dataset processato insieme prima di aggiornare i pesi. Un batch da 32 significa che il modello vede 32 esempi, calcola il gradiente medio su tutti e 32, poi aggiorna i pesi. Più grande il batch, più stabile il gradiente ma più lenta ogni iterazione. (Cap. 4, 7)

Batch Normalization Tecnica che normalizza le attivazioni di ogni layer (media≈0, varianza≈1) dopo ogni mini-batch. Stabilizza il training, permette learning rate più alti e riduce l'overfitting. Va applicata SOLO durante il training, non durante l'inferenza (usa `model.train()` e `model.eval()`). (Cap. 5)

Benchmark Test standard per confrontare modelli diversi su un task specifico. MMLU misura la conoscenza generale, HumanEval misura la qualità del codice, HellaSwag misura il senso comune. Attenzione alla data contamination: i modelli potrebbero aver visto le risposte durante il training. (Cap. 19)

Bias (del modello)

64. Il parametro bias `b` in $y = wx + b$: sposta la soglia di attivazione di un neurone. 2. La tendenza sistematica del modello a sbagliare in un certo modo — per esempio, performare peggio su certi gruppi demografici. I due significati sono completamente diversi: il primo è un parametro tecnico, il secondo è un problema etico. (Cap. 2, 5)

BPE — Byte Pair Encoding Algoritmo di tokenizzazione che divide il testo in sottoparole frequenti invece di parole intere. "intelligenza" diventa tre token: "intel" + "ligen" + "za". Permette di gestire qualsiasi testo con un vocabolario fisso. (Cap. 12)

C

Chain Rule Regola del calcolo differenziale che permette di derivare funzioni composte: se $z = f(g(x))$, allora $dz/dx = (df/dg) \cdot (dg/dx)$. È il motore matematico della backpropagation nelle reti profonde. (Cap. 3, 5)

Chunking Processo di divisione di documenti lunghi in blocchi più piccoli prima di indicizzarli in un vector database per il RAG. La dimensione ottimale è tipicamente 300–600 token con un overlap di 50–100 token tra chunk consecutivi. (Cap. 14, 24)

Classificazione Task in cui il modello assegna un input a una categoria discreta. Spam/non-spam, gatto/cane/uccello, sentimento positivo/negativo/neutro sono tutti task di classificazione. (Cap. 6)

CNN — Convolutional Neural Network Architettura di rete neurale specializzata per dati con struttura spaziale (immagini, audio spettrogrammato). Usa filtri convoluzionali che scorrono sull'input invece di connettere ogni pixel a ogni neurone, riducendo drasticamente il numero di parametri. (Cap. 9)

Context Window La quantità massima di token che un LLM può "tenere in memoria" durante una singola interazione. Claude e GPT-4 hanno finestre da 200K e 128K token rispettivamente. Superata questa lunghezza, il modello "dimentica" i token più vecchi. (Cap. 12)

Cross-Entropy Loss Funzione di perdita standard per la classificazione. Penalizza il modello proporzionalmente a quanto è sicuro della risposta sbagliata: un modello che dice "97% sicuro che sia sbagliato" riceve una penalità molto più alta di uno incerto. (Cap. 3, 4)

Cross-Validation Tecnica per stimare le performance di un modello dividendo il dataset in K parti (fold) e addestrando/valutando K volte, usando ogni volta una parte diversa come validation set. Più robusto di un singolo split. (Cap. 6)

D

Data Augmentation Generazione artificiale di nuovi esempi di training tramite trasformazioni degli esempi esistenti. Per immagini: flip, rotazione, ritaglio, variazione di colore. Riduce l'overfitting aumentando la varietà dei dati senza raccogliere nuovi dati reali. Va applicata SOLO sul training set. (Cap. 4, 9)

Data Leakage Errore critico in cui informazioni del test set "trapelano" nel training, invalidando la valutazione. Esempio: calcolare la media del dataset intero per normalizzare invece di usare solo la media del training set. Il modello risulta artificialmente bravo in valutazione ma fallisce in produzione. (Cap. 2)

Dataset Collezione di dati usata per addestrare, validare o testare un modello. Ogni dataset è tipicamente diviso in training set (impara), validation set (ottimizza iperparametri), test set (valuta una sola volta). (Cap. 2)

Dead Neuron Neurone con ReLU che produce sempre output zero perché la sua pre-attivazione è sempre negativa. Il gradiente è zero, quindi smette di imparare. Si evita con inizializzazione He, learning rate non troppo alto, o usando Leaky ReLU. (Cap. 5)

Deep Learning Sottinsieme del Machine Learning che usa reti neurali con molti strati ("deep" = profondo). "Deep" si riferisce alla profondità della rete, non alla profondità del pensiero. È la tecnologia dietro il riconoscimento di immagini, la traduzione automatica e gli LLM. (Cap. 1)

DPO — Direct Preference Optimization Tecnica alternativa a RLHF per allineare un LLM alle preferenze umane. Invece di addestrare un reward model separato, ottimizza direttamente il modello su coppie di preferenze (risposta A è meglio di risposta B). Più stabile di RLHF, diventata lo standard nel 2024–2026. (Cap. 12)

Dropout Tecnica di regolarizzazione che durante il training "spegne" casualmente una frazione dei neuroni a ogni forward pass. Il modello non può fare affidamento su nessun neurone specifico,

sviluppando rappresentazioni più robuste. Disattivato durante l'inferenza (`model.eval()`). (Cap. 4, 5)

DVC — Data Version Control Strumento per versionare dataset e modelli grandi che non possono essere committati su Git. Funziona come Git ma per file binari pesanti, salvandoli su storage remoto (S3, GCS, Azure) e tenendo nel repository solo un piccolo file `.dvc` di riferimento. (Cap. 20)

E

Early Stopping Tecnica di regolarizzazione che interrompe il training quando la validation loss smette di migliorare per N epoch consecutive (patience). Ripristina i pesi del punto migliore. Evita l'overfitting senza dover sapere in anticipo il numero esatto di epoch. (Cap. 4)

Embedding Vettore denso di numeri reali che rappresenta un dato (parola, frase, immagine) in modo che elementi semanticamente simili abbiano vettori vicini nello spazio. "gatto" e "felino" hanno embedding simili; "gatto" e "database" li hanno distanti. Fondamentale per il RAG e il meccanismo di attenzione. (Cap. 14)

Encoder / Decoder Nel Transformer originale: l'encoder processa l'input con attenzione bidirezionale; il decoder genera l'output con attenzione causale (non guarda i token futuri). I modelli LLM moderni (GPT, LLaMA, Claude) usano solo il decoder. (Cap. 11)

Epoch Un passaggio completo su tutto il dataset durante il training. Se il dataset ha 10.000 esempi e il batch size è 32, un'epoch consiste di ~313 aggiornamenti dei pesi. (Cap. 4)

F

F1 Score Media armonica di precision e recall: $F1 = 2 \cdot (P \cdot R) / (P + R)$. Bilancia le due metriche ed è utile quando entrambe sono importanti. Vale tra 0 (pessimo) e 1 (perfetto). (Cap. 6)

Feature Una caratteristica numerica di un campione usata come input al modello. La lunghezza di un'email, il numero di link, la presenza di certe parole sono tutte feature. La scelta e costruzione delle feature si chiama feature engineering. (Cap. 2)

Feature Importance Misura di quanto ogni feature ha contribuito alle predizioni di un modello. Random Forest e XGBoost la calcolano automaticamente in base a quanto ogni feature riduce l'impurità nei nodi degli alberi. Utile per capire cosa ha imparato il modello e per selezionare le feature più informative. (Cap. 6)

Fine-tuning Processo di riaddestrare un modello pre-addestrato su un dataset specifico per adattarlo a un task o dominio particolare. Permette di partire dalle capacità generali del modello base invece di addestrare da zero. LoRA e QLoRA sono tecniche di fine-tuning efficienti. (Cap. 18)

Forward Pass Il calcolo che va dall'input all'output in una rete neurale: i dati scorrono in avanti attraverso i layer, ognuno applicando la sua trasformazione. Produce la predizione del modello. Seguito dal backward pass (backpropagation) durante il training. (Cap. 4, 5)

Function Calling (Tool Use) Meccanismo che permette a un LLM di richiedere l'esecuzione di una funzione esterna (strumento) con parametri specifici invece di rispondere con testo libero.

Permette agli agenti di interagire con il mondo reale: cercate informazioni, esegui codice, chiama API. (Cap. 15)

G

GELU — Gaussian Error Linear Unit Funzione di attivazione usata nei Transformer moderni. Più morbida di ReLU: non va esattamente a zero per valori negativi piccoli, ma si avvicina. Empiricamente produce risultati leggermente migliori di ReLU nelle architetture basate su attenzione. (Cap. 5, 11)

Gradient Descent Algoritmo di ottimizzazione che aggiorna i pesi del modello nella direzione opposta al gradiente della loss. "Opposta" perché il gradiente punta verso la salita della loss, e noi vogliamo scendere. La dimensione del passo è controllata dal learning rate. (Cap. 3, 4)

Gradiente Vettore delle derivate parziali di una funzione rispetto a tutte le sue variabili. Nella backpropagation, il gradiente della loss rispetto a ogni peso dice: "se aumenti questo peso di un po', la loss aumenta o diminuisce? Di quanto?". Punta sempre nella direzione di salita più ripida. (Cap. 3)

Gradient Clipping Tecnica che limita la norma del gradiente a un valore massimo prima dell'aggiornamento dei pesi. Previene l'exploding gradient (gradiente che esplode a valori enormi) tipico delle RNN e reti molto profonde. `torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)`. (Cap. 7, 10)

GRU — Gated Recurrent Unit Versione semplificata dell'LSTM con meno parametri. Combina forget gate e input gate in un unico update gate, eliminando il cell state separato. Spesso raggiunge performance comparabili all'LSTM con training più veloce. (Cap. 10)

Guardrail Componente di sicurezza che si interpone tra l'input/output e il modello, applicando policy di sicurezza configurabili. Può essere basato su regex (veloce, rigido) o su un LLM classificatore (flessibile, costoso). NeMo Guardrails e LlamaFirewall sono framework open source. (Cap. 22)

H

Hidden State Il vettore di "memoria" di una RNN o LSTM che viene passato da un passo temporale al successivo. Accumula informazioni sul contesto passato. Nell'LSTM ci sono due hidden state: h (output del passo) e c (cell state, memoria a lungo termine). (Cap. 10)

HuggingFace Piattaforma e libreria open source che ospita migliaia di modelli pre-addestrati, dataset e spazi interattivi. La libreria `transformers` è lo standard de facto per caricare e usare LLM in Python. (App. B)

Human-in-the-loop Pattern architetturale che richiede approvazione umana prima di eseguire azioni irreversibili o ad alto impatto. Un agente che sta per inviare un'email o eliminare file deve chiedere conferma all'utente. Fondamentale per sistemi agentici sicuri. (Cap. 22)

HyDE — Hypothetical Document Embedding Tecnica RAG avanzata: invece di embeddare la query dell'utente direttamente, si genera prima una risposta ipotetica alla query, poi si usa quella

per il retrieval. Le risposte ipotetiche sono semanticamente più simili ai documenti rilevanti rispetto alle domande grezze. (Cap. 14)

Hyperparameter (Iperparametro) Parametro che tu scegli prima del training e che non viene imparato automaticamente dal modello. Learning rate, batch size, numero di layer, dropout rate sono tutti iperparametri. I parametri del modello (pesi e bias) invece si imparano automaticamente. (Cap. 4)

I

Iperparametro → vedi *Hyperparameter*

Inferenza Usare un modello già addestrato per fare predizioni su nuovi dati. Contrapposto al training: durante l'inferenza i pesi non cambiano, il dropout è disattivato, e il calcolo dei gradienti è disabilitato (`torch.no_grad()`). (Cap. 1)

K

KL Divergence Kullback-Leibler Divergence: misura quanto una distribuzione di probabilità si discosta da un'altra. Non è simmetrica: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. Usata nell'RLHF per penalizzare il modello se si allontana troppo dal modello originale durante il fine-tuning. (Cap. 3)

K-Means Algoritmo di clustering non supervisionato che raggruppa i dati in K cluster scegliendo iterativamente i centroidi che minimizzano la distanza media intra-cluster. Richiede di specificare K in anticipo; l'elbow method aiuta a sceglierlo. (Cap. 6)

L

Label (Etichetta) La risposta corretta per un esempio nel supervised learning. In un dataset di email spam, la label è "spam" o "non spam". Il modello impara a predire la label a partire dalle feature. (Cap. 2)

Latency (Latenza) Tempo che intercorre tra l'invio di una richiesta e la ricezione della risposta. In produzione si misura al 50° percentile (P50), 95° (P95) e 99° (P99). Il P95 rappresenta il caso più comune di utente "sfortunato" e è la metrica più utile per la user experience. (Cap. 20)

Layer (Strato) Gruppo di neuroni in una rete neurale che riceve input dal layer precedente e passa output al layer successivo. I layer nascosti trasformano progressivamente le feature; il layer di output produce la predizione finale. (Cap. 5)

Learning Rate Quanto grandi sono i passi dell'aggiornamento dei pesi durante il gradient descent. Troppo alto: il modello salta sopra il minimo e la loss oscilla o esplode. Troppo basso: il training converge ma richiede molto più tempo. È l'iperparametro più critico da ottimizzare. (Cap. 4, 7)

LLM — Large Language Model Modello di deep learning (basato su Transformer) addestrato su enormi quantità di testo con l'obiettivo di predire il token successivo. Da questo task emergono capacità di linguaggio, ragionamento e conoscenza generale. GPT-4, Claude, LLaMA sono tutti LLM. (Cap. 12)

LLM-as-Judge Tecnica di valutazione in cui si usa un LLM potente (es. Claude) per valutare la qualità delle risposte di un altro modello. Scalabile e spesso allineato con il giudizio umano, ma soggetto a bias come la preferenza per risposte verbose. (Cap. 19)

LoRA — Low-Rank Adaptation Tecnica di fine-tuning efficiente che aggiunge due matrici piccole (A e B) ai layer del modello invece di aggiornare tutti i parametri. Riduce i parametri addestrabili del 99% mantenendo il 90% dei risultati del full fine-tuning. Gli adapter LoRA sono file leggeri (<300 MB) separati dal modello base. (Cap. 18)

Loss Function (Funzione di Perdita) Funzione che misura quanto è sbagliata la predizione del modello. Durante il training si cerca di minimizzarla. MSE per la regressione, Cross-Entropy per la classificazione sono le più comuni. (Cap. 4)

LSTM — Long Short-Term Memory Architettura di rete ricorrente che risolve il vanishing gradient delle RNN semplici introducendo un cell state e tre porte (forget, input, output) che controllano il flusso di informazioni. Capace di memorizzare dipendenze a lungo raggio nelle sequenze. (Cap. 10)

M

MCP — Model Context Protocol Protocollo open standard proposto da Anthropic (2024) per connettere LLM a strumenti esterni in modo standardizzato. Un server MCP espone Tool (azioni), Resources (dati) e Prompts (workflow). Claude Desktop lo supporta nativamente. (Cap. 16)

MLflow Strumento per il tracking degli esperimenti di machine learning: registra parametri, metriche, modelli e artifact di ogni training run. Permette di confrontare esperimenti diversi e di riprodurre i risultati. (Cap. 20)

MLP — Multi-Layer Perceptron Rete neurale con strati di neuroni completamente connessi (ogni neurone connesso a tutti quelli del layer successivo). La forma più semplice di rete neurale profonda. Ottima per dati tabellari, meno efficiente di CNN per immagini o Transformer per testo. (Cap. 5)

Model Drift Degradazione delle performance di un modello in produzione nel tempo, causata dal cambiamento della distribuzione dei dati reali rispetto a quella del training set. Può essere silenzioso: il modello non crasha, risponde semplicemente sempre peggio. Richiede monitoring continuo. (Cap. 20)

Modello Il "programma" che ha imparato dai dati. In pratica: un file (.pt, .bin, .safetensors) che contiene miliardi di numeri (i pesi) e l'architettura per usarli. Non capisce nulla — ma produce output sorprendentemente utili. (Cap. 1)

Momentum Tecnica che aggiunge inerzia al gradient descent: invece di seguire solo il gradiente corrente, accumula una "velocità" media dei gradienti passati. Riduce le oscillazioni e accelera la convergenza in direzioni coerenti. (Cap. 7)

MSE — Mean Squared Error Errore quadratico medio: media dei quadrati delle differenze tra predizioni e valori reali. Loss standard per la regressione. Penalizza gli errori grandi più di quelli piccoli (per via del quadrato). La radice quadrata (RMSE) ha la stessa unità di misura dei dati originali. (Cap. 4)

Multi-Head Attention Versione del meccanismo di attenzione che esegue h teste in parallelo, ognuna con le proprie matrici Q, K, V. Ogni testa impara a prestare attenzione a relazioni diverse (soggetto-verbo, pronomi-antecedente, ecc.). Gli output vengono concatenati e riproiettati. (Cap. 11)

N

nn.Module La classe base di PyTorch da cui eredita ogni rete neurale. Richiede di definire `__init__` (dove dichiarare i layer) e `forward` (dove descrivere il flusso dei dati). Gestisce automaticamente i parametri, il salvataggio e il caricamento del modello. (Cap. 8)

O

One-Hot Encoding Conversione di variabili categoriche in vettori binari: ogni categoria diventa una colonna con 1 dove la categoria è presente e 0 altrove. "Milano", "Roma", "Napoli" diventano [1,0,0], [0,1,0], [0,0,1]. Necessario perché i modelli lavorano solo con numeri. (Cap. 2)

OWASP Top 10 per LLM Lista delle 10 vulnerabilità più critiche per sistemi basati su LLM, pubblicata dall'OWASP. Include prompt injection, data poisoning, excessive agency e altre. Aggiornata periodicamente; la versione 2025 è il riferimento corrente. (Cap. 21)

Overfitting Quando il modello memorizza il training set invece di generalizzare. Sintomi: loss di training molto bassa, loss di validation alta e crescente. Analogia: uno studente che memorizza le risposte degli esercizi del libro invece di capire i concetti. (Cap. 4)

P

Padding

65. In CNN: pixel artificiali aggiunti ai bordi dell'immagine per mantenere le dimensioni dopo la convoluzione. 2. In NLP: token speciali `[PAD]` aggiunti alle sequenze più corte per portarle alla stessa lunghezza in un batch. (Cap. 9)

Parametri (Pesi) I numeri dentro il modello che cambiano durante il training. GPT-4 ha ~1.8 trilioni di parametri. Più parametri = più capacità di modellare relazioni complesse, ma più dati e compute necessari. (Cap. 1)

Perceptron Il modello di neurone artificiale più semplice: somma pesata degli input + bias, poi funzione a gradino come attivazione. Converge sui dati linearmente separabili ma non può risolvere XOR. Il punto di partenza storico del deep learning moderno. (Cap. 5)

Pooling Operazione che riduce le dimensioni spaziali di una feature map prendendo il massimo (MaxPooling) o la media (AvgPooling) in finestre locali. Riduce i parametri, introduce invarianza alla posizione e allarga il campo recettivo nei layer successivi. (Cap. 9)

Pre-training La fase iniziale e più costosa di addestramento di un LLM: il modello viene addestrato su trilioni di token con l'obiettivo di predire il token successivo. Da questo emergono capacità

generali di linguaggio e conoscenza. Il pre-training costa milioni di dollari per i modelli grandi. (Cap. 12)

Precision Di tutti i campioni che il modello ha classificato come positivi, quanti lo erano davvero? Alta precision = pochi falsi allarmi. Formula: $TP / (TP + FP)$. (Cap. 6)

Prompt Engineering Disciplina di progettare istruzioni per LLM che producano output utili in modo affidabile. Include zero-shot, few-shot, chain-of-thought e altre tecniche. Non è "scrivere istruzioni creative" — è un ciclo iterativo di progettazione, test e misurazione. (Cap. 13)

Prompt Injection Attacco in cui input malevoli cercano di sovrascrivere le istruzioni del system prompt. Diretta: l'utente include le istruzioni nell'input. Indiretta: le istruzioni sono nascoste in documenti recuperati dal RAG. Una delle vulnerabilità più critiche dei sistemi LLM. (Cap. 21, 22)

Q

QLoRA Combinazione di quantizzazione 4-bit del modello base con addestramento LoRA degli adapter. Permette il fine-tuning di modelli da 7B su GPU consumer da 16 GB. Qualità leggermente inferiore a LoRA FP16 ma molto più accessibile. (Cap. 18)

Quantizzazione Riduzione della precisione numerica dei pesi del modello: da FP32 (32 bit) a BF16 (16 bit) o INT8/INT4. Riduce l'uso di memoria e aumenta la velocità di inferenza con perdita di qualità minima se fatta correttamente. (Cap. 17)

R

RAG — Retrieval-Augmented Generation Tecnica che combina recupero di documenti rilevanti da una knowledge base con generazione di testo da un LLM. Risolve due limiti degli LLM: conoscenza obsoleta e assenza di dati privati. Non richiede di riaddestrare il modello. (Cap. 14)

Random Forest Ensemble di alberi decisionali addestrati su sottoinsiemi casuali dei dati e delle feature. La predizione finale è il voto di maggioranza. Robusto, richiede poco tuning, fornisce feature importance. Ancora SOTA su molti task tabellari. (Cap. 6)

Recall Di tutti i campioni effettivamente positivi, quanti ne ha catturati il modello? Alta recall = pochi falsi negativi. Formula: $TP / (TP + FN)$. In un sistema antifrode, alta recall significa poche frodi non rilevate. (Cap. 6)

Regressione Task in cui il modello predice un valore continuo (prezzo, temperatura, stipendio). Contrapposto alla classificazione che predice categorie discrete. La loss standard per la regressione è MSE. (Cap. 6)

ReLU — Rectified Linear Unit Funzione di attivazione: $ReLU(x) = \max(0, x)$. Semplice, veloce, non satura per $x > 0$. Il default per i layer nascosti di CNN e MLP. Problema del dead neuron per $x < 0$ cronico. (Cap. 5)

ResNet Architettura CNN con skip connections che permettono di addestrare reti molto profonde (50, 101, 152 layer) senza vanishing gradient. Le skip connections danno al gradiente una "scorciatoia" diretta verso i layer iniziali. Ancora ampiamente usata come backbone. (Cap. 9)

RLHF — Reinforcement Learning from Human Feedback Tecnica per allineare un LLM alle preferenze umane. Valutatori umani confrontano coppie di risposte; le loro preferenze addestrano un reward model; il LLM viene ottimizzato per massimizzare il reward. Usato in ChatGPT, Claude e altri assistenti. DPO è un'alternativa più stabile. (Cap. 12)

RMSNorm Versione semplificata della Layer Normalization usata nei LLM moderni (LLaMA, Mistral). Normalizza dividendo per la radice quadrata della media dei quadrati, senza calcolare media e varianza. Più veloce di LayerNorm. (Cap. 12)

RNN — Recurrent Neural Network Rete neurale con uno stato interno (hidden state) che scorre da un passo temporale al successivo, permettendo di modellare sequenze. Soffre di vanishing gradient su sequenze lunghe. Oggi largamente sostituita da Transformer, ma ancora utile per streaming e sequenze molto lunghe. (Cap. 10)

RoPE — Rotary Positional Embedding Tecnica di encoding della posizione usata nei LLM moderni (LLaMA, Mistral). Invece di sommare un vettore di posizione fisso, ruota i vettori Q e K nell'operazione di attenzione. Generalizza meglio a sequenze più lunghe di quelle viste durante il training. (Cap. 12)

S

Self-Attention Meccanismo in cui ogni token in una sequenza calcola i suoi pesi di attenzione rispetto a tutti gli altri token nella stessa sequenza. Permette di catturare dipendenze a qualsiasi distanza senza il problema del vanishing gradient delle RNN. (Cap. 11)

SFT — Supervised Fine-Tuning Prima fase di allineamento di un LLM dopo il pre-training: il modello viene addestrato su esempi curati di (domanda, risposta ideale) per imparare il formato e lo stile di una conversazione utile. (Cap. 12)

SHAP Tecnica per spiegare le predizioni di qualsiasi modello ML calcolando il contributo di ogni feature alla predizione. Basata sulla teoria dei giochi (valori di Shapley). Utile per audit di modelli e per identificare feature problematiche. (Progetto Vol. 1)

Sigmoid Funzione di attivazione che schiaccia qualsiasi valore nell'intervallo (0, 1). Usata per output di classificazione binaria (interpretato come probabilità). Soffre di saturazione per valori estremi → vanishing gradient. Non usare nei layer nascosti. (Cap. 5)

Skip Connection Connessione diretta che bypassa uno o più layer, sommando l'input all'output del blocco. Introdotta da ResNet, permette al gradiente di fluire direttamente verso i layer iniziali senza attraversare tutti i layer intermedi. (Cap. 9)

SMOTE — Synthetic Minority Oversampling Technique Tecnica per bilanciare dataset sbilanciati generando nuovi esempi sintetici della classe minoritaria interpolando tra esempi reali. Va applicata SOLO sul training set per evitare data leakage. (Progetto Vol. 1, Cap. 6)

Softmax Funzione che converte un vettore di logit (numeri grezzi) in una distribuzione di probabilità che somma a 1. L'output di ogni classificatore multi-classe passa attraverso softmax prima di interpretarlo come probabilità. (Cap. 3, 5)

SVM — Support Vector Machine Algoritmo che trova l'iperpiano con massimo margine tra le classi. Il kernel trick permette di separare classi non linearmente separabili proiettandole

implicitamente in spazi ad alta dimensionalità. Ottimo su dataset piccoli con alta dimensionalità. (Cap. 6)

T

Temperature Parametro che controlla la "casualità" delle predizioni di un LLM dividendo i logit prima del softmax. Temperature bassa (0.1) → predizioni quasi deterministiche. Temperature alta (1.5) → output più creativo e vario. Per task fattuali usa temperature bassa; per brainstorming usa alta. (Cap. 12)

Tensor L'unità fondamentale di PyTorch. Concettualmente identico a un array NumPy (scalare, vettore, matrice, o array N-dimensionale), ma con due superpoteri: può girare su GPU e traccia le operazioni per il calcolo automatico dei gradienti. (Cap. 8)

Token L'unità minima in cui il testo viene spezzato prima di essere elaborato da un LLM. Non corrisponde necessariamente a una parola: può essere una sottoparola, un carattere, o un simbolo. "intelligenza" → 3 token. Il contesto di un LLM si misura in token. (Cap. 2, 12)

Training Il processo con cui il modello impara dai dati. Consiste nel ripetere migliaia di volte il ciclo: predici → misura l'errore → calcola i gradienti → aggiorna i pesi. (Cap. 1, 4)

Transfer Learning Riusare un modello addestrato su un task generico (ImageNet, testo del web) per un task specifico diverso, modificando solo i layer finali. Permette di ottenere ottimi risultati anche con pochi dati, perché i layer iniziali hanno già imparato feature generali utili. (Cap. 9, 18)

Transformer Architettura di rete neurale basata esclusivamente sul meccanismo di attenzione, senza ricorrenza. Introdotta nel paper "Attention Is All You Need" (2017). Permette elaborazione parallela di tutti i token, attenzione globale senza vanishing gradient, e scalabilità eccellente. La base di quasi tutti gli LLM moderni. (Cap. 11)

U

Underfitting Quando il modello è troppo semplice per catturare i pattern nei dati. Sintomi: loss alta sia sul training che sul validation set. Cause: modello troppo piccolo, training troppo breve, learning rate troppo basso. (Cap. 4)

V

Vanishing Gradient Problema nelle reti profonde in cui il gradiente diventa così piccolo durante la backpropagation (moltiplicando tante volte per valori < 1) che i layer iniziali smettono di imparare. Risolto da ReLU (per reti profonde), LSTM (per sequenze) e skip connections (per reti molto profonde). (Cap. 5, 10)

Vector Database Database ottimizzato per memorizzare vettori di embedding e cercare i più simili a una query in modo efficiente. ChromaDB (locale, per prototipi), Pinecone (cloud), pgvector (PostgreSQL) e FAISS sono le opzioni principali. Fondamentale per il RAG. (Cap. 14)

vLLM Framework per servire LLM in produzione ad alta throughput. Usa continuous batching (raggruppa dinamicamente le richieste) e PagedAttention (gestisce la KV-cache come memoria virtuale). Espone un'API compatibile con OpenAI. (Cap. 20)

W

Weight Decay Regularizzazione L2 che aggiunge alla loss un termine proporzionale alla somma dei quadrati dei pesi. Spinge i pesi verso zero, riducendo l'overfitting. In AdamW è implementato in modo corretto, disaccoppiato dall'adattamento del learning rate. (Cap. 4, 7)

X

XGBoost Implementazione ottimizzata del Gradient Boosting con regularizzazione L1/L2, gestione nativa dei NaN e parallelizzazione efficiente. SOTA su dati tabellari nelle competizioni Kaggle. Usa early stopping per trovare automaticamente il numero ottimale di alberi. (Cap. 6)

Z

Zero-shot / Few-shot Tecniche di prompting per LLM. Zero-shot: solo l'istruzione, nessun esempio. Few-shot: istruzione più 2–5 esempi input→output nel prompt. Il few-shot migliora la consistenza del formato ma non la comprensione del task. (Cap. 13)

Ultima revisione: Edizione 2026

Appendice B

Cheatsheet Python per l'IA

NumPy · pandas · scikit-learn · PyTorch · HuggingFace · Anthropic API

Questa appendice è un riferimento rapido. Non spiega i concetti — li trovi nei capitoli. Qui trovi la sintassi corretta quando hai dimenticato come si scrive qualcosa.

NumPy — Calcolo Numerico

```
import numpy as np

# --- Creazione array ---
a = np.array([1, 2, 3])          # da lista Python
b = np.zeros((3, 4))             # matrice 3x4 di zeri
c = np.ones((2, 3))             # matrice 2x3 di uno
d = np.random.randn(5, 10)      # valori casuali (normale std)
e = np.random.rand(5, 10)       # valori casuali (uniforme 0-1)
f = np.arange(0, 10, 2)         # [0, 2, 4, 6, 8]
g = np.linspace(0, 1, 100)      # 100 valori equidistanti tra 0 e 1
h = np.eye(4)                   # matrice identità 4x4

# --- Shape e reshape ---
a.shape                          # (3,)
b.reshape(2, 6)                 # stessi dati, nuova forma
b.flatten()                     # appiattisce in 1D
np.expand_dims(a, axis=0)        # (3,) → (1, 3)
a[:, np.newaxis]                # (3,) → (3, 1)

# --- Operazioni ---
a + b; a * b; a ** 2             # elemento per elemento
a @ b                           # prodotto matriciale (dot product)
np.dot(a, b)                    # equivalente a @
a.sum(); a.mean(); a.std(); a.max(); a.min() # riduzioni
a.sum(axis=0)                   # somma per colonna
a.sum(axis=1)                   # somma per riga

# --- Indicizzazione ---
a[2]                            # elemento singolo
a[1:4]                          # slice
a[[0, 2, 4]]                    # fancy indexing
a[a > 0]                        # boolean indexing
a[a > 0] = 0                     # assegnazione condizionale

# --- Funzioni utili ---
np.argmax(a)                     # indice del massimo
np.argsort(a)                    # indici ordinati
np.unique(a)                     # valori unici
np.concatenate([a, b], axis=0)  # concatena lungo asse
```

```

np.stack([a, b], axis=0)          # crea nuova dimensione
np.linalg.norm(a)                 # norma L2
np.linalg.norm(a, axis=1)         # norma L2 per ogni riga
np.clip(a, 0, 1)                  # tronca a [0, 1]
np.where(a > 0, a, 0)              # equivalente a ReLU

# --- Seed per riproducibilità ---
np.random.seed(42)

```

pandas — Manipolazione Dati

```

import pandas as pd
import numpy as np

# --- Caricamento ---
df = pd.read_csv("file.csv")
df = pd.read_csv("file.csv", sep=";", encoding="utf-8")
df = pd.read_json("file.json")
df = pd.read_excel("file.xlsx", sheet_name="Foglio1")

# --- Ispezione ---
df.shape          # (righe, colonne)
df.dtypes         # tipi di colonna
df.head(5)        # prime 5 righe
df.tail(5)        # ultime 5 righe
df.describe()     # statistiche descrittive (numeriche)
df.info()         # dtypes + valori non-null
df.isnull().sum() # NaN per colonna
df.duplicated().sum() # righe duplicate
df["col"].value_counts() # frequenza di ogni valore
df["col"].nunique() # numero di valori unici

# --- Selezione ---
df["colonna"]      # serie
df[["col1", "col2"]] # DataFrame con più colonne
df.loc[0]          # riga per label/indice
df.iloc[0]         # riga per posizione
df.loc[df["col"] > 5] # filtro per condizione
df.query("col > 5 and col2 == 'A'") # filtro con stringa

# --- Modifica ---
df["nuova"] = df["a"] + df["b"] # nuova colonna
df.drop(columns=["col"])        # elimina colonna
df.rename(columns={"old": "new"}) # rinomina
df.drop_duplicates()            # rimuovi duplicati
df.reset_index(drop=True)       # resetta l'indice

# --- Valori mancanti ---
df.dropna()          # elimina righe con NaN
df.dropna(subset=["col"]) # elimina solo se NaN in "col"
df.fillna(0)         # sostituisce NaN con 0
df["col"].fillna(df["col"].median()) # sostituisce con mediana

# --- Aggregazione ---
df.groupby("col")["val"].mean() # media per gruppo

```



```

df.groupby("col").agg({"val": ["mean", "std", "count"]})
df.pivot_table(values="val", index="a", columns="b", aggfunc="mean")

# --- Merge e join ---
pd.merge(df1, df2, on="chiave", how="left") # left join
pd.concat([df1, df2], axis=0, ignore_index=True) # unisci righe

# --- Salvataggio ---
df.to_csv("output.csv", index=False)
df.to_json("output.jsonl", orient="records", lines=True)

# --- JSONL (per dataset LLM) ---
import json
# Scrivi
with open("dataset.jsonl", "w") as f:
    for row in df.itertuples():
        f.write(json.dumps(row._asdict(), ensure_ascii=False) + "\n")
# Leggi
with open("dataset.jsonl") as f:
    dati = [json.loads(riga) for riga in f]

```

scikit-learn — Machine Learning Classico

```

from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    roc_auc_score, average_precision_score, f1_score,
    mean_squared_error, r2_score
)

# --- Split ---
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- Preprocessing ---
scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr) # fit + transform
X_te_s = scaler.transform(X_te) # SOLO transform (mai fit sul test)

# --- Pipeline completa ---
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), colonne_numeriche),
    ("cat", OneHotEncoder(drop="first"), colonne_categoriche),
])
pipeline = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(n_estimators=100)),
])
pipeline.fit(X_tr, y_tr)
pipeline.predict(X_te)

```

```

# --- Modelli principali ---
from sklearn.linear_model import LogisticRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
import xgboost as xgb

# Classificatori
lr = LogisticRegression(C=1.0, class_weight="balanced", max_iter=1000)
rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1)
svm = SVC(kernel="rbf", probability=True)
xgbm = xgb.XGBClassifier(n_estimators=200, eval_metric="auc", verbosity=0)

# Regressori
ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)

# Clustering
km = KMeans(n_clusters=3, random_state=42, n_init=10)
km.fit(X); km.labels_; km.inertia_

# --- Cross-validation ---
scores = cross_val_score(
    modello, X, y,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring="roc_auc",
    n_jobs=-1
)
print(f"AUC: {scores.mean():.4f} ± {scores.std():.4f}")

# --- Metriche ---
print(classification_report(y_te, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_te, y_prob):.4f}")
print(f"AUC-PR: {average_precision_score(y_te, y_prob):.4f}")
cm = confusion_matrix(y_te, y_pred)

```

PyTorch — Deep Learning

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, TensorDataset

# --- Device setup ---
device = "cuda" if torch.cuda.is_available() else "cpu"

# --- Tensor ---
t = torch.tensor([1.0, 2.0, 3.0])
t = torch.zeros(3, 4)
t = torch.randn(3, 4)
t = t.to(device)
t.shape; t.dtype; t.device

```

```

# Conversioni
t.numpy()                # tensor → numpy (solo CPU)
torch.from_numpy(arr)    # numpy → tensor
t.item()                 # tensor scalare → Python float
t.detach()               # stacca dal grafo computazionale

# Operazioni
t @ t.T                  # moltiplicazione matriciale
t.sum(dim=0)             # somma per colonna
t.mean(dim=1)            # media per riga
t.argmax(dim=1)          # indice del massimo per riga
F.softmax(t, dim=-1)     # softmax sull'ultima dimensione
F.relu(t)                # ReLU

# --- Dataset personalizzato ---
class MioDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)
    def __len__(self): return len(self.X)
    def __getitem__(self, i): return self.X[i], self.y[i]

loader = DataLoader(MioDataset(X, y), batch_size=32, shuffle=True)

# --- Architettura ---
class MiaRete(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(10, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 2)
        )
    def forward(self, x):
        return self.net(x)

modello = MiaRete().to(device)

# Parametri
sum(p.numel() for p in modello.parameters())          # totale
sum(p.numel() for p in modello.parameters() if p.requires_grad) # addestrabili

# --- Ottimizzatori ---
optim_adam = torch.optim.Adam(modello.parameters(), lr=1e-3)
optim_adamw = torch.optim.AdamW(modello.parameters(), lr=1e-3, weight_decay=0.01)
optim_sgd = torch.optim.SGD(modello.parameters(), lr=0.01, momentum=0.9)

# --- Scheduler ---
sched = torch.optim.lr_scheduler.CosineAnnealingLR(optim_adamw, T_max=100)
sched = torch.optim.lr_scheduler.OneCycleLR(optim_adamw, max_lr=1e-3,
total_steps=1000)
sched = torch.optim.lr_scheduler.ReduceLROnPlateau(optim_adamw, patience=5)

# --- Loss functions ---
loss_ce = nn.CrossEntropyLoss()          # classificazione multiclasse

```

```

loss_bce = nn.BCELoss()                # classificazione binaria (dopo sigmoid)
loss_mse = nn.MSELoss()                 # regressione

# --- Training loop ---
def train_one_epoch(modello, loader, optimizer, loss_fn, device):
    modello.train()
    total_loss = 0
    for X_b, y_b in loader:
        X_b, y_b = X_b.to(device), y_b.to(device)
        optimizer.zero_grad()
        pred = modello(X_b)
        loss = loss_fn(pred, y_b)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(modello.parameters(), max_norm=1.0)
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(loader)

# --- Valutazione ---
def evaluate(modello, loader, device):
    modello.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X_b, y_b in loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            pred = modello(X_b)
            correct += (pred.argmax(1) == y_b).sum().item()
            total += len(y_b)
    return correct / total

# --- Salvataggio e caricamento ---
torch.save(modello.state_dict(), "modello.pth")

modello_caricato = MiaRete().to(device)
modello_caricato.load_state_dict(torch.load("modello.pth", map_location=device))
modello_caricato.eval()

# --- GPU monitoring ---
torch.cuda.memory_allocated() / 1e9    # VRAM usata (GB)
torch.cuda.memory_reserved() / 1e9     # VRAM riservata (GB)
torch.cuda.empty_cache()                # libera cache GPU

```

HuggingFace Transformers

```

from transformers import (
    AutoTokenizer, AutoModelForCausalLM, AutoModelForSequenceClassification,
    TrainingArguments, Trainer
)
from datasets import Dataset, load_dataset
from peft import LoraConfig, get_peft_model, TaskType, PeftModel
import torch

# --- Tokenizer ---
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3.1-8B-Instruct")

```

```

# Tokenizza un testo
tokens = tokenizer("Ciao mondo!", return_tensors="pt")
tokens["input_ids"].shape                # (1, N_token)
tokens["attention_mask"]                 # 1 = token reale, 0 = padding

# Chat template
messages = [
    {"role": "system", "content": "Sei un assistente utile."},
    {"role": "user", "content": "Come stai?"}
]
testo = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)

# Batch con padding
tokens = tokenizer(
    ["Testo uno", "Testo due più lungo"],
    padding=True, truncation=True, max_length=512, return_tensors="pt"
)

# --- Caricamento modello ---
modello = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto"                # distribuisce automaticamente su GPU disponibili
)

# Caricamento in 4-bit (QLoRA)
from transformers import BitsAndBytesConfig
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
modello_4bit = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3.1-8B-Instruct",
    quantization_config=bnb_config,
    device_map="auto"
)

# --- Generazione testo ---
inputs = tokenizer("Il machine learning è", return_tensors="pt").to(device)
with torch.no_grad():
    output = modello.generate(
        **inputs,
        max_new_tokens=100,
        temperature=0.7,
        do_sample=True,
        top_p=0.9,
        repetition_penalty=1.1
    )
testo = tokenizer.decode(output[0][inputs["input_ids"].shape[1]:],
skip_special_tokens=True)

# --- LoRA config ---
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],

```

```

    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)
modello_lora = get_peft_model(modello, lora_config)
modello_lora.print_trainable_parameters()

# --- Dataset HuggingFace ---
dataset = load_dataset("imdb") # dataset pubblico
dataset["train"][0] # primo esempio
dataset["train"].filter(lambda x: len(x["text"]) > 100) # filtra
dataset["train"].map(lambda x: {"tokens": tokenizer(x["text"])}) # tokenizza

# --- Salva e carica adapter LoRA ---
modello_lora.save_pretrained("./adapter_lora")
tokenizer.save_pretrained("./adapter_lora")

# Ricarica
modello_base = AutoModelForCausalLM.from_pretrained("...", load_in_4bit=True)
modello_ft = PeftModel.from_pretrained(modello_base, "./adapter_lora")

# Merge adapter nei pesi base
modello_merged = modello_lora.merge_and_unload()
modello_merged.save_pretrained("./modello_finale")

```

Anthropic API

```

import anthropic
import os

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# --- Chiamata base ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Spiegami cos'è il RAG."}]
)
testo = risposta.content[0].text
print(f"Tokens usati: {risposta.usage.input_tokens} input,
{risposta.usage.output_tokens} output")

# --- Con system prompt ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    system="Sei un esperto di machine learning. Rispondi in italiano.",
    messages=[{"role": "user", "content": "Cos'è l'overfitting?"}]
)

# --- Conversazione multi-turno ---
storia = []
def chat(messaggio: str) -> str:
    storia.append({"role": "user", "content": messaggio})
    r = client.messages.create(

```

```

        model="claude-sonnet-4-20250514",
        max_tokens=1024,
        system="Sei un assistente utile.",
        messages=storia
    )
    risposta_testo = r.content[0].text
    storia.append({"role": "assistant", "content": risposta_testo})
    return risposta_testo

# --- Tool use ---
tools = [{
    "name": "calcola",
    "description": "Esegue un calcolo matematico",
    "input_schema": {
        "type": "object",
        "properties": {
            "espressione": {"type": "string", "description": "Espressione da calcolare"}
        },
        "required": ["espressione"]
    }
}]

risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "Quanto fa 127 × 456?"}]
)

# Processa la risposta del tool
for blocco in risposta.content:
    if blocco.type == "tool_use":
        print(f"Tool: {blocco.name}, Input: {blocco.input}")

# --- Output strutturato (JSON garantito) ---
risposta = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=256,
    messages=[
        {"role": "user", "content": "Estrai nome e email da: 'Ciao, sono Marco Rossi, marco@test.it'"},
        {"role": "assistant", "content": "{}"} # prefill
    ]
)

import json
dati = json.loads("{} + risposta.content[0].text)

# --- Streaming ---
with client.messages.stream(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Scrivi una poesia sull'IA."}]
) as stream:
    for testo in stream.text_stream:
        print(testo, end="", flush=True)

# --- Modelli disponibili (2026) ---
# claude-opus-4-6 → massima capacità

```

```
# claude-sonnet-4-20250514 → bilanciamento qualità/velocità
# claude-haiku-4-5-20251001 → veloce ed economico (guardrail, classificatori)
```

Snippet Utili — Vari

```
# === METRICHE DI SIMILARITÀ ===
import numpy as np

def similarita_coseno(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-8))

# === CHUNKING SEMPLICE ===
def chunking(testo: str, dim: int = 500, overlap: int = 100) -> list[str]:
    parole = testo.split()
    chunks = []
    i = 0
    while i < len(parole):
        chunk = " ".join(parole[i:i + dim])
        chunks.append(chunk)
        i += dim - overlap
    return chunks

# === SOFTMAX ===
def softmax(logits: np.ndarray) -> np.ndarray:
    e = np.exp(logits - logits.max())
    return e / e.sum()

# === GRADIENT DESCENT MANUALE ===
def gradient_descent(f, grad_f, x0, lr=0.01, n_steps=100):
    x = x0
    for _ in range(n_steps):
        x = x - lr * grad_f(x)
    return x

# === SEED GLOBALE ===
import random, numpy as np, torch
def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
set_seed(42)

# === TIMER DI CONTESTO ===
import time
from contextlib import contextmanager

@contextmanager
def timer(nome: str = ""):
    t0 = time.time()
    yield
    print(f"{nome}: {time.time() - t0:.2f}s")

with timer("training"):
```



```

    modello.fit(X, y)

# === PROGRESS BAR ===
from tqdm import tqdm
for batch in tqdm(loader, desc="Training"):
    pass

# === SALVA E CARICA JSON/PICKLE ===
import json, pickle

# JSON (leggibile)
with open("config.json", "w") as f:
    json.dump({"lr": 1e-3, "epochs": 100}, f)
with open("config.json") as f:
    config = json.load(f)

# Pickle (qualsiasi oggetto Python)
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# === GPU MEMORY INFO ===
if torch.cuda.is_available():
    print(f"VRAM totale: {torch.cuda.get_device_properties(0).total_memory /
1e9:.1f} GB")
    print(f"VRAM usata: {torch.cuda.memory_allocated() / 1e9:.2f} GB")
    print(f"VRAM libera: {(torch.cuda.get_device_properties(0).total_memory -
torch.cuda.memory_allocated()) / 1e9:.2f} GB")

```

Versione librerie di riferimento (2026): *numpy*≥1.26, *pandas*≥2.1, *scikit-learn*≥1.4, *torch*≥2.3, *transformers*≥4.40, *anthropic*≥0.28, *peft*≥0.10

Appendice C

Risorse Aggiornate

Corsi · Paper fondamentali · Community · Strumenti

Ogni risorsa qui elencata è stata selezionata con un criterio preciso: accessibile, aggiornata, e con il miglior rapporto qualità/tempo. Non è un elenco esaustivo — è una mappa.

Corsi e Formazione

Per iniziare (se vuoi consolidare le basi)

fast.ai — Practical Deep Learning for Coders fast.ai/course Il corso più pratico esistente. Parte dal codice funzionante e spiega la teoria solo dopo. Ottimo complemento a questa guida per chi vuole più pratica immediata. Gratuito.

3Blue1Brown — Neural Networks (YouTube) youtube.com/3blue1brown Playlist "Neural Networks" — le visualizzazioni più belle della matematica dietro le reti neurali. Consigliato per capire intuitivamente backpropagation e attenzione. Gratuito.

Andrej Karpathy — Neural Networks: Zero to Hero github.com/karpathy/nn-zero-to-hero Serie di video in cui Karpathy costruisce un GPT da zero in Python puro. Denso, tecnico, eccellente. Il video "Let's build GPT" è un must-watch dopo aver completato il Volume 2. Gratuito.

Per approfondire (post-guida)

Stanford CS224N — Natural Language Processing with Deep Learning web.stanford.edu/class/cs224n Il corso universitario più completo su NLP con Transformer. Slide, video e assignments disponibili gratuitamente online.

DeepLearning.AI — Specialization di Andrew Ng deeplearning.ai La specializzazione in 5 corsi è ancora il percorso strutturato più completo per chi vuole fondamenta solide con certificazione. A pagamento (con audit gratuito).

Hugging Face Course huggingface.co/learn Corso ufficiale HuggingFace su Transformer, fine-tuning e deployment. Direttamente aggiornato dal team. Gratuito.

Paper Fondamentali

Questi sono i paper che ogni professionista del settore dovrebbe aver letto — almeno l'abstract e le figure. Ordinati cronologicamente.

Architetture

"Attention Is All You Need" — Vaswani et al., 2017 arxiv.org/abs/1706.03762 Il paper del Transformer. L'architettura che sta alla base di tutto. Leggere le sezioni 3 (Model Architecture) e 4 (Why Self-Attention) è sufficiente.

"Deep Residual Learning for Image Recognition" — He et al., 2015 arxiv.org/abs/1512.03385 Le skip connections di ResNet. Tre pagine che hanno rivoluzionato il training di reti profonde.

"BERT: Pre-training of Deep Bidirectional Transformers" — Devlin et al., 2018 arxiv.org/abs/1810.04805 Il primo modello a dimostrare il potere del pre-training bidirezionale su larga scala.

"Language Models are Few-Shot Learners" — Brown et al., 2020 arxiv.org/abs/2005.14165 Il paper di GPT-3. Introduce il concetto di few-shot learning emergente dalla scala.

Scaling e LLM

"Scaling Laws for Neural Language Models" — Kaplan et al., 2020 arxiv.org/abs/2001.08361 Le prime leggi di scala: come la qualità cresce in modo prevedibile con parametri, dati e compute.

"Training Compute-Optimal Large Language Models" (Chinchilla) — Hoffmann et al., 2022 arxiv.org/abs/2203.15556 La revisione delle leggi di scala: i modelli erano sottoaddestranti rispetto alla loro dimensione.

"Llama 2: Open Foundation and Fine-Tuned Chat Models" — Touvron et al., 2023 arxiv.org/abs/2307.09288 Architettura e training di LLaMA 2. Ottimo per capire come si costruisce un LLM open source moderno.

Allineamento e RLHF

"Training language models to follow instructions with human feedback" — Ouyang et al., 2022 arxiv.org/abs/2203.02155 Il paper di InstructGPT: come RLHF trasforma un base model in un assistente utile.

"Direct Preference Optimization: Your Language Model is Secretly a Reward Model" — Rafailov et al., 2023 arxiv.org/abs/2305.18290 DPO: come ottimizzare il modello direttamente sulle preferenze umane senza un reward model separato.

Fine-tuning Efficiente

"LoRA: Low-Rank Adaptation of Large Language Models" — Hu et al., 2021 arxiv.org/abs/2106.09685 Il paper che ha reso il fine-tuning accessibile. Algoritmo chiaro, esperimenti convincenti.

"QLoRA: Efficient Finetuning of Quantized LLMs" — Dettmers et al., 2023 arxiv.org/abs/2305.14314 Come fare fine-tuning su GPU consumer con quantizzazione 4-bit. Fondamentale per il Cap. 18.

RAG e Agenti

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — Lewis et al., 2020 arxiv.org/abs/2005.11401 Il paper originale del RAG. Introduce l'idea di combinare retrieval e generazione.

"ReAct: Synergizing Reasoning and Acting in Language Models" — Yao et al., 2022 arxiv.org/abs/2210.03629 Il pattern ReAct per gli agenti. Ragionamento esplicito interallacciato con azioni.

Sicurezza

"Universal and Transferable Adversarial Attacks on Aligned Language Models" — Zou et al., 2023 arxiv.org/abs/2307.15043 Dimostra che i jailbreak possono essere trovati automaticamente con ottimizzazione. Fondamentale per il Cap. 21.

"Prompt Injection Attacks Against GPT-3" — Perez & Ribeiro, 2022 arxiv.org/abs/2302.12173 La prima analisi sistematica degli attacchi di prompt injection.

Community e Forum

Hugging Face Forum discuss.huggingface.co Il forum più attivo per domande tecniche su Transformer, PEFT, fine-tuning. Risposto spesso dagli stessi autori delle librerie.

r/MachineLearning reddit.com/r/MachineLearning Discussioni su paper nuovi, risultati di ricerca, trend del settore. Segnale/rumore decente.

r/LocalLLaMA reddit.com/r/LocalLLaMA Community dedicata ai modelli open source locali. Benchmarks, guide pratiche, discussioni su hardware.

Discord di fast.ai discord.gg/fastai Community molto attiva, orientata ai praticanti più che ai teorici.

Twitter/X: account da seguire

- [@karpathy](https://twitter.com/karpathy) — ex OpenAI, guide tecniche eccellenti
 - [@ylecun](https://twitter.com/ylecun) — Chief AI Scientist di Meta, prospettiva accademica
 - [@DrJimFan](https://twitter.com/DrJimFan) — ricercatore NVIDIA, focus su agenti e robotica
 - [@NousResearch](https://twitter.com/NousResearch) — team open source su LLM e fine-tuning
-

Newsletter e Blog Tecnici

The Batch — DeepLearning.AI deeplearning.ai/the-batch Newsletter settimanale di Andrew Ng. Segnale alto, rumore basso. Gratuita.

Import AI — Jack Clark jack-clark.net Una delle newsletter più dense del settore. Per chi vuole seguire la frontiera della ricerca.

Lil'Log — Lilian Weng (OpenAI) lilianweng.github.io Post tecnici approfonditi su argomenti come RLHF, agenti, sicurezza. Qualità eccezionale.

Sebastian Raschka's Blog sebastianraschka.com/blog Guide pratiche su fine-tuning, LLM applicati, con molto codice. Ottimo per practitioners.

Eugene Yan eugeneyan.com Focus su ML in produzione: sistema di raccomandazione, valutazione LLM, MLOps.

Strumenti e Piattaforme

Ambienti di sviluppo

Google Colab colab.research.google.com GPU gratuita (T4), zero configurazione. Punto di partenza consigliato.

Kaggle Notebooks kaggle.com/code GPU gratuita, dataset integrati, competizioni. Ottimo per praticare con problemi reali.

Lightning.ai lightning.ai Cloud con GPU, storage persistente, e l'ecosistema PyTorch Lightning integrato. Buona alternativa a Colab per progetti più seri.

Modelli e API

Anthropic API docs.anthropic.com Claude Sonnet e Haiku. Eccellente documentazione, cookbook con esempi pratici.

OpenAI API platform.openai.com GPT-4o e fine-tuning via API. Ampio ecosistema di integrazioni.

HuggingFace Hub huggingface.co/models Il maggior repository di modelli open source. LLaMA, Mistral, Qwen, e migliaia di modelli fine-tuned.

Ollama ollama.com Il modo più semplice per far girare LLM open source in locale. Un comando per scaricare e avviare LLaMA, Mistral, Phi.

MLOps e Deployment

Weights & Biases wandb.ai Tracking di esperimenti, visualizzazione delle curve di training, confronto run. Gratuito per uso personale.

MLflow mlflow.org Open source, self-hosted. Alternativa a W&B senza dipendenze cloud.

Docker Hub hub.docker.com Repository di immagini Docker. Cerca [vllm](#), [chroma](#), [ollama](#) per trovare immagini pronte.

Sicurezza e Valutazione

LangSmith (LangChain) smith.langchain.com Tracing e valutazione per sistemi LLM. Utile per debugging di pipeline RAG e agenti.

OWASP LLM Top 10 owasp.org/www-project-top-10-for-large-language-model-applications Il riferimento per la sicurezza dei sistemi LLM. Aggiornato annualmente.

Garak github.com/leondz/garak Framework open source per il red-teaming automatizzato di LLM.

Datasets di Riferimento

| Dataset | Task | Dimensione | Dove trovarlo | |---|---|---|---| | MNIST | Classificazione immagini | 70K immagini | [torchvision.datasets.MNIST](https://pytorch.org/vision/datasets/MNIST) | | CIFAR-10 | Classificazione immagini | 60K immagini | [torchvision.datasets.CIFAR10](https://pytorch.org/vision/datasets/CIFAR10) | | ImageNet | Classificazione immagini | 1.4M immagini | image-net.org (modelli pre-addestrati su HuggingFace) | | IMDB | Sentiment analysis | 50K

recensioni | `datasets.load_dataset("imdb")` | | MMLU | Conoscenza generale LLM | 14K domande
| `datasets.load_dataset("cais/mmlu")` | | Alpaca | Instruction following | 52K esempi |
`datasets.load_dataset("tatsu-lab/alpaca")` | | OpenHermes | Fine-tuning conversazionale | 1M
esempi | HuggingFace Hub | | CC-News | News in italiano | variabile | HuggingFace Hub: `cc_news` |
| Titanic | Classificazione tabellare | 891 righe | `sklearn.datasets` / Kaggle | | Credit Card Fraud |
Rilevamento anomalie | 284K righe | Kaggle: `mlg-ulb/creditcardfraud` |

Risorse verificate: Edizione 2026. I link online possono cambiare — usa i nomi per cercare le versioni più recenti.

Appendice D

Roadmap di Studio Consigliata

Con milestone verificabili

Questa roadmap è pensata per uno sviluppatore che studia part-time — circa 10 ore a settimana. I tempi indicati sono stime medie: c'è chi li dimezza e chi li raddoppia, ed entrambi vanno bene. L'importante è non saltare le milestone: sono i checkpoint che confermano che hai davvero capito, non solo letto.

Come usare questa roadmap

Ogni fase ha:

- **Obiettivi di apprendimento:** cosa imparerai
- **Milestone verificabili:** esercizi concreti che dimostrano la comprensione
- **Tempo stimato:** per chi studia ~10 ore/settimana
- **Segnale di prontezza:** come sapere se sei pronto per la fase successiva

Non andare alla fase successiva finché non hai completato almeno il 70% delle milestone della fase corrente. È più efficace consolidare bene che procedere velocemente con lacune.

FASE 1 — Fondamenta del ML

Corrispondente al Volume 1, Capitoli 1–7 **Tempo stimato: 6–8 settimane**

Obiettivi

- Capire la differenza tra programmazione tradizionale e ML
- Saper preparare un dataset per il training (split, preprocessing, encoding)
- Saper addestrare e valutare un modello di ML classico
- Capire cosa succede matematicamente durante il training

Milestone verificabili

M1.1 — Setup e primo codice

- [] Google Colab configurato con GPU T4
- [] Esecuzione senza errori degli snippet del Cap. 1 (verifica installazione)
- [] Completato l'Esercizio 2 del Cap. 1 (classificazione spam con regole manuali)

M1.2 — Dati

- [] Esplorazione del dataset Titanic con EDA completa (Cap. 2, Esercizio 1)

- ☐ Pipeline di preprocessing senza data leakage (Cap. 2, Esercizio 2)
- ☐ Sai rispondere: perché `scaler.fit()` va fatto SOLO sul training set?

M1.3 — Matematica

- ☐ Implementato gradient descent manuale su $f(w) = (w-7)^2$ (Cap. 3, Esercizio 2)
- ☐ Sai spiegare cosa misura la cross-entropy loss con parole tue
- ☐ Completato l'Esercizio 3 del Cap. 3 (cross-entropy a mano)

M1.4 — Training loop

- ☐ Training loop da zero senza `nn.Module` (Cap. 4, Esercizio 1)
- ☐ Hai osservato l'overfitting empiricamente e applicato dropout (Cap. 4, Esercizio 2)
- ☐ Sai leggere una curva di training e diagnosticare: overfitting, underfitting, learning rate errato

M1.5 — Reti neurali

- ☐ Forward pass manuale su MLP 2-layer (Cap. 5, Esercizio 1)
- ☐ Backpropagation manuale su problema XOR (Cap. 5, Esercizio 2)
- ☐ Sai spiegare perché ReLU ha sostituito Sigmoid nei layer nascosti

M1.6 — ML classico

- ☐ Confronto sistematico di 5 modelli sullo stesso dataset (Cap. 6, Esercizio 1)
- ☐ XGBoost con hyperparameter tuning via RandomizedSearchCV (Cap. 6, Esercizio 2)
- ☐ Sai scegliere il modello giusto per un problema tabellare generico

M1.7 — Ottimizzatori

- ☐ Confronto empirico di 4 ottimizzatori (Cap. 7, Esercizio 1)
- ☐ Implementazione training loop completo con early stopping e scheduler (Cap. 7)

M1.8 — Progetto finale Vol. 1

- ☐ Pipeline ML completa sul dataset frodi: dati → preprocessing → confronto modelli → test set
- ☐ Report con AUC-PR, confusion matrix, analisi del trade-off precision/recall
- ☐ Sai spiegare perché AUC-PR è più onesta di AUC-ROC su dataset sbilanciati

Segnale di prontezza per la Fase 2

Sei pronto se puoi costruire autonomamente una pipeline ML completa su un nuovo dataset tabellare in meno di 3 ore, senza guardare il codice della guida.

FASE 2 — Deep Learning e LLM

Corrispondente al Volume 2, Capitoli 8–17 **Tempo stimato: 10–14 settimane**

Obiettivi

- Padroneggiare PyTorch per il deep learning
- Capire e saper usare CNN, RNN/LSTM e Transformer
- Costruire un chatbot con API LLM

- Costruire un sistema RAG funzionante
- Implementare un agente con tool use

Milestone verificabili

M2.1 — PyTorch

- ☐ Classificatore MNIST con CNN (Cap. 8–9, >98% accuracy)
- ☐ Transfer learning con ResNet su CIFAR-10 o STL-10 (Cap. 9, Esercizio)
- ☐ Sai muovere tensor tra CPU e GPU e spiegare perché i device devono coincidere

M2.2 — Sequenze

- ☐ LSTM per predizione di serie temporale (Cap. 10, Esercizio)
- ☐ Classificatore di sentiment con GRU bidirezionale (Cap. 10, Esercizio)
- ☐ Sai spiegare il vanishing gradient e perché LSTM lo mitiga

M2.3 — Transformer

- ☐ Mini-Transformer costruito da zero per classificazione testo (Cap. 11, >85% su IMDB)
- ☐ Implementazione della causal mask per generazione autoregressiva (Cap. 11, Esercizio)
- ☐ Sai spiegare Query, Key, Value con un'analogia del mondo reale

M2.4 — LLM e Prompt Engineering

- ☐ Chatbot con memoria persistente e system prompt personalizzato (Cap. 12)
- ☐ Confronto empirico zero-shot vs few-shot vs CoT sullo stesso task (Cap. 13, Esercizio)
- ☐ LLM-as-Judge funzionante su un tuo dataset di valutazione (Cap. 13)

M2.5 — RAG

- ☐ Sistema Q&A su almeno 2 PDF tuoi reali (Cap. 14, Esercizio)
- ☐ Confronto di due configurazioni di chunking con metriche quantitative (Cap. 14, Esercizio)
- ☐ Sai spiegare quando usare re-ranking vs hybrid search

M2.6 — Agenti

- ☐ Agente con ricerca web reale che risponde con fonti (Cap. 15, Esercizio)
- ☐ Agente con memoria persistente e limite di budget token (Cap. 15, Esercizio)
- ☐ Hai implementato almeno una protezione contro i loop infiniti

M2.7 — MCP

- ☐ Server MCP con almeno 4 tool, 1 resource, 1 prompt (Cap. 16, Esercizio)
- ☐ Server connesso a Claude Desktop e testato in conversazione reale
- ☐ Sai spiegare la differenza tra Tool, Resource e Prompt in MCP

M2.8 — Hardware

- ☐ Hai profilato un modello e misurato l'uso di VRAM (Cap. 17, Esercizio)
- ☐ Sai scegliere tra FP32, BF16, INT8, INT4 in base al caso d'uso

M2.9 — Progetto finale Vol. 2

- ☐ Chatbot RAG con agente integrato: risponde a domande, cerca sul web, usa un MCP server

- ☐ Sistema funzionante end-to-end con almeno 20 documenti nel knowledge base
- ☐ Hai testato con almeno 30 domande e documentato la qualità delle risposte

Segnale di prontezza per la Fase 3

Sei pronto se puoi costruire e deployare localmente un chatbot RAG con agente da zero in meno di una giornata lavorativa.

FASE 3 — Produzione e Sicurezza

Corrispondente al Volume 3, Capitoli 18–24 **Tempo stimato: 8–12 settimane**

Obiettivi

- Fine-tuning con LoRA/QLoRA su dati propri
- Sistema di valutazione automatizzata con LLM-as-Judge
- Deploy containerizzato con monitoring
- Comprensione degli attacchi e implementazione delle difese
- Progetto finale professionale completo

Milestone verificabili

M3.1 — Fine-tuning

- ☐ Fine-tuning LoRA di LLaMA 8B su almeno 200 esempi propri (Cap. 18, Esercizio)
- ☐ Confronto base vs fine-tuned con metriche quantitative
- ☐ Hai testato il catastrophic forgetting e documentato i risultati
- ☐ Sai spiegare la differenza tra usare fine-tuning per "comportamenti" vs "fatti"

M3.2 — Valutazione

- ☐ Eval set di almeno 60 esempi con 3 categorie (core, edge, trabocchetto)
- ☐ Pipeline LLM-as-Judge automatizzata con report JSON (Cap. 19, Esercizio)
- ☐ Test sistematici di hallucination su 20 domande trabocchetto
- ☐ Almeno un ciclo di miglioramento documentato con metriche prima/dopo

M3.3 — Deployment

- ☐ Chatbot RAG containerizzato con Docker Compose (Cap. 20, Esercizio)
- ☐ Dashboard Grafana con almeno 3 metriche (latenza, traffico, errori)
- ☐ Load test con Locust su almeno 10 utenti simultanei
- ☐ Sai interpretare le 4 golden signal e diagnosticare un problema di performance

M3.4 — Sicurezza Attacchi

- ☐ Report di red-teaming con almeno 7 test eseguiti (Cap. 21, Esercizio)
- ☐ Hai trovato e documentato almeno 2 vulnerabilità reali nel tuo sistema
- ☐ Sai spiegare la differenza tra injection diretta e indiretta con esempi concreti

M3.5 — Sicurezza Difese

- ☐ `InputValidator` implementato e integrato nell'endpoint (Cap. 22, Esercizio)

- [] System prompt testato contro 8 attacchi standard con punteggio di robustezza
- [] Rate limiting e security monitoring attivi in produzione
- [] Il sistema supera $\geq 75\%$ degli attacchi del red-team precedente

M3.6 — IA nel lavoro

- [] Workflow IA per un task lavorativo reale tuo (Cap. 23, Esercizio)
- [] Accuracy $\geq 85\%$ su dataset storico
- [] WorkflowLogger attivo con statistiche settimanali

M3.7 — Progetto finale

- [] Assistente verticale su un dominio reale — tutte le 6 fasi completate
- [] Pass rate $\geq 80\%$ sull'eval set
- [] Docker Compose funzionante con `docker compose up -d`
- [] Report di red-teaming con vulnerabilità trovate e risolte
- [] README che permette setup in meno di 5 comandi
- [] Tutti i punti della checklist di consegna del Cap. 24 verificati

Segnale di completamento

Hai completato la roadmap se riesci a descrivere a un collega sviluppatore, senza note, il ciclo completo: raccolta dati → modello → valutazione → deploy → sicurezza → iterazione.

Dopo la Guida — Dove andare

Se vuoi specializzarti in NLP/LLM

- Studia i paper fondamentali dell'Appendice C (in ordine cronologico)
- Partecipa a una competizione Kaggle NLP
- Contribuisci a un progetto open source su HuggingFace
- Approfondisci: Constitutional AI, interpretabilità dei modelli, watermarking

Se vuoi specializzarti in Computer Vision

- Stanford CS231n (corso online gratuito)
- Implementa da zero una delle architetture più recenti (EfficientNet, DINO, SAM)
- Studia la diffusion: Denoising Diffusion Probabilistic Models

Se vuoi specializzarti in MLOps/Production

- Approfondisci Kubernetes per il deployment distribuito
- Studia Apache Kafka per il data streaming
- Implementa un sistema di A/B testing per modelli ML
- Approfondisci: model distillation, quantizzazione avanzata, ONNX

Se vuoi specializzarti in Sicurezza IA

- Segui il blog di Anthropic sulla sicurezza
- Studia l'interpretabilità meccanicistica (Circuits work di Anthropic)
- Approfondisci: adversarial robustness, membership inference attacks, differential privacy

Se vuoi lavorare nell'industria

- Costruisci un portfolio di 3 progetti completi su GitHub con README chiari
- Contribuisci a HuggingFace, LangChain o simili con piccole pull request
- Scrivi post tecnici su ciò che hai imparato — insegnare consolida la comprensione
- Partecipa alle community dell'Appendice C

Tabella di Autovalutazione

Usala ogni mese per monitorare i tuoi progressi. "✓" = lo faccio autonomamente, "△" = lo faccio con aiuto, "X" = ancora da imparare.

Competenza	Autovalutazione	Data	--- --- ---	Preparare un dataset ML (split, encoding, normalizzazione)		Scegliere il modello giusto per un problema tabellare		Costruire e addestrare una CNN in PyTorch		Spiegare il meccanismo di attenzione		Costruire un sistema RAG funzionante		Costruire un agente con tool use		Fare fine-tuning LoRA su GPU consumer		Costruire un eval set e misurare la qualità con LLM-as-Judge		Deployare un servizio ML con Docker		Eseguire un red-team e documentare le vulnerabilità		Implementare input validation e guardrail		Costruire un workflow IA affidabile per un task reale	
------------	-----------------	------	-------------	--	--	---	--	---	--	--------------------------------------	--	--------------------------------------	--	----------------------------------	--	---------------------------------------	--	--	--	-------------------------------------	--	---	--	---	--	---	--

Roadmap aggiornata: Edizione 2026. I tempi indicati presuppongono ~10 ore/settimana di studio attivo (con codice, non solo lettura).

Appendice E

Soluzioni Commentate degli Esercizi Pratici

Volume 2 — Costruire

Ricorda: guarda la soluzione solo dopo aver tentato in autonomia. Il valore è nel processo, non nel risultato.

Capitolo 8 — Deep Learning con PyTorch

Esercizio finale — Fashion-MNIST

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"

# Fashion-MNIST ha normalizzazione diversa da MNIST
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,)) # mean, std specifici di Fashion-
MNIST
])

train_data = datasets.FashionMNIST("./data", train=True, download=True,
transform=transform)
test_data = datasets.FashionMNIST("./data", train=False, download=True,
transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)

classi = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
          'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Architettura base (3 layer)
class Classificatore(nn.Module):
    def __init__(self, n_hidden=512, dropout=0.2):
        super().__init__()
        self.rete = nn.Sequential(
            nn.Linear(784, n_hidden), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(n_hidden, 256), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(256, 10)
        )
    def forward(self, x):
        return self.rete(x.view(x.size(0), -1))
```

```

# Con quarto layer (punto 3)
class ClassificatoreDeep(nn.Module):
    def __init__(self, dropout=0.2):
        super().__init__()
        self.rete = nn.Sequential(
            nn.Linear(784, 512), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(512, 256), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(dropout), # quarto layer
aggiunto
            nn.Linear(128, 10)
        )
    def forward(self, x):
        return self.rete(x.view(x.size(0), -1))

def train_and_eval(modello, loader_tr, loader_te, n_epochs=10, lr=1e-3):
    opt = torch.optim.Adam(modello.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    modello = modello.to(device)
    acc_storia = []
    for epoch in range(n_epochs):
        modello.train()
        for X, y in loader_tr:
            X, y = X.to(device), y.to(device)
            opt.zero_grad()
            loss_fn(modello(X), y).backward()
            opt.step()
        modello.eval()
        corretti = 0
        with torch.no_grad():
            for X, y in loader_te:
                X, y = X.to(device), y.to(device)
                corretti += (modello(X).argmax(1) == y).sum().item()
        acc = corretti / len(loader_te.dataset)
        acc_storia.append(acc)
        print(f"Epoca {epoch+1:2d}: {acc*100:.2f}%")
    return acc_storia

# Risultati attesi:
# MLP base (3 layer, dropout 20%): ~87-88% dopo 10 epoche
# MLP deep (4 layer, dropout 20%): ~88-89% dopo 10 epoche (+0.5-1%)
# MLP base (dropout 40%): ~86-87% — leggermente peggio in accuratezza,
# ma curva di training più stabile

```

Risposta alla domanda 3 (quarto layer): Il miglioramento tipico è +0.5–1%. Non drammatico. Per Fashion-MNIST, il bottleneck non è la profondità ma la struttura dell'architettura — le CNN (Cap. 9) ottengono >92% sullo stesso dataset.

Risposta alla domanda 4 (dropout 40%): La loss di training scende più lentamente (il modello deve essere più robusto per sopravvivere allo spegnimento del 40%). La validation loss è più stabile e diverge meno dal training. Ma l'accuratezza finale può essere leggermente inferiore: il dropout del 40% è aggressivo per questo problema.

Risposta alla domanda 5 (classi confuse): Le coppie più difficili sono:

- **Shirt vs T-shirt/top:** visivamente molto simili
- **Coat vs Pullover:** forma simile
- **Sandal vs Sneaker:** entrambe scarpe

Questo ha senso: anche gli umani farebbero fatica con immagini 28×28 in scala di grigi di questi capi.

Capitolo 9 — Reti Convoluzionali

Esercizio finale — STL-10 con transfer learning

```
import torch, torch.nn as nn
import torchvision.models as models
from torchvision import transforms, datasets
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"

# STL-10 ha immagini 96×96 a colori, 500 esempi per classe (training)
transform_train = transforms.Compose([
    transforms.Resize(96),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(96, padding=8),
    transforms.ToTensor(),
    transforms.Normalize((0.4467, 0.4398, 0.4066), (0.2603, 0.2566, 0.2713))
])
transform_test = transforms.Compose([
    transforms.Resize(96),
    transforms.ToTensor(),
    transforms.Normalize((0.4467, 0.4398, 0.4066), (0.2603, 0.2566, 0.2713))
])

train_data = datasets.STL10("./data", split="train", download=True,
transform=transform_train)
test_data = datasets.STL10("./data", split="test", download=True,
transform=transform_test)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True, num_workers=2)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False, num_workers=2)

# --- Modello A: ResNet18 pre-addestrato ---
def crea_resnet_transfer():
    m = models.resnet18(weights="DEFAULT")
    for p in m.parameters(): p.requires_grad = False # congela tutto
    m.fc = nn.Sequential(nn.Linear(512, 256), nn.ReLU(), nn.Dropout(0.3),
nn.Linear(256, 10))
    return m.to(device)

# --- Sblocco progressivo dopo 10 epoche (punto 2) ---
def sblocca_layer4(modello, lr_fine=1e-4):
    for p in modello.layer4.parameters():
        p.requires_grad = True
    # Crea un nuovo ottimizzatore che includerà i parametri appena sbloccati
    return torch.optim.Adam(
        filter(lambda p: p.requires_grad, modello.parameters()),
        lr=lr_fine
    )

# Risultati tipici attesi:
```

```
# ResNet18 (solo head, 15 epoche): 85-88%
# ResNet18 + sblocco layer4 (5 epoche extra): 88-91% (+2-3%)
# CNN da zero su STL-10: 60-70% (dataset piccolo = difficile)

# La differenza drammatica CNN-da-zero vs transfer learning dimostra
# il valore dei feature generali imparati su ImageNet
```

Risposta alla domanda 3 (quando il pre-addestrato raggiunge il massimo della CNN da zero): Tipicamente entro le prime 3–5 epoche. Il pre-addestrato parte già da un punto molto buono grazie alle feature di ImageNet, mentre la CNN da zero deve imparare tutto dai pochi dati di STL-10.

Risposta alla domanda 4 (immagini sistematicamente difficili): Le classi più confuse su STL-10 sono tipicamente:

- Cervo e Cavallo: forma del corpo simile
- Auto e Camion: difficile con 96×96 senza zoom
- Uccello e Aeroplano: forma a "ali" simile in certi angoli

Capitolo 10 — Sequenze: RNN e LSTM

Esercizio finale — Sentiment IMDB con LSTM vs GRU

```
import torch, torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from datasets import load_dataset
from collections import Counter
import numpy as np

# Caricamento e preparazione
dataset = load_dataset("imdb")

# Costruzione vocabolario
def build_vocab(texts, max_vocab=20000):
    counter = Counter()
    for text in texts:
        counter.update(text.lower().split())
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, _ in counter.most_common(max_vocab - 2):
        vocab[word] = len(vocab)
    return vocab

train_texts = [x["text"] for x in dataset["train"]]
vocab = build_vocab(train_texts)

MAX_LEN = 256

class IMDBDataset(Dataset):
    def __init__(self, split):
        self.data = [(x["text"], x["label"]) for x in dataset[split]]
    def __len__(self): return len(self.data)
    def __getitem__(self, i):
        text, label = self.data[i]
        ids = [vocab.get(w, 1) for w in text.lower().split()][:MAX_LEN]
```



```

        ids += [0] * (MAX_LEN - len(ids))
        return torch.tensor(ids), torch.tensor(label)

train_ds = IMDBDataset("train")
test_ds = IMDBDataset("test")
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32)

device = "cuda" if torch.cuda.is_available() else "cpu"
EMBED_DIM = 128
VOCAB_SIZE = len(vocab)

# --- Modello A: LSTM ---
class LSTMSentiment(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(VOCAB_SIZE, EMBED_DIM, padding_idx=0)
        self.lstm = nn.LSTM(EMBED_DIM, 128, num_layers=2,
                             batch_first=True, dropout=0.3)
        self.fc = nn.Linear(128, 2)
    def forward(self, x):
        emb = self.embed(x)
        out, (h, _) = self.lstm(emb)
        # Punto 3: prova sia "ultimo passo" che "media"
        # Ultimo passo: h[-1] ← migliore per LSTM (cattura contesto finale)
        # Media: out.mean(dim=1) ← spesso simile o leggermente peggio
        return self.fc(h[-1])

# --- Modello B: GRU bidirezionale ---
class GRUSentiment(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(VOCAB_SIZE, EMBED_DIM, padding_idx=0)
        self.gru = nn.GRU(EMBED_DIM, 64, num_layers=2,
                           batch_first=True, dropout=0.3, bidirectional=True)
        self.fc = nn.Linear(64 * 2, 2) # *2 per bidirezionale
    def forward(self, x):
        emb = self.embed(x)
        out, h = self.gru(emb)
        # Concatena ultimo hidden state forward e backward
        h_cat = torch.cat([h[-2], h[-1]], dim=1)
        return self.fc(h_cat)

# Risultati attesi dopo 5 epoche:
# LSTM (ultimo passo): ~88-90%
# LSTM (media): ~87-89%
# GRU bidirezionale: ~89-91% — spesso leggermente migliore
# Tempo training GRU: ~20-30% più veloce di LSTM

```

Risposta domanda 3 (ultimo passo vs media): Per la classificazione del sentiment, l'ultimo passo è generalmente migliore perché cattura il contesto finale della recensione dove spesso si trova il giudizio complessivo ("...nel complesso deludente."). La media può essere utile quando l'informazione è distribuita uniformemente nella sequenza.

Risposta domanda 4 (esempi con predizioni diverse): Le recensioni ambigue di solito iniziano con aspetti positivi e finiscono negativi (o viceversa). L'LSTM con "ultimo passo" tende a pesare di più il finale, la GRU bidirezionale considera entrambe le direzioni. Su recensioni equilibrate la GRU è spesso più accurata.

Risposta domanda 5 (lunghezza 512 vs 256): Il tempo di training circa raddoppia. Il motivo è che la LSTM è $O(T)$ nella lunghezza della sequenza: ogni token deve essere processato in sequenza. Raddoppiare la lunghezza raddoppia il numero di passi. Nei Transformer (Cap. 11) il costo è $O(T^2)$ — ancora peggio per sequenze lunghe.

Capitolo 11 — Il Transformer

Esercizio finale — Decoder autoregressivo per generazione testo

```
import torch, torch.nn as nn, math
import urllib.request

# Scarica La Divina Commedia
url = "https://www.gutenberg.org/cache/epub/1012/pg1012.txt"
try:
    testo = urllib.request.urlopen(url).read().decode("utf-8")
    testo = testo[2000:200000] # rimuovi header e taglia a 200K caratteri
except:
    testo = "Nel mezzo del cammin di nostra vita mi ritrovai per una selva oscura"

# Vocabolario a livello di carattere
chars = sorted(set(testo))
ch2id = {c: i for i, c in enumerate(chars)}
id2ch = {i: c for i, c in enumerate(chars)}
VOCAB = len(chars)

# Causal mask
def causal_mask(seq_len, device):
    """Matrice triangolare inferiore: il token i può vedere solo j ≤ i."""
    mask = torch.tril(torch.ones(seq_len, seq_len, device=device))
    return mask.unsqueeze(0).unsqueeze(0)

print("Verifica causal mask 5×5:")
print(causal_mask(5, "cpu").squeeze())
# Output atteso:
# [[1, 0, 0, 0, 0],
#  [1, 1, 0, 0, 0],
#  [1, 1, 1, 0, 0],
#  [1, 1, 1, 1, 0],
#  [1, 1, 1, 1, 1]]

# Funzione di generazione
def genera(modello, tokenizer_ch2id, tokenizer_id2ch, seed, n_caratteri,
temperatura=1.0):
    modello.eval()
    device = next(modello.parameters()).device
    contesto = [tokenizer_ch2id.get(c, 0) for c in seed]
    MAX_CTX = 128 # lunghezza massima del contesto

    risultato = list(seed)
    with torch.no_grad():
        for _ in range(n_caratteri):
            ctx = contesto[-MAX_CTX:] # tronca al contesto massimo
            x = torch.tensor(ctx, dtype=torch.long, device=device)
            logits = modello(x) # (1, len_ctx, vocab)
```

```

logits_ultimo = logits[0, -1, :] # logit dell'ultimo token

# Temperature sampling
logits_scaled = logits_ultimo / temperatura
probs = torch.softmax(logits_scaled, dim=-1)
prossimo = torch.multinomial(probs, num_samples=1).item()

contesto.append(prossimo)
risultato.append(tokenizer_id2ch[prossimo])

return "".join(risultato)

# Risultati attesi con temperature diverse:
# temperatura=0.3: testo più "prevedibile", ripete pattern frequenti
#                  "nel mezzo del cammin nel mezzo del cammin..."
# temperatura=0.8: buon bilanciamento, testo comprensibile ma vario
# temperatura=1.2: più creativo ma inizia a perdere coerenza
#                  parole inventate, senso meno chiaro

```

Risposta domanda 4 (temperature diverse): La coerenza è massima attorno a $T=0.5-0.8$. Con $T<0.3$ il testo diventa ripetitivo e meccanico. Con $T>1.3$ iniziano ad apparire caratteri improbabili e parole non esistenti.

Risposta domanda 5 (5 vs 20 epoche): Dopo 5 epoche il modello produce caratteri validi ma raramente parole complete. Dopo 20 epoche inizia a produrre parole italiane riconoscibili e strutture sintattiche. I layer iniziali imparano prima la struttura di carattere (lettera seguita da lettera), i layer successivi le strutture morfologiche e sintattiche. Questo riflette la gerarchia delle rappresentazioni nelle reti neurali profonde.

Capitolo 12 — LLM

Esercizio finale — Chatbot con memoria persistente e auto-valutazione

```

import anthropic, json, os
from datetime import datetime

client = anthropic.Anthropic()
MODELLO = "claude-sonnet-4-20250514"

# --- Memoria persistente ---
def carica_storia(file="storia_chat.json"):
    if os.path.exists(file):
        with open(file) as f: return json.load(f)
    return []

def salva_storia(storia, file="storia_chat.json"):
    with open(file, "w") as f:
        json.dump(storia, f, ensure_ascii=False, indent=2)

# --- Valutatore automatico (punto 3) ---
def valuta_risposta_auto(domanda: str, risposta: str) -> dict:
    """Chiede a Claude di valutare la propria risposta su 3 criteri."""
    r = client.messages.create(
        model="claude-haiku-4-5-20251001", # modello veloce per la valutazione

```

```

        max_tokens=200,
        messages=[{"role": "user", "content": f""Valuta questa risposta su una scala
1-5.

Domanda: {domanda}
Risposta: {risposta}

Criteri: accuratezza, completezza, chiarezza

Rispondi SOLO con JSON: {"accuratezza": N, "completezza": N, "chiarezza": N,
"media": N}""}]
    )
    try:
        return
    json.loads(r.content[0].text.replace("` ` `json", "").replace("` ` `", "").strip())
    except:
        return {"accuratezza": 0, "completezza": 0, "chiarezza": 0, "media": 0}

# --- Loop principale ---
SYSTEM = ""Sei un esperto di [inserisci il tuo dominio].
Rispondi sempre in italiano con tono professionale.
Se non sei sicuro di qualcosa, dillo esplicitamente.""

storia = carica_storia()
punteggi = []

print("Chatbot avviato. Scrivi 'esci' per terminare, 'statistiche' per i
punteggi.\n")
while True:
    domanda = input("Tu: ").strip()
    if domanda.lower() == "esci":
        salva_storia(storia)
        print("Storia salvata. Arrivederci!")
        break
    if domanda.lower() == "statistiche":
        if punteggi:
            media = sum(p["media"] for p in punteggi) / len(punteggi)
            print(f"Punteggio medio sessione: {media:.2f}/5 ({len(punteggi)}
risposte)")
        continue

    storia.append({"role": "user", "content": domanda})
    r = client.messages.create(
        model=MODELLO, max_tokens=1024, system=SYSTEM, messages=storia
    )
    risposta_testo = r.content[0].text
    storia.append({"role": "assistant", "content": risposta_testo})

    # Tronca la storia per non superare il context limit
    if len(storia) > 20:
        storia = storia[-20:]

    # Valutazione automatica
    val = valuta_risposta_auto(domanda, risposta_testo)
    punteggi.append(val)

    print(f"\nAssistente: {risposta_testo}")

```

```
print(f"[Qualità: A={val['accuratezza']}/5 C={val['completezza']}/5  
Ch={val['chiarezza']}/5 | Media={val['media']}/5]\n")
```

Risposta domanda 4 (analisi punteggi bassi): Le risposte con punteggi bassi di solito cadono in queste categorie:

- Domande molto specifiche di dominio (conoscenza limitata del modello)
- Domande ambigue senza contesto sufficiente
- Domande che richiedono dati numerici precisi (il modello tende all'imprecisione)

Risposta domanda 5 (chain-of-thought): Aggiungere "Pensa step by step prima di rispondere" migliora quasi sempre le risposte su domande che richiedono ragionamento. I punteggi di "completezza" e "accuratezza" tendono a salire di 0.3–0.7 punti in media. Non cambia molto per domande fattuali semplici.

Capitolo 13 — Prompt Engineering

Esercizio finale — Sistema A/B testing prompt

Guida ai risultati attesi:

Per la **classificazione ticket di supporto** (esempio tipico):

| Versione | Accuracy attesa | Note | |---|---|---| | v1 — zero-shot | 75–82% | Buona per ticket chiari, fallisce sugli ambigui | | v2 — few-shot (3 esempi) | 83–89% | Migliora sulla consistenza del formato | | v3 — definizioni + CoT | 85–91% | Migliore sugli edge case, più token usati | | v4 — corretta per errori | 87–92% | Dipende da come sono stati analizzati gli errori |

Risposta domanda 4 (LLM-as-Judge vs giudizio umano): La concordanza tipica è del 75–85% sui casi chiari (entrambi danno 4–5 o 1–2). La discordanza appare sui casi "borderline" (il modello dà 3, tu daresti 4 o viceversa). Il modello tende al sycophancy: dare punteggi più alti quando la risposta è lunga e scorrevole, anche se manca di precisione.

Risposta domanda 5 (complessità del prompt): No, la versione più complessa non è necessariamente la migliore. Per task di classificazione con categorie chiare, few-shot con 3 esempi spesso supera un prompt elaborato con definizioni e CoT. La complessità aggiuntiva aiuta principalmente per task ambigui o quando il modello deve gestire casi edge numerosi. Il principio: usa il minimo di complessità che supera la tua soglia di qualità.

Capitolo 14 — RAG

Esercizio finale — Q&A su knowledge base con metriche

Risposta domanda 5 (retrieval vs generazione: come diagnosticare):

```
def diagnosi_rag(domanda, risposta_sistema, chunks_recuperati, riferimento):  
    """  
    Diagnostica se l'errore è nel retrieval o nella generazione.  
    """  
    import anthropic  
    client = anthropic.Anthropic()
```

```

# Step 1: verifica se i chunk giusti erano tra quelli recuperati
r1 = client.messages.create(
    model="claude-haiku-4-5-20251001", max_tokens=100,
    messages=[{"role": "user", "content": f""La risposta corretta a questa
domanda è:
"{riferimento}"

Questi chunk contengono informazioni sufficienti per rispondere?
{chr(10).join(chunks_recuperati[:3])}]

Rispondi: SI o NO e in 10 parole il motivo."""]
)
retrieval_ok = "SI" in r1.content[0].text.upper()

# Step 2: se il retrieval era ok, il problema è nella generazione
if retrieval_ok:
    diagnosi = "PROBLEMA DI GENERAZIONE: i chunk giusti c'erano ma il modello non
li ha usati bene"
else:
    diagnosi = "PROBLEMA DI RETRIEVAL: i chunk rilevanti non sono stati
recuperati"

return diagnosi, retrieval_ok

```

Risultati tipici:

- ~40% dei casi: problema di retrieval (i chunk giusti non erano nel top-k)
- ~35% dei casi: problema di generazione (i chunk c'erano ma la risposta era sbagliata)
- ~25% dei casi: la domanda era fuori scope del knowledge base

Il re-ranking aiuta soprattutto nel caso del retrieval: il chunk giusto era al rank 8, il re-ranker lo porta al rank 2.

Capitolo 15 — Agenti IA

Esercizio finale — Agente analista di mercato

Risposta domanda 4 (coerenza dei report): I report su aziende grandi e note (Apple, Google, Tesla) sono molto più coerenti e completi di quelli su aziende di nicchia o regionali. L'agente ha difficoltà con:

- Informazioni finanziarie precise e recenti (i dati cambiano continuamente)
- Aziende non anglofone con poca presenza sui siti in inglese
- Distinzione tra informazioni di ieri e di 2 anni fa

Risposta domanda 5 (informazioni inventate): La fase più rischiosa è la **sintesi**: l'agente riceve 5 snippet da 5 fonti diverse, li combina, e può "interpolare" dettagli che non erano esplicitamente in nessuna fonte. Il retrieval grezzo è solitamente fedele, ma la sintesi introduce allucinazioni sottili.

Come aggiungere verifica delle fonti:

```

def verifica_citazione(affermazione: str, fonti: list[str]) -> dict:
    ""Verifica che ogni affermazione sia supportata da almeno una fonte.""

```

```

client = anthropic.Anthropic()
fonti_str = "\n---\n".join(fonti[:3])
r = client.messages.create(
    model="claude-haiku-4-5-20251001", max_tokens=150,
    messages=[{"role": "user", "content": f""Questa affermazione è supportata
dalle fonti?

Affermazione: {affermazione}
Fonti: {fonti_str}

Rispondi con JSON: {{"supportata": true/false, "fonte_numero": N_o_null}}""}]
)
return json.loads(r.content[0].text.replace("`json", "").replace("`", ""))

```

Capitolo 16 — MCP

Esercizio finale — Server MCP completo

Risposta domanda 5 (migliorare le descrizioni dei tool):

Le descrizioni che guidano meglio il modello hanno queste caratteristiche:

```

# ✗ Descrizione vaga
@mcp.tool()
def leggi(percorso: str) -> str:
    """Legge un file.""" # Troppo vago – quando usarlo? con quali file?
    ...

# ✓ Descrizione precisa con contesto d'uso
@mcp.tool()
def leggi_documento(nome_file: str) -> str:
    """
    Legge il contenuto di un documento dalla directory dei dati.

    Usa questo tool quando l'utente chiede di vedere o analizzare il contenuto
    di un file specifico. Fornisci solo il nome del file (es. 'report.txt'),
    non il percorso completo.

    Chiama lista_documenti() prima se non sai quali file sono disponibili.

    Args:
        nome_file: nome del file da leggere (solo nome, non percorso)
    Returns:
        contenuto del file come stringa, o messaggio di errore
    """
    ...

```

Il modello usa le docstring per decidere QUANDO e COME chiamare i tool. Più la descrizione è specifica su "quando usarlo" e "cosa aspettarsi in ritorno", meno errori di uso si vedono.

Capitolo 17 — Hardware e CUDA

Esercizio finale — Profilazione e ottimizzazione VRAM

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def profila_modello(nome_modello, precision=torch.float16):
    """Misura VRAM usata da un modello con batch size crescenti."""
    torch.cuda.empty_cache()

    modello = AutoModelForCausalLM.from_pretrained(
        nome_modello, torch_dtype=precision, device_map="cuda"
    )
    tokenizer = AutoTokenizer.from_pretrained(nome_modello)

    risultati = []
    for batch_size in [1, 2, 4, 8]:
        try:
            torch.cuda.empty_cache()
            vram_pre = torch.cuda.memory_allocated() / 1e9

            # Forward pass
            input_ids = torch.randint(0, 1000, (batch_size, 512)).cuda()
            with torch.no_grad():
                _ = modello(input_ids)

            vram_post = torch.cuda.memory_allocated() / 1e9
            risultati.append({
                "batch_size": batch_size,
                "vram_gb": round(vram_post, 2),
                "incremento_gb": round(vram_post - vram_pre, 2)
            })
        except torch.cuda.OutOfMemoryError:
            risultati.append({"batch_size": batch_size, "vram_gb": "OOM"})
            break

    return risultati

# Trucchi per ridurre l'uso di VRAM:
# 1. Usa torch.no_grad() durante l'inferenza (risparmia memoria per i gradienti)
# 2. Svuota la cache tra le chiamate: torch.cuda.empty_cache()
# 3. Processa in batch piccoli e accumula i risultati
# 4. Usa BF16 invece di FP32 (dimezza la VRAM dei pesi)
# 5. Usa 4-bit quantization (QLoRA) per ridurre ulteriormente
```

Progetto di Fine Volume 2 — Chatbot RAG con Agente

Guida alla valutazione del progetto

Checklist di completamento:

- [] Il sistema risponde correttamente a domande sul knowledge base
- [] L'agente cerca sul web quando la domanda è fuori dal KB


```
(test: chiedi una notizia recente)
[ ] Il server MCP custom espone almeno 2 tool funzionanti
[ ] Le fonti vengono citate nelle risposte
[ ] Il sistema gestisce gracefully le domande fuori scope
    ("Non ho informazioni su questo argomento")
[ ] La latenza P50 è < 10 secondi su una GPU standard
```

Problemi comuni e soluzioni:

| Problema | Causa probabile | Soluzione | |---|---|---| | Il RAG non trova i chunk giusti | Chunking troppo grande o piccolo | Prova dim=400 con overlap=80 | | L'agente cerca sempre sul web invece di usare il RAG | Soglia di distanza troppo alta | Abbassa `soglia_distanza` da 0.8 a 0.6 | | Il modello allucinante nonostante il contesto | System prompt non abbastanza esplicito | Aggiungi "Rispondi SOLO con le informazioni nel contesto" | | Loop infiniti dell'agente | Tool che restituisce errori ripetuti | Implementa `chiamate_viste` per bloccare duplicati |

Fine delle soluzioni del Volume 2. Le soluzioni del Volume 1 si trovano nell'Appendice E — Volume 1. Le soluzioni del Volume 3 si trovano nell'Appendice E — Volume 3.